MACOS Command Reference

Table of Contents

MACOS Command Reference....0
Companion document: the MACOS Manual....0
The four surfaces....1
Where things live....1
Related tools....2
Where the examples are....2
Prerequisite tags used throughout....2
Status of this document....2
CLI syntax conventions....2
SMACOS syntax conventions....3
mmacos syntax conventions (MATLAB)....4
pymacos syntax conventions (Python)....5
Part I — Engine command catalog....6
Part II — mmacos / pymacos function catalog....69
Session and user options: one default each....105
Part III — Higher-level MATLAB layers (current state)....108
Appendix — CLI-suite coverage of the engine commands....117
Appendix — binding coverage of the engine commands....117

MACOS Command Reference

One optical engine, four ways to drive it.

This document is the command-level reference for the MACOS optical modeling system and its programming interfaces: a catalog of every command and function, one at a time, with its syntax.

Companion document: the MACOS Manual

This reference is a lookup document; the **MACOS Manual** is the learning document. The Manual covers the concepts this catalog only names — geometry and coordinate conventions (VptElt, PsiElt, the global

frame), prescription structure, ray-grid and source setup, reference surfaces and diffraction modeling, linear optical models, and SMACOS integration — and works complete examples end to end (Appendix A pairs each example prescription with a journal file that drives a full analysis).

The Manual lives beside this document in the repo (`docs/macos-manual/`, built as `macosMan.pdf` — see the README there) and is referenced throughout this catalog as "manual §N". If you are new to MACOS: read Manual Sections 1–4 first, run one Appendix A example, and use this reference from then on.

The four surfaces

MACOS is a Fortran optical analysis engine: exact ray tracing, differential ray tracing (linear optical models), and diffraction propagation, built for *system-level* modeling — optics on dynamic structures, segmented and deformable mirrors, controlled optics — rather than lens design. There are four ways to use it:

1. **The `macos` CLI** — an interactive command interpreter. You type commands (about 140 of them; Part I of this reference) at the `MACOS>` prompt; the engine prompts for whatever arguments it needs. Sessions can be recorded to journal (`.jou`) files and replayed with `EXEcute`. This is the exploratory surface and the place to learn what the engine can do.

2. **SMACOS** — the same engine as a Fortran subroutine (`libsmacos.a`). Your program issues the *same command strings* programmatically and passes arguments through a fixed buffer list. This is the integration surface for custom Fortran drivers and is what every higher-level binding is built on. Same command catalog (Part I); calling convention in the SMACOS chapter.

3. **mmacos** — a MATLAB package (`+macos`) wrapping the engine through a single MEX gateway. Functions like `macos.load_rx`, `macos.trace`, `macos.opd`, `macos.perturb`, `macos.dw_dx` give MATLAB-native access (matrices in, matrices out) plus higher-level drivers the CLI does not have: multi-channel Jacobians, the CALIB optimizer interface, segment/Zernike grid-basis generators, and the parametric telescope **design layer** (`macos.design.*`). Parts II and III.

4. **pymacos** — a Python package wrapping the same engine via f2py, with numpy arrays and (mostly) the same function vocabulary as mmacos. Part II documents both side by side; where only one language has a function, the entry says so.

All four share one prescription format (`.in` files), one state model, and one set of units conventions, because underneath they are the same compiled engine. The bindings (3, 4) go through a common wrapper layer, `macos_f90/macos_api_mod.F90` — each Part II entry names the underlying routine, which is the ground truth for argument meanings.

Where things live

Piece	Location	Build
engine + CLI (`macos`)	`nasa-jpl/macos` repo, `macos_f90/`	`source ./makeall.sh` (cmake; gfortran or Intel ifx)
SMACOS library	same repo	built by `makeall.sh`/`makems.sh` (`libsmacos.a`)
mmacos	`MACOS_resources/mmacos/`	see its README (MEX build against libsmacos)
pymacos	`MACOS_resources/pymacos/`	see its README (f2py build)

Related tools

Two companions ship alongside the four surfaces:

- **SegMirMaker** (`MACOS_resources/segmirmaker/`) — standalone utility for converting a monolithic primary mirror into a many-segment one: it generates the segmented-mirror prescription (`.presc`) and segment-influence (`Hx.m`) files from a parent-mirror description, including FreeForm parents. Use it before hand-editing segment entries into a prescription. See manual §4.4.11.
- **GMI** (`MACOS_resources/GMI/`) — a MATLAB MEX front end built directly on SMACOS: a complete forward simulator (multi-channel sensitivities, detector images) in one gateway. Beyond being a tool in its own right, `GMI.F` is the reference example of a real SMACOS application — the minimal driver of the SMACOS chapter, grown to production size.

Where the examples are

Worked examples are not enumerated in this reference; they live with each surface:

- **CLI / engine** — `docs/macos-manual/examples/`: the manual's Appendix A prescriptions, each `<name>.in` paired with a `<name>.jou` journal that drives a complete analysis (`EXEcute <name>` after loading).
- **mmacos** — `MACOS_resources/mmacos/examples/`; further runnable demos sit in `mmacos/design/` (telescope design layer) and `mmacos/sensitivities/` (Jacobian / dw_* drivers).
- **pymacos** — `MACOS_resources/pymacos/tests/`: the test suite doubles as usage examples.

Prerequisite tags used throughout

- **[Rx]** — needs a loaded prescription (`OLD` or `NEW` first).
- **[BLD]** — needs a built linear model (`BUild` first).
- **[DIFF]** — needs a propagated wavefront (`PROpagate` first).

Status of this document

Part I and Part II skeletons are **generated from the engine and the binding sources** (`tools/gen_cmdref.py`), so the catalog cannot drift from the code; entries marked *TODO* are awaiting expanded prose. Part III (design layer, channels) documents packages that are under active development and may change faster than the rest.

CLI syntax conventions

Command names and minimum match

Command names are shown with a **capitalized prefix and lowercase tail**: `PERturb`, `SUMmarize`, `PROpagate`. The capitalized part is the *minimum-match abbreviation* — the dispatcher accepts any prefix at least that long, case-insensitively. So `per`, `pert`, and `perturb` all invoke PERturb. When two commands share a prefix the longer spelling wins disambiguation; when in doubt type the full name.

The dialog model

Most commands take no arguments on the command line. Instead the engine prompts for each input in turn, showing the expected form and often a default in brackets:

```
MACOS>perturb
Enter element number [1]: 3
```

```
Enter 6-vector perturbation:
 (dx dy dz rx ry rz): 0 0 1e-6 0 0 0
```

- Hitting return accepts the bracketed default.
- Vectors are entered space-separated on one line.

The *Syntax:* and *Defaults:* lines of Part I

Each engine entry's notes open with two lines (added 2026-10-17, written from the CLI suite's transcripts and the dispatcher):

- ***Syntax:*** the command with its arguments in the order of its prompts. Any leading run of them may be typed on the command line — the tokens of the line answer the prompts in order (`pix 5 64 2e-6` is `pix`, then `5` to the element prompt, `64` to the pixel count, `2e-6` to the pixel size) and the first prompt without an answer is asked. `[x]` has a default (return takes it), `<x>` has none.
- ***Defaults:*** where each default comes from — the deck, the element or value the last command used, a session option (`25_defaults.md`), or a literal.
- Fortran-style numbers are accepted: `1e-6`, `1d-6`, `0.25`, `1d22`.
- `Ctrl-C` at the `MACOS>` prompt exits cleanly.

Sessions, journals, prescriptions

A typical first session:

```
$ macos
MACOS model size: 256
MACOS>old
Enter file name: Cassegrain
MACOS>summarize
MACOS>spot
 ...
MACOS>quit
```

- The **model size** prompt (before the first `MACOS>`) sizes the runtime arrays; 256 is a good default for the bundled examples, and Return takes it. The answer must be one of 128, 256, 512, 1024, 2048, 4096, 8192: anything else is refused, naming it, and asked again; five refusals, or end of input, stop MACOS with exit status 1.
- `OLD` loads an existing `.in` prescription (the `.in` extension is implied); `NEW` builds one interactively; `SAVe` writes the current system back out.
- `JOUrnal` starts recording your commands to a `.jou` file; `EXEcute` replays one. The examples directory pairs each `<name>.in` with a `<name>.jou` that drives a complete analysis.
- `VALidate` checks a prescription file for structural errors.

Reading Part I entries

Each entry gives the command name (with minimum match), prerequisite tags, a one-line effect, and — where expanded — the dialog, behavior notes, and related commands. Every entry applies equally to SMACOS (next chapter): the command string is the same; only the argument delivery differs.

SMACOS syntax conventions

SMACOS is the engine as a callable Fortran subroutine. One entry point executes any Part I command:

```
CALL SMACOS(command, CARG, DARG, IARG, LARG, RARG,
&           OPDMat, RaySpot, RMSWFE, PixArray)
```

- `command` — `CHARACTER(LEN=132)`, the command string (same names and minimum-match rules as the CLI).

- `CARG(9)` — `CHARACTER(LEN=32)` arguments (file names, element names, option keywords).
- `DARG(9)` — `REAL*8` arguments; `IARG(9)` — `INTEGER`; `LARG` — `LOGICAL`; `RARG(9)` — `REAL*4`.
- `OPDMat(mpts,mpts)`, `RaySpot(mRay,2)`, `RMSWFE`, `PixArray(mPix,mPix)` — result buffers the engine fills for output commands (OPD, SPOT, PIXel, …).

Arguments land in the same order the CLI would have prompted for them: where the CLI asks "Enter prescription file name", SMACOS expects it in `CARG(1)`; numeric prompts map to `DARG`/`IARG` slots in prompt order. The manual's Section 9 documents the mapping for the commonly-scripted commands, and the CLI dialog itself (run the command interactively once) is the reliable way to see what a command wants.

Before the first call the runtime must be initialized with the model size — the programmatic equivalent of the CLI's startup prompt:

```
call macos_init_all(256)
macos_init = .true.
```

A complete minimal driver (load, trace, OPD RMS) is listed in manual Section 9.5; `smacos_dvr` in the build tree is a ready-made interactive driver linked against `libsmacos.a`, useful for testing call sequences before embedding them.

Two practical notes:

- SMACOS shares no state with any other copy of the engine in the process; sizes fixed at `macos_init_all` time bound all later work.
- Commands that would page output or draw graphics in the CLI run silently or write files instead; result data comes back through the buffer arguments.

The higher-level bindings (mmacos, pymacos) wrap this same entry point plus the finer-grained routines of `macos_api_mod.F90`; if you are writing new Fortran, consider calling `macos_api_mod` directly — it is the supported programmatic surface and each Part II entry names the routine involved.

mmacos syntax conventions (MATLAB)

mmacos is a MATLAB package: everything lives in the `macos.` namespace (the `+macos` package directory), backed by one MEX gateway (`mmacos.mexa64`) linked against the engine.

Setup

Build the engine (`makeall.sh` or `makegfortran.sh`), then build the mex per the mmacos README, and put the mmacos `src/` directory on the MATLAB path. Then:

```
macos.init(256);                    % model size, once per session
macos.load_rx('Cassegrain.in');
macos.trace();
W = macos.opd();                    % N x N OPD matrix
```

Conventions

- **Everything numeric is `double`.** The MEX dispatcher reads all numeric inputs as REAL(8); pass `double(...)` values — never `int32` or `logical` types. Element indices are 1-based doubles.
- **Getter/setter pairs.** Element and source properties come as `get_*`/`set_*` pairs (`get_elt_vpt`/`set_elt_vpt`, …). Setters write engine state directly; for rigid-body moves of real optics prefer `macos.perturb`, which keeps linked coordinate frames consistent.
- **Trace state.** Output functions (`opd`, `spot`, `intensity`, …) read the *most recent trace*; call `macos.trace()` (or a driver that traces internally) after any change. Setters mark the state dirty so stale results are not silently reused.

- **Units** follow the loaded prescription's BaseUnits/WaveUnits; `macos.sys_units` reports them, `macos.cbm` gives the base-unit → meter conversion.
- **Errors**: functions raise MATLAB errors with the engine message; most also return an `ok` flag when called with output arguments.
- `macos.Session` provides an object wrapper (load/trace/query in one handle) if you prefer OO style; the plain functions are equivalent.

Higher-level drivers

Beyond one-call wrappers, mmacos adds drivers with no CLI equivalent — documented in Parts II and III:

- `macos.dw_dx`, `dw_dz_zernike`, `dw_dsurf`, `dw_dgrid` (+ `_multi` variants): wavefront Jacobians over perturbation channels.
- `macos.calib*`: the native CALIB optimizer interface.
- `macos.segment_grid_basis`, `gs_zernike_segment_basis`, `write_grid_file`: segment/Zernike grid-basis generation.
- `macos.design.*`: parametric telescope construction and wavefront solving (Part III).
- `macos.channels.*`: the perturbation-channel classes the Jacobian drivers consume (Part III).

Help is embedded: `help macos.opd`, `help macos.dw_dx`, etc.

pymacos syntax conventions (Python)

pymacos is a Python package wrapping the same engine through an f2py-compiled extension. It works with numpy throughout.

Setup

Build per the pymacos README (compiles the engine sources into the `pymacosf90` extension), then:

```
import pymacos

pymacos.init(256)                    # model size, once per session
pymacos.load('Cassegrain.in')
pymacos.trace_rays(-1)               # trace to the last surface
W = pymacos.opd()                    # numpy (N, N) OPD array
```

Conventions

- **numpy in, numpy out.** Vectors and matrices are numpy arrays; element indices are 1-based ints, and many functions accept an array of element IDs to operate on several at once.
- **Negative element indices** count from the end of the train: `-1` is the last surface (image), `-2` the one before it — handy for "trace to the exit pupil" idioms like `trace_rays(-2)`.
- **Combined getter/setters.** Where mmacos has `get_x`/`set_x` pairs, pymacos usually has one function whose optional argument selects the mode: `elt_vpt(srf)` reads, `elt_vpt(srf, vpt)` writes. Part II entries list both languages' forms together.
- **Naming**: snake_case primaries with some camelCase aliases kept for backward compatibility (`traceWavefront → trace_rays`); use the snake_case names in new code.
- **Trace state** works as in mmacos: outputs reflect the most recent trace; re-trace after changing the system.
- **Errors** raise Python exceptions carrying the engine message.
- Docstrings are complete — `help(pymacos.trace_rays)` gives the full argument contract, including shapes and ranges.

Coverage relative to mmacos

Part II flags per-entry gaps. As of this writing the multi-channel Jacobian drivers, CALIB interface, grid-basis generators, and the design layer exist on the MATLAB side only; pymacos covers the full session/source/element/trace/output core plus the FreeForm and Zernike surface interfaces.

Part I — Engine command catalog

Commands below are available from the interactive CLI and, with few exceptions, from a `CALL SMACOS(command, ...)` invocation (see the SMACOS chapter for the argument convention; purely interactive conveniences — shell/file utilities like `!`, PWD, CD, RX/LS/LL, VI/EMAcs, and a few others noted per entry — exist only in the CLI build). The **capitalized prefix** of each name is the minimum-match abbreviation. Prerequisite tags: [Rx] needs a loaded prescription (OLD/NEW), [BLD] a built linear model (BUILD), [DIFF] a diffraction output (the command itself propagates the wavefront; there is no separate PROpagate command).

Session & Files

Quit, EXIT, BYE

Minimum match: `Quit`

Exit MACOS.

CLI suite: `ATMOS_DMBUILD.jou`, `BEAM_gaussian.jou`, `BINARY_CENTER.jou`, `BUILD_refused_export.jou`, `EXPORT.jou`, `GLASS_mod_save.jou`, `LINEAR_MODEL.jou`, `MOD_spaces.jou`, `ORS_SRS.jou`, `PARTIALS_inputray.jou`, `ZERN_mode_count.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — session / shell convenience.

Syntax: `QUIT` (also `EXIT`, `BYE`); no prompt.

Defaults: none; the journal file, if one is open, is closed.

No dialog. Closes the graphics device and any open output and journal files, then stops the process. The dispatcher accepts a single `Q`; EXIT and BYE need all their letters. Under SMACOS only QUIT is matched, and it RETURNs control to the calling program instead of stopping — like the RETURN command, plus the file/graphics cleanup. *Related:* JOUrnal, EXEcute.

HELP

Minimum match: `HELP`

This command list.

CLI suite: `HELP_POL.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — session / shell convenience.

Syntax: `HELP`; pages with "Hit return to continue… [0]".

Defaults: the paging prompt takes return (0 = continue).

```
MACOS>help
 MACOS command summary
 ...
 Hit return to continue... [0]:
```

Paginated: hit return at each `Hit return to continue...` prompt. Lists every dispatched command in 15 categories, with the minimum-match casing and the [Rx]/[BLD]/[DIFF] prerequisite tags. The text is `macos_help.inc` — the same single source this catalog is generated from. The dispatcher accepts `HEL`. *Related:* STatus.

REset

Minimum match: `REset`

Reset options & defaults to startup.

CLI suite: `SESSION_INFO.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — `init` (re-allocation restores every startup default).

Syntax: `RESET`; no prompt.

Defaults: none. Resets the trace state, the obscuration option to 1 and the plot type to GRAY (its start-up value; 1.2 (i), 2026-10-22).

No dialog. Resets analysis state and options to startup defaults: clears the trace/build/propagate/perturb flags, the obscuration option, the stretch type, the surface-plot type (back to GRAY, whatever NONE/SLICE/… set), beam type, and pixel settings. The loaded prescription is NOT unloaded, and a STOP that was preset in the Rx is preserved. Suite: `SESSION_INFO.jou`. *Related:* MREset, STatus, LREset.

STatus

Minimum match: `STatus`

What is loaded / built / propagated.

CLI suite: `HELP_POL.jou`, `MODE_centroid.jou`, `MODE_cir.jou`, `MODE_nolneg.jou`, `MODE_polarization.jou`, `MODE_scalar.jou`, `MRESET.jou`, `SESSION_INFO.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — session / shell convenience.

Syntax: `STATUS`; no prompt.

Defaults: none.

Needs a loaded Rx. Prints one screen of session state: the loaded file name, current element for OPD calculations, current element for WF/spot calculations, the ray-trace obscuration option, the current WF plot type (SLICE/GRAY/CONTOUR/…), and the composed-image, pixel-locate and polarization flags. No side effects. *Related:* SUMmarize, ELTS, REset.

SUMmarize

Minimum match: `SUMmarize`

Pretty-print the optical system.

CLI suite: `SESSION_INFO.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — session / shell convenience.

Syntax: `SUM`; no prompt.

Defaults: none.

```
MACOS>summarize
 Optical Train Summary:
  Tracing rays past   4 elements.
  Aperture grid type = Circular  with  513 grid points.
  Elt   1: Reflector   : Conic    with Kc=-1.0000D+00, ...
```

One line per element: element type, surface type, conic constant KcElt, vertex radius KrElt, and nECoord; ends with the number of output coordinates. The same summary is printed automatically after every successful OLD/NEW load. *Related:* ELTS, SHOw, STatus.

ELTS

Minimum match: `ELTS`

List elements (type, surface).

CLI suite: `SESSION_INFO.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — Zernike / analysis set (B.2, second).

Syntax: `ELTS`; no prompt.

Defaults: none.

```
MACOS>elts
 Elements in optical prescription:
 Element name = PM          ID =   1
 Element name = SM          ID =   2
```

Needs a loaded Rx. Quick way to map element names to the numeric element ids used by SHOw, PERturb, COOrd, etc. *Related:* SHOw, SUMmarize.

EXEcute

Minimum match: `EXEcute`

Run a .jou journal file.

CLI suite: `CALIB_journal_prompts.jou`, `JOURNAL_EXE.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — session / shell convenience.

Syntax: `EXE <file>` — the journal file name, no default.

Defaults: none for EXE itself; the prompts inside the journal take the answers the journal holds, and any prompt the journal leaves unanswered takes its own default (an exhausted input stack reads as return).

```
MACOS>execute
Enter file name: run1
Reading from journal file (run1.jou)
```

`.jou` is appended automatically. Commands are read until end of file; journal files may EXEcute other journal files (up to 10 deep). `%` comments and blank lines are ignored. `LQW` is an accepted alias. A missing file prints a warning and returns to the prompt. SMACOS: file name in CARG(1). *Related:* JOUrnal, ECHo.

JOUrnal

Minimum match: `JOUrnal`

Generate a .jou journal file.

CLI suite: `JOURNAL_EXE.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — session / shell convenience.

Syntax: `JOU <file>` opens a journal; a second `JOU` with no name closes it.

Defaults: the file name has no default while opening; closing asks nothing.

```
MACOS>journal
Enter journal file name: run1
Writing journal file run1.jou...
 ...
MACOS>journal
Closing journal file run1.jou.
```

A toggle: the first call starts recording every command (an existing file of the same name is deleted first), the second call stops. Defaults accepted by hitting return are written to the journal too, so a replay reproduces the session. *Related:* EXEcute, ECHo.

ECHo

Minimum match: `ECHo <text>`

Print to stdout (annotate console / journal output).

CLI suite: `SESSION_INFO.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — session / shell convenience.

Syntax: `ECHO <text>` — the rest of the line is printed.

Defaults: none.

```
MACOS>echo starting perturbation study
starting perturbation study
```

Prints the rest of the line verbatim; a bare `echo` prints a blank line. Remaining tokens on the line are consumed so they are not re-dispatched as commands. Useful for marking phases in journal-driven runs. *Related:* JOUrnal, EXEcute.

MREset

Minimum match: `MREset`

Reset MACOS model size (128..8192).

CLI suite: `MRESET.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — `init(model_size)`.

Syntax: `MRESET [size]` — one of 128, 256, 512, 1024, 2048, 4096, 8192.

Defaults: size [128]: the literal 128, not the size in force (the prompt is the dialog's, not the session's).

```
MACOS>mreset
Enter new size (128, 256, 512, 1024, 2048, 4096, 8192): [128]: 512
 Resetting MACOS model size...
```

The size may be given inline (`mreset 512`); invalid sizes re-prompt. Re-allocates all model arrays, reprints the run-limits banner, reloads the glass table, and re-initializes session state INCLUDING the loaded-prescription flag — re-load the Rx (OLD) afterwards. Interactive CLI only (bindings size the model at init). *Related:* REset, OLD.

!

Minimum match: `! <cmd>`

Run a shell command.

CLI suite: `ATMOS_placement.jou`, `ATMOS_seed.jou`, `BINARY_CENTER.jou`, `BWK.jou`, `CALIB_SPOT_mixed.jou`, `CALIB_failed_trials.jou`, `CALIB_fov_weight.jou`, `CALIB_journal_prompts.jou`, `CALIB_save_roundtrip.jou`, `COAT_legacy.jou`, `COORD_ACOOR.jou`, `EMACS_VI.jou`, `EXPORT.jou`, `FILTERS_MULTISPEC.jou`, `FOV_SETC_SAOPT.jou`, `GLASS_mod_save.jou`, `GRIDFILE_beside.jou`, `IMG.jou`, `LINEAR_MODEL.jou`, `LOH_tokens.jou`, `METCALC.jou`, `NEW.jou`, `OBS_POLYGON.jou`, `ORS_SRS.jou`, `OUTCORD_missing.jou`, `PGD_ODRAW.jou`, `PGD_reopen_units.jou`, `PLOCATE_NOPLOC.jou`, `PLOT_TYPES.jou`, `PREAD.jou`, `PROPTYPE_numeric.jou`, `PWIN.jou`, `SHELL_LONG.jou`, `SINT.jou`, `SPCENTER.jou`, `STARTUP_size.jou`, `WINDOW_GBS.jou`, `WLENS_SWL.jou`, `ZERN_mode_count.jou`, `ZERN_normhex.jou`, `ZERN_refused.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — session / shell convenience.

Syntax: `! <shell line>` — the rest of the line, up to 256 characters, goes to the shell.

Defaults: none.

```
MACOS>! ls -l *.jou
```

Passes the rest of the line to the system shell; remaining tokens belong to the shell command and are not re-dispatched. Interactive CLI only. *Related:* PWD, CD, RX.

PWD, CD

Minimum match: `PWD`

Working directory shell helpers.

CLI suite: `BINARY_CENTER.jou`, `BUILD_refused_export.jou`, `COAT_legacy.jou`, `COORD_ACOOR.jou`, `EXPORT.jou`, `FILTERS_MULTISPEC.jou`, `GLASS_mod_save.jou`, `GPERTURB.jou`, `GRIDFILE_beside.jou`, `IMG.jou`, `LINEAR_MODEL.jou`, `NEW.jou`, `NSRefractor_grid.jou`, `OBS_POLYGON.jou`, `PGD_ODRAW.jou`, `PGD_reopen_units.jou`, `PLOCATE_NOPLOC.jou`, `PREAD.jou`, `RX_FID_LS.jou`, `SINT.jou`, `SPCENTER.jou`, `UDSINIT.jou`, `WINDOW_GBS.jou`, `ZCOEF_zero_rays.jou`, `ZERN_mode_count.jou`, `ZERN_normhex.jou`, `ZERN_refused.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — session / shell convenience.

Syntax: `CD <folder>` — the path, no default; `PWD` prints the working directory.

Defaults: none.

```
MACOS>cd
Enter folder path [no default]: ../examples
  Current Folder:
  /home/user/examples
```

PWD prints the current folder with no dialog. CD prompts for a path (also accepted inline: `cd ../examples`), changes directory, and echoes the new folder; failures print a system error. Interactive CLI only. *Related:* RX, ! .

RX, LS, LL

Minimum match: `RX`

List .in files / current directory.

CLI suite: `LINEAR_MODEL.jou`, `RX_FID_LS.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — session / shell convenience.

Syntax: `RX` (the .in files of the working directory), `LS` (every file), `LL` (the long listing); no prompt.

Defaults: none. LS and LL match exactly (1.2 (j)): `lspot`, `llo` are their own commands.

```
MACOS>rx
  Listing Files in: /home/user/examples
    1  Cassegrain.in     2  CoroExample.in     3  eltest.in
 The number next to a Rx filename is file_id.
 Use FID command to load a Rx file. Syntax: FID file_id
```

RX lists only `.in` prescription files, numbered for FID. LS and LL list every file in the current directory (`DIr` is also accepted). LS and LL match exactly since 2026-10-17 — before, any command starting with those letters (LSPot, LLO) was taken as a listing. Interactive CLI only. *Related:* FID, OLD, CD.

VI, EMAcs

Minimum match: `VI`

Open file in editor.

CLI suite: `EMACS_VI.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — session / shell convenience.

Syntax: `VI <file>` or `EMACS <file>` — the file name, no default; the command token as typed is the editor run.

Defaults: none.

```
MACOS>vi
Enter filename [no default]: Cassegrain.in
```

Launches the editor on the file via the system shell and waits for it to exit. The command as typed becomes the editor command, so type `emacs` in full (`ema` runs a program named ema: "sh: ema: not found"). The file-name token is passed verbatim, so `vi` with the "name" `+q!` opens and quits at once — the suite's leg. Handy for touching up a prescription before OLD/VALidate. Interactive CLI only. *Related:* OLD, VALidate, RX.

Prescription I/O

NEW

Minimum match: `NEW`

Start a new optical system.

CLI suite: `NEW.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — interactive element-entry dialog.

Syntax: `NEW <file>` — the name of the prescription to create (must not exist), then the source dialog (chief-ray direction and position, source distance, index, extinction, base units, wavelength units, wavelength, flux, grid type, aperture, grid points, the input-plane axes, segments for a segmented grid, the element count), then each element's dialog, the review ("Is this correct?") and the output frame (xOut, yOut).

Defaults: every source and element prompt shows its default in brackets and takes return, except the base units, the element count and each element's vertex, which have none; the index and extinction default to 1 and 0, the wavelength to 632.8 nm, the grid to Circular, the grid points to 256; an element's rotation point defaults to its vertex and its principal axis to (0,0,-1).

```
MACOS>new
Enter file name: MyLens
```

Starts the interactive prescription-definition dialog: source questions first (chief-ray direction and position, source distance, index and extinction, base units — no default — wavelength units, wavelength, flux, grid type, aperture, grid points, the input-plane x and y axes, segments for a segmented grid, then the element count), then each element in turn (name, element type, surface type, vertex — no default — rotation point, principal axis, conic data for a curved surface, aperture, obscurations, Fresnel distance, propagation type, element coordinates, glass), the finished prescription for review ("Is this correct?") and the output frame (xOut, yOut); see Section 4 of the manual. The name must be unused — an existing `.in` file prints "already exists. Please use a new name or use OLD." and re-prompts. On completion the new system is written to `<name>.in` and is LOADED (an OPD runs at once). `q` at the name prompt aborts. Before 2026-10-17 the named file stayed empty and the deck went to `fort.77` in the working directory (PLAN_CONSOLIDATION 1.2 (r)). *Related:* OLD, SAVe, MODify.

OLD

Minimum match: `OLD <file>`

Load a .in prescription.

CLI suite: `AMP_PHASE_REAL_PIX.jou`, `ATMOS_DMBUILD.jou`, `ATMOS_placement.jou`, `ATMOS_seed.jou`, `AVAR_VARS_MVAR_DVAR.jou`, `BEAM_GAUSS.jou`, `BEAM_gaussian.jou`, `BINARY_CENTER.jou`, `BUILD_refused_export.jou`, `BWK.jou`, `CALIB_SPOT_asph.jou`, `CALIB_SPOT_mixed.jou`, `CALIB_SPOT_rigid.jou`, `CALIB_SPOT_zern.jou`, `CALIB_WFE_asph.jou`, `CALIB_WFE_mixed.jou`, `CALIB_WFE_rigid.jou`, `CALIB_WFE_zern.jou`, `CALIB_ZWF_asph.jou`, `CALIB_ZWF_mixed.jou`, `CALIB_ZWF_rigid.jou`, `CALIB_ZWF_zern.jou`, `CALIB_beam_under_spot.jou`, `CALIB_failed_trials.jou`, `CALIB_fov_weight.jou`, `CALIB_history.jou`, `CALIB_journal_prompts.jou`, `CALIB_keywords.jou`, `CALIB_save_roundtrip.jou`, `CALIB_setup.jou`, `CALIB_stalled.jou`, `CENTER_FFP_PFP_FDP.jou`, `COAT_legacy.jou`, `COMPOSE_ADD_NOISE.jou`, `COORD_ACOOR.jou`, `COORD_offaxis_frame.jou`, `DOPD_identical.jou`, `EFL_SYSPROP.jou`, `EXPORT.jou`, `FILTERS_MULTISPEC.jou`, `FOV_SETC_SAOPT.jou`, `GLASS_mod_save.jou`, `GPERTURB.jou`, `GRIDFILE_beside.jou`, `GRIN_unsupported.jou`, `HELP_POL.jou`, `IMAGEDEMO_proptype.jou`, `IMG.jou`, `INT_LOG.jou`, `IndRef_stamp.jou`, `JOURNAL_EXE.jou`, `LINEAR_MODEL.jou`, `LONG_PATH.jou`, `MANUAL_examples.jou`, `METCALC.jou`, `MODE_centroid.jou`, `MODE_cir.jou`, `MODE_load_notice.jou`, `MODE_nolneg.jou`, `MODE_polarization.jou`, `MODE_scalar.jou`, `MOD_spaces.jou`, `MRESET.jou`, `MTF_LOS_ROWCOL.jou`, `NEW.jou`, `NSRefractor_grid.jou`, `OBS_NONE.jou`, `OBS_POLYGON.jou`, `OPD.jou`, `OPD_reference.jou`, `ORS_SRS.jou`, `OUTCORD_missing.jou`, `OptAsph_order.jou`, `PARTIALS_inputray.jou`, `PERTURB.jou`, `PGD_ODRAW.jou`, `PGD_reopen_units.jou`, `PLOCATE_NOPLOC.jou`, `PLOT_TYPES.jou`, `PREAD.jou`, `PROPTYPE_numeric.jou`, `RAY_MAP.jou`, `REGRID.jou`, `RRAY_PRAY_TPR_FSR.jou`, `Rx_header_unknown.jou`, `SAVE_roundtrip.jou`, `SEGRAYTRACE.jou`, `SESSION_INFO.jou`, `SINT.jou`, `SPCENTER.jou`, `SPOT_text.jou`, `SRAY_SRT.jou`,

STOP_FEX_SXP.jou, SYSPROP_after_OPD.jou, TRACE_keeps_rx.jou, UDSINIT.jou, WARN_obscured_elt.jou, WINDOW_GBS.jou, WLENS_SWL.jou, XPS.jou, ZABERR_ZRM_DOPD.jou, ZCOEF_zero_rays.jou, ZERN_mode_count.jou, ZERN_normhex.jou, ZERN_refused.jou, ZernType_default.jou, ZernType_extfringe.jou, ZrnGr_refusal.jou, nGridpts_clamp.jou (cli_tests/cmd/; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — load_rx.

Syntax: OLD <file> (also LOAD) — the prescription file, .in optional.

Defaults: the file name has no default; a load wipes the trace state and keeps the five session modes (POLARIZATION, VECTOR/SCALAR, LNEG, CENTROID, CIR) and the OPD reference, printing the non-default ones. A deck that carries a CALIB setup (Table 13 keys) is announced in one line (LOAD: CALIB setup in force from the prescription: ...). A deck with no output-coordinate block (nOutCord= / Tout=) loads: the beam frame at the last element is proposed (z from the previous vertex to the last vertex, x from the element's xObs=) and the CLI asks Keep the proposed output coordinates? [YES]: – NO reads nOutCord and the Tout rows; the bindings take the proposal. A numeric PropType= (0..15) resolves to its name, 0 = Geometric; an unrecognized value is refused at load, by name (1.2 (dd)). Gates: OUTCORD_missing.jou, PROPTYPE_numeric.jou, CALIB_save_roundtrip.jou.

```
MACOS>old
Enter file name: Cassegrain
 Input file Cassegrain.in being loaded.
```

The name may be given inline (old Cassegrain); .in is appended if absent, and LOAd is an identical alias. The file is pre-validated (same check as VALidate) before parsing; a bad file aborts the load and re-prompts (q aborts). A successful load prints the Optical Train Summary and resets trace/build/ propagate state. SMACOS: file name in CARG(1). *Related:* NEW, FID, VALidate, RX.

FID

Minimum match: FID <id>

Load by file id (from RX listing).

CLI suite: RX_FID_LS.jou (cli_tests/cmd/; each journal is the entry's example and its regression test).

Binding: **missing** — field / beam set.

Syntax: FID [id] — the file id.

Defaults: id [1]: the literal 1 (the dialog's, not the last id).

```
MACOS>fid
Enter file id: [1]: 3
```

Loads the prescription numbered in the RX listing; if RX has not been run yet, FID prints the listing first. q aborts; a bad id prints "** Invalid file id; can not load file". Interactive CLI shortcut for OLD. *Related:* RX, OLD.

VALidate

Minimum match: VALidate <file>

Syntax-check a .in prescription without loading it.

CLI suite: VALIDATE.jou (cli_tests/cmd/; each journal is the entry's example and its regression test).

Binding: **missing** — the validator (validate_prescription.F90) has no api entry.

Syntax: `VAL <file>` — the prescription to check, `.in` optional.

Defaults: the file name has no default.

```
MACOS>validate
Enter file name (.in optional): Cassegrain
 Cassegrain.in: OK
```

Pure check, no state change. On failure the message names the offending line and key, e.g. `line 76: key "ZernType" has no value`. The same validation runs automatically at the start of every OLD/LOAD. Interactive CLI only. *Related:* OLD, NEW.

SAVe

Minimum match: `SAVe`

Write current system to .in.

CLI suite: `CALIB_save_roundtrip.jou`, `GLASS_mod_save.jou`, `GRIDFILE_beside.jou`, `NEW.jou`, `OUTCORD_missing.jou`, `PROPTYPE_numeric.jou`, `SAVE_roundtrip.jou`, `UDSINIT.jou`, `ZERN_normhex.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `save_rx`.

Syntax: `SAVE <file>` — the new prescription's name; `.in` IS appended (`save saved` writes `saved.in`, `save saved.in` writes `saved.in.in`).

Defaults: the name has no default; the deck is written from the Rx arrays as loaded, never from the trace state, with the keys the SAVE round-trip gate lists. Since 2026-10-18 the CALIB setup is among them: every Table 13 keyword whose value differs from its load default (target, variable elements and bounds, fields and weights, beam targets, cap and tolerance) is written, so a setup made with SETC / AVAR / AFOV or `calib_set_*` transfers through the saved deck, and the next OLD prints `LOAD: CALIB setup in force from the prescription: ...`. A legacy numeric `PropType=` is written as its name (`0` is Geometric). A deck loaded without an output-coordinate block gets the proposed (or typed) `nOutCord=` / `Tout=` block written. Gates: `CALIB_save_roundtrip.jou`, `PROPTYPE_numeric.jou`, `OUTCORD_missing.jou`.

```
MACOS>save
Enter new input file name: MyCass
 File MyCass.in being created.
```

Needs a loaded Rx. Writes the current in-memory prescription — including MODify and PERturb changes — to `<name>.in`. If the file exists: `Input file MyCass.in exists.  Replace? [YES]:` (answer `n` to be re-prompted for another name). SMACOS: CARG(1)=file name, CARG(2)=replace answer. *Related:* NEW, MODify, PERturb.

EXPort

Minimum match: `EXPort`

Write select results to file.

CLI suite: `BUILD_refused_export.jou`, `EXPORT.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only, legacy** — the internal linear model, superseded by the dw_dx route (harvest through the api, build the model outside); the analytic sensitivity forms cover only some surfaces (Dave 2026-10-10).

Syntax: `EXPORT [mode] [rays] [item] …` — the mode (BINary, TEXt, MFIle, NAStran, H for help, Quit), which rays (CHFRay only, ALLRays, NOChfray), then items until Q (CMAtrix, RayIndex, RAYPos, RAYDir, RAYL, H).

Defaults: mode BINARY; rays [CHFRay]; each item prompt [QUIT], so return ends the list. The C-matrix item needs a BUILD first.

```
MACOS>export
 Current element is 1
Enter EXPORT mode (BINary, TEXt, MFIle, NAStran, H or Quit): [BINARY]: tex
 Exporting data to Cassegrain.txt
Export data for CHFRay only, ALLRays, or NOChfray? [CHFRay]:
EXPORT (CMAtrix, RayIndex, RAYPos, RAYDir, RAYL, H or Q): [QUIT]: raypos
```

Needs traced, built, or propagated data, else “No data has been generated yet.” The output file is named after the loaded prescription plus the mode extension (.bin/.txt/.m). Exported ray sets honor the OBS obscuration option. `H` prints help; `Q` ends the loop. SMACOS: CARG(1..5) supply the dialog answers. *Related:* OBS, BUild, TEXt, BINary, MAT.

SHOw

Minimum match: `SHOw <elt>` — *needs:* [Rx] [Rx]

Print all data for one element MODify - interactively edit one element.

CLI suite: `CALIB_SPOT_asph.jou`, `CALIB_SPOT_mixed.jou`, `CALIB_SPOT_rigid.jou`, `CALIB_SPOT_zern.jou`, `CALIB_WFE_asph.jou`, `CALIB_WFE_mixed.jou`, `CALIB_WFE_rigid.jou`, `CALIB_WFE_zern.jou`, `CALIB_ZWF_asph.jou`, `CALIB_ZWF_mixed.jou`, `CALIB_ZWF_rigid.jou`, `CALIB_ZWF_zern.jou`, `CALIB_failed_trials.jou`, `COAT_legacy.jou`, `IMAGEDEMO_proptype.jou`, `IndRef_stamp.jou`, `OptAsph_order.jou`, `TRACE_keeps_rx.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **direct setter** — the `elt_*` getters (`elt_info_get`, `elt_obs_get`, `elt_vpt`, `elt_psi`, `elt_kc`, `elt_kr`, …).

Syntax: `SHOW [elt]` — the element to print, 0 for the source aperture.

Defaults: elt [0]: the literal 0 — SHOW does not remember the last element.

```
MACOS>show
Enter number of element (0=aperture): [0]: 2
```

SHOw prints every stored parameter of one element; 0 prints the source/aperture data. SMACOS: element number in IARG(1). MODify (minimum match `MOD`) opens an assignment sub-prompt:

```
MACOS>mod
MOD: [q]: KrElt(2)=-2.842
```

Assignments are `<var>=<value>` with no spaces around `=`; successful mods echo back, `h` shows syntax help, `q` quits. MODify invalidates trace/build/propagate state; changes live in memory until SAVe. Under SMACOS, MODIFY runs no dialog — it only marks the Rx modified and resets those flags. *Related:* ELTS, SAVe, PERturb.

Source & Wavelength

CHIefray

Minimum match: `CHIefray`

Chief ray as the FEX/SXP pupil-sphere axis (the default).

CLI suite: `MODE_load_notice.jou`, `OPD_reference.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — `xp_fnd(mode=1)` / `sxp_fnd(1)` per call; the session axis is read by `modes_get`.

Syntax: `CHIEFRAY`; no prompt (the counterpart of CENTROID).

Defaults: none. A session mode: it survives loads until CENTROID is given; the LOAD notice names it only when CENTROID is in force.

A session mode (2026-10-11, Dave's ruling): the setting survives every LOAD until it is changed again – set once, before or after a deck is loaded. Each LOAD prints one line naming the session modes not at their defaults, e.g. `LOAD: session modes in force: POLARIZATION, VECTOR, CENTROID, NOLNEG, CIR` (nothing when all are default); `STATUS` shows them too. `RESET` returns four of them to the defaults (no polarization, scalar, the chief-ray FEX axis, LNEG); the CIR/GIR palette is kept.

```
MACOS>chiefray
 FEX/SXP pupil-sphere axis: CHIEF RAY (default)
```

No input. This pair (CHIefray / CENTRoid) selects the AXIS of the exit-pupil reference sphere FEXit/SXP write: chief psi = -(chief direction) – THE DEFAULT (all platforms since 2026-08-27; the API always chose per call: xp_fnd mode 1 = chief, 0 = centroid) centroid psi = unit(beam centroid at the next plane - EP crossing) On an obscured, segmented, or aberrated beam the downstream centroid walks off the chief, so the centroid axis TILTS the reference sphere and leaves pure tip/tilt (plus piston against a chief-type OPD reference) in the OPD – frame terms with no aberration content. Every FEX/SXP run prints which axis it used. The Rx keyword `FEXCentroid= N` additionally VETOES the centroid axis for a deck. The doctrine extends to the mmacos pupil_find layer: the written pupil reference is ALWAYS chief-tied (pupil_find calls fex mode 1 internally and writes the fit-surface/chief-ray crossing); beam-collective quantities – the centroid axis here, the cone-bundle vertex and cone-fit normal there – are DIAGNOSTICS, never references (writing them was measured to inject pure tilt or make the reference tilt-blind). The mmacos supervisors surface this same toggle: the dw_d*_multi family and run_sensitivities take `'fex_axis' 'chief'|'centroid'` for the FEX per-field reset (chief default; refused when combined with reset_xp_method='pupil_find', which is chief-tied by doctrine). CHIefray does NOT set the chief-ray position/direction — use PERturb 0, STOp, FFP or PFP for that. *Related:* CENTRoid, FEXit.

WLENS

Minimum match: `WLENS`

List Rx optimization wavelengths.

CLI suite: `WLENS_SWL.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — field / beam set.

Syntax: `WLENS`; no prompt — lists the deck's wavelengths with their ids.

Defaults: none.

```
MACOS>wlens
 Number of wavelengths used for design optimization = 2
 Wavelength  1 =  6.328000000D-07
 Wavelength  2 =  5.500000000D-07
```

Read-only, no input: lists the design-optimization wavelength list loaded from the prescription (nOptWavelen/opt_wavelen). It does not set anything; use SWL to make one of them the active wavelength. *Related:* SWL, SETC, CALIB.

SWL

Minimum match: `SWL`

Select wavelength by index.

CLI suite: `WLENS_SWL.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — field / beam set.

Syntax: `SWL [id]` — the wavelength id from the WLENS list.

Defaults: id [the id in force]: 1 after a load, then the last SWL.

```
MACOS>swl
 2 wavelengths available:
   6.328000000E-07  5.500000000E-07
Enter ID of wavelength to use [1]: 2
  Wavelength set to  5.500000000E-07
```

Selects (not sweeps) one wavelength by index from the prescription's optimization wavelength list: sets Wavelen, updates refractor indices for the new wavelength, and invalidates trace/build/propagate state. A bad ID prints "** Invalid wavelength ID". Interactive CLI only. *Related:* WLENS, SFOV (select field of view, also CLI only).

MULtispec

Minimum match: `MULtispec`

Multi-color image stacking.

CLI suite: `FILTERS_MULTISPEC.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — through `compose_*` (B.2, image-plane set).

Syntax: `MULTISPEC [elt] [star]` — the element to propagate to and the star type of the filter in force.

Defaults: elt [the last propagation element]; star [1]. Needs a filter read by RFILT whose star-type count is non-zero — a filter written by SFILT from NFILT reads back with none (1.2 (o)), so MULTISPEC refuses every type with "Invalid star type".

```
MACOS>multispec
Enter number of element where data is to be generated: [5]: 5
Enter StarType (1-2): [1]: 1
Enter number of pixels per side: [1024]: 256
Enter size of pixel: [1.E-3]: 1e-3
```

For each wavelength in the loaded filter data, propagates to the element and accumulates the flux-weighted image into the pixel array (a COMPOSE/ADD loop over the filter passband, scaled by the star type's flux curve). Requires filter data (RFIlt/NFIlt), else "Load filter data first."; overwrites any previous COMPOSEd image. SMACOS: IARG(1)=element, IARG(2)=star type, IARG(3)=pixels per side, RARG(1)=pixel size. *Related:* RFIlt, NFIlt, SFIlt, COMpose.

NFIlt, RFIlt, SFIlt

Minimum match: `NFIlt`

Load / read / save MULtispec filter data.

CLI suite: `FILTERS_MULTISPEC.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — image-plane post-processing (B.2, third).

Syntax: `NFILT [name] [λ₁ f₁ λ₂ f₂ … 0]` defines a filter at the prompts (name, then wavelength / flux pairs until a 0 wavelength); `SFILT [file]` writes it; `RFILT <file>` reads one.

Defaults: NFILT name and SFILT file [the loaded deck's name]; each wavelength [0.00000] so return ends the list; each flux [0.00000]; RFILT's file has no default.

```
MACOS>rfilt
Enter filter file name: myfilt
 Filter includes information for  10 wavelengths and  2 star types.
MACOS>nfilt
Enter filter name: [Cassegrain]: myfilt
Enter next wavelength in base units (0 when done): [0.]: 5e-7
Enter flux value at this wavelength: [0.]: 1.0
MACOS>sfilt
Enter new filter file name: [Cassegrain]: myfilt
 Filter file created.
```

These manage MULtispec filter data (the help one-liner "narrow / regular / spectral filter" is a misnomer): RFIlt reads a `.filt` file, NFIlt enters wavelength/flux pairs interactively (one star type; 0 ends the loop), SFIlt writes the data to `.filt`, silently overwriting. `.filt` is appended to the given name (a path is allowed). The format: a header line `<name> <nWavelen>`, one line of wavelengths, then one line of fluxes per star type – what SFIlt writes and RFIlt reads back (before 2026-10-18 SFIlt wrote no flux rows and MULtispec then refused every star type; 1.2 (o)). SMACOS: RFIlt/SFIlt take the file name in CARG(1). *Related:* MULtispec.

ATMosphere

Minimum match: `ATMosphere`

Apply atmospheric phase screen.

CLI suite: `ATMOS_DMBUILD.jou`, `ATMOS_placement.jou`, `ATMOS_seed.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — image-plane post-processing (B.2, third).

Syntax: `ATM [elt] [r0] [λ] [tilt] [grid]` — the element that carries the phase screen, Fried r0, the wavelength r0 is quoted at, the tilt participation 0–1, and RAY or a separate ATMospheric grid.

Defaults: elt [the last propagation element]; r0 [0.25]; λ [the deck's wavelength]; tilt [1.0]; grid RAY. The screen is drawn from the session's random sequence: `SEED n` before ATM makes it reproducible (identical on ifx and gfortran); unseeded, the start-up seed is time-based. It is a Kolmogorov screen (structure function within 6 % of 6.88 (r/r0)^(5/3) over 24 seeds), applied once as a thin phase plate: each ray's path changes by the screen and its direction by the screen's gradient, so the exit-pupil wavefront equals the screen and the spot is −EFL × its gradient (`ATMOS_seed.jou`; record `cli_tests/records/atmos_fix_2026-10-18.md`). The RAY grid samples the screen by ray-grid index and tilts along xGrid / yGrid, so it is refused by name where the ray grid at the element is not the source grid — after a fold, an image rotation or an

inversion; the element type does not matter (the Cassegrain's secondary is accepted; `ATMOS_placement.jou`). The ATMospheric grid places the screen in the element's local beam frame instead. Both model one layer conjugate to the element, seen identically by every field point.

```
MACOS>atm
Enter number of atmosphere phase screen element: [0]: 3
Enter atmosphere r0: [0.25]: 0.25
Enter atmosphere wavelength: [6.328E-7]:
Enter atmosphere tilt participation (0-1): [1.]:
Use RAY grid or separate ATMospheric grid? [RAY]:
```

Not an on/off toggle: traces the beam to the element, then adds an atmospheric-turbulence phase screen there once. r0 defaults to Aperture/16; the ATM-grid option adds an "Enter grid spacing:" prompt (clamped to >= dxElt). r0, wavelength and tilt are read single precision. SMACOS: IARG(1)=element, RARG(1)=r0, RARG(2)=wavelength, RARG(3)=tilt participation, CARG(1)=grid option, DARG(1)=grid spacing (non-RAY only). *Related:* OPD, SEEd.

SETC

Minimum match: `SETC`

Set design-optimization params.

CLI suite: `FOV_SETC_SAOPT.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — `calib_set_iter`, `calib_set_tol`, `calib_set_*`.

Syntax: `SETC [target] [maxit] [tol] [nfov] [weights] [wfile]` — the CALIB controls: the target (WFE, ZWF, Beam; SPOT cannot be chosen here, 1.2), the iteration cap, the tolerance, the field count, the field weights, whether a target-wavefront file is used.

Defaults: each prompt shows the value in force from the deck's Opt* keys (OptTarget, OptMaxItrs, OptTol, the field list and OptFOVWt) or their defaults (WFE, 10, 1e-12, the deck's fields, weights 1, no file); return keeps it.

```
MACOS>setc
 Setting options for MACOS design optimization ...
Set target of design optimization (WFE,ZWF,Beam): [WFE]:
Set maximum iterations for optimization: [20]:
Set convergence tolerance for optimization: [1.E-6]:
Set number of field of view: [1]:
Set field of view weights: [1.]:
Set target wavefront file for optimization (no/yes): [no]:
```

SETC (undocumented alias: SETP) sets design-optimization (CALIB) parameters, not stop-aperture options. ZWF adds "Enter number of Zernike modes:" / "Enter Zernike modes" prompts; answering yes to the target-wavefront question adds format (bin/ascii) and file-name prompts. Under SMACOS the dialog is compiled out — set parameters one at a time via SMACOS_SetOptParam (see dopt_mod.F). SAOpt chooses how source-adjusting commands (STOp etc.) move the source:

```
MACOS>saopt
Choose a source adjustment option, DIr or POs? [DIr]:
```

DIr re-points ChfRayDir (refused for a collimated source); POs translates ChfRayPos. SMACOS: SAOpt option in CARG(1). *Related:* CALib, VARS, WLENS, STOp.

SAOpt

Minimum match: `SAOpt`

Source-adjustment mode.

CLI suite: `FOV_SETC_SAOPT.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — source-adjustment mode.

Syntax: `SAOPT [DIR|POS]` — source adjustment by direction or by position.

Defaults: [DIR].

```
MACOS>saopt
Choose a source adjustment option, DIr or POs? [DIr]:
 - Source direction changes when ajusting source
```

Chooses which source property the source-adjusting commands (STOp chief-ray aiming, source optimization in CALib) move: DIr re-points ChfRayDir, POs translates ChfRayPos. DIr is refused for a collimated source ("** Can not change source direction for collimated source"). Until SAOpt is run the default is direction-fixed for a collimated source and position-fixed for a point source. Unrecognized input leaves the mode unchanged and reports the current one. The confirmation line's "ajusting" typo is in the code. SMACOS: option in CARG(1). *Related:* STOp, SETC, CALib.

Ray Tracing

RAY

Minimum match: `RAY <i>` — *needs:* [Rx]

Trace ray i; print state at each surface.

CLI suite: `ATMOS_DMBUILD.jou`, `ATMOS_placement.jou`, `ATMOS_seed.jou`, `CENTER_FFP_PFP_FDP.jou`, `RAY_MAP.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `ray_trace`.

Syntax: `RAY [i] [i] … 0` — ray numbers, one per prompt, 1 the chief ray; the prompt repeats until 0.

Defaults: i [0]: return ends the list. Ray numbers are the MAP numbering of the source grid.

```
MACOS>ray
Enter number of ray (1=chief ray, 0=quit): [0]: 1
 Ray 1 is the chief ray.
 Ray 1 segment from Element 0 (InputRay) to Element 1 (...):
  Starting point= ...  End point= ...  Direction= ...
  Segment Length= ...  Total Length= ...
  Index(i-1)= ...      Index(i)= ...
```

Prints start/end point, direction, geometric segment length, index-scaled running total and refractive indices for every surface-to-surface segment; obscured/undefined rays are called out per element. For non-chief rays the header adds aperture and WF grid coordinates plus adjacent-ray numbers (see MAP). Invalidates build/trace/propagate state. SMACOS: ray number in IARG(1). *Related:* MAP, SEGraytrace, SRAy, PRAy.

SEGRAYTrace

Minimum match: `SEGRAYTrace <i>` — *needs:* [Rx]

Trace a ray through segment i s center (uses RptElt).

CLI suite: `SEGRAYTRACE.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `segray_trace`.

Syntax: `SEGRAYTRACE [iseg] [iseg] … 0` — segment numbers, one per prompt, until 0.

Defaults: iseg [0]: return ends the list.

```
MACOS>segraytrace
Enter segment number (0=quit): [0]: 3
 Ray 1 segment from Element 0 (InputRay) to Element 1 (...):
  ...per-surface printout as for RAY...
Enter segment number (0=quit): [0]:
```

Loops until 0. The element must be a Segment element ("Element N is not a Segment."). Re-aims the chief-ray slot at the segment's reference point RptElt — a lateral offset for a collimated source, a new ray direction for a point source — and traces that single ray to the last element (or RayTgtElt) with the full per-surface printout of RAY. Invalidates the build/trace/propagate state. SMACOS: no LoadStack entry — the segment prompt takes its default (0 = quit immediately). *Related:* RAY, SRAy, MAP.

PRAy, RRAy

Minimum match: `PRAy`

Find pupil / marginal rays.

CLI suite: `RRAY_PRAY_TPR_FSR.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `ref_rays`.

Syntax: `PRAY` prints, `RRAY` resets; no prompt.

Defaults: none.

```
MACOS>rray
  Source aperture diameter =      4.0000 m
 At source aperture:
   Ref. ray XY offsets from chief ray: ...,  ID = ...
  Reference ray indices (chief + 4 marginal) =  1 ...
```

Ray finders, not printers. RRAy finds the chief + 4 marginal ray indices at the +/-x, +/-y edges of the source aperture; PRAy finds 8 "pupil" rays on a 0.9-radius ring. TPR then traces each found ray to the final (or RayTgtElt) element and prints its position there; it requires RRAy or PRAy first ("** Need to run RRAY command before executing TRR command" — sic). None take arguments. TPR prints each ray's end-element position (since 2026-10-17; before that it read `RayPosHist`, which only `SaveRayPosH= Y` fills, and printed 0 elsewhere). Engine quirk: PRAy's first ring ray requests normalized position (9.0,0), apparently intended as (0.9,0). *Related:* FSR, RAY, MAP.

TPR

Minimum match: `TPR`

Trace the PRAy/RRAy rays.

CLI suite: `RRAY_PRAY_TPR_FSR.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — image-plane post-processing (B.2, third).

Syntax: `TPR`; no prompt.

Defaults: none. Since 2026-10-17 (42c7ade, PLAN_CONSOLIDATION 1.2 (e)) TPR prints the position CTRACE leaves at the end element (`TPR ray 63 at element 6: x y z`), on every deck; it no longer reads `RayPosHist`, which only `SaveRayPosH= Y` fills, and it skips the slots RRAy did not fill (it used to trace "ray 0"). A ray lost before the end element says so.

No dialog. Traces each reference ray found by the last RRAy or PRAy (chief + 4 marginal, or the 0.9-radius pupil ring) from the source to the final element (or RayTgtElt) and prints one line per ray with its end-element position; the positions are also kept in `RRayPosHis` for the beam-walk commands. Without a prior RRAy/PRAy it aborts with "** Need to run RRAY command before executing TRR command" (sic). See PRAy for the finder half of the pair. SMACOS: no LoadStack entry. *Related:* PRAy, RRAy, RAY.

MAP

Minimum match: `MAP`

Print ray-to-segment map.

CLI suite: `RAY_MAP.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — Zernike / analysis set (B.2, second).

Syntax: `MAP`; no prompt.

Defaults: none.

```
MACOS>map
 Map of rays at input aperture:
   ...ray numbers on the source grid...
Hit return to continue  [0]:
 Map of segments at input aperture:
```

Prints ray numbers laid out on the source-aperture grid, tracing the grid first if needed. The chief ray (1) is not shown; with more than ~15 rays across, only a subset is labelled. Segmented systems (GridType>=3) get a second map of segment numbers. After OPD/SPOt etc., rays that became undefined disappear from the map. SMACOS: nothing is printed — the srcMap ray/segment map structure is filled instead. *Related:* RAY, OBS, SPOt.

COOrd

Minimum match: `COOrd <elt>` — *needs:* [Rx]

Beam coordinates at element.

CLI suite: `COORD_ACOOR.jou`, `COORD_offaxis_frame.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — beam coordinates at an element (the frames alone are `elt_csys_get`).

Syntax: `COORD [elt]` — the element whose beam frame is printed.

Defaults: elt [1]: the literal 1.

```
MACOS>coord
Enter element number: [1]: 5
 Computed xLocal= ...
 Computed yLocal= ...
 Computed zLocal= ...
```

Prints the beam coordinate axes at the element: projections of the source xGrid/yGrid axes, computed from four differential chief rays (angle offsets cover the near field, position offsets the far field). Signs may flip abruptly near a focus (Section 5.3.3). Invalidates trace/build/propagate state. SMACOS: element number in IARG(1). *Related:* ACOor, SPOt, STOp.

ACOor

Minimum match: `ACOor`

All-element coords (xyzLocal.txt).

CLI suite: `COORD_ACOOR.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — all-element coordinates to xyzLocal.txt.

Syntax: `ACOOR [first] [last]` — the element range written to xyzLocal.txt.

Defaults: first [1], last [1]: literals, so return alone writes element 1 only.

```
MACOS>acoor
Enter first element number: [1]: 1
Enter last element number: [1]: 6
 Result is saved in file xyzLocal.txt
```

"All COOrdinates", not anchor coordinates: runs the COOrd computation for every element in the range and writes xLocal/yLocal/zLocal per element to `xyzLocal.txt` (replaced each run) instead of the screen. Non-optical element types in the range are skipped with a warning; last must be >= first. *Related:* COOrd.

SYSprop

Minimum match: `SYSprop` — *needs:* [Rx]

First-order system properties.

CLI suite: `EFL_SYSPROP.jou`, `SYSPROP_after_OPD.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — `sysprop`.

Syntax: `SYSPROP [fp]` — the focal-plane element the plate scale is taken at.

Defaults: fp [nElt], the last element. The pixel lines come from the pitch of the last diffraction propagation (INT or PIX) to that element and say so when none has run; an OPD neither sets nor clears it (1.2 (d), Dave's ruling 2026-10-18). The api's `sysprop` / `first_order_properties` follow the same rule.

```
MACOS>sysprop
Enter focal-plane element for plate scale: [6]:

 First-order system properties:
  Entrance pupil diameter (m)        =  ...
  Central obscuration ratio          =  ...
  Wavelength (m)                     =  ...
  lambda/D (arcsec)                  =  ...
  lambda/D (rad)                     =  ...
  Effective focal length (BaseUnits) =  ...
  F-number                           =  ...
  Nyquist focal sampling (BaseUnits) =  ...
```

Runs the EFL chief+marginal-ray analysis, then derives and prints the full first-order report (sysprop_mod, shared with the pymacos/mmacos bindings). Requires a source at infinity ("**Need source at infinity to run EFL command!" aborts). The aperture/wavelength quantities always print; the EFL

block needs the marginal-ray trace to succeed (it can vignette on complex relays). The pixel-based lines (lambda/D per px, plate scale arcsec/px, pixel pitch) appear only after a diffraction propagation (OPD, INT or PIX) has set the pixel scale at the chosen element — a hint prints otherwise. SMACOS: IARG(1)=focal-plane element. *Related:* EFL, OPD, PIXel, FEXit.

EFL

Minimum match: `EFL` — *needs:* [Rx]

System effective focal length / f-ratio / plate scale.

CLI suite: `EFL_SYSPROP.jou`, `SYSPROP_after_OPD.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `sysprop`.

Syntax: `EFL`; no prompt.

Defaults: none.

```
MACOS>efl
  Estimated system effective focal length = 12.0000 m
  Estimated system focal ratio = 6.0000
  Estimated system plate scale = 2.8648 arcsec/mm
```

No input. Differential chief-ray estimate (efl.inc); focal ratio = EFL/Aperture, plate scale = 206265/(Aperture*F#) arcsec/mm (base unit assumed mm when BaseUnits is none). The estimate can be inaccurate for decentered systems; the (undocumented) SYSPROP command gives the fuller first-order report. Works in SMACOS (the printout is CLI-only; results are kept internally).* Related:* FEXit.

BWK

Minimum match: `BWK` — *needs:* [Rx]

Chief-ray walk vs nominal.

CLI suite: `BWK.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — chief-ray walk.

Syntax: `BWK`; no prompt.

Defaults: none. Needs `CalBWk= Y` in the deck (the flag is reset before the parse since 6ccd86d).

```
MACOS>bwk
 iElt=  1   ChfRayWalk =   0.000012345 ...
```

No input. Requires `CalBWk= Y` in the prescription (else "** Need set CalBWk= Y in prescription"). Traces the chief ray, then prints for every element the walk of the chief-ray incidence point from its nominal (unperturbed) position, projected onto element-local (x, y, surface-normal) axes. Typically used after PERturb to quantify beam walk. Interactive CLI only. Until 2026-10-16 the CLI reset the `CalBWk` flag after the load, so BWK always refused; fixed, gate `cli_tests/cmd/BWK.jou`. *Related:* PERturb, RAY.

FEXit

Minimum match: `FEXit`

Find exit pupil.

CLI suite: `STOP_FEX_SXP.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `xp_fnd`.

Syntax: `FEX [elt] [accept]` — the exit-pupil return surface to add, then whether to keep it.

Defaults: elt [nElt−1]: the element before the last; accept [YES]. The axis is the chief ray (CENTROID switches it to the beam centroid); the radius is the beam on the SURFACE of iElt+1.

```
MACOS>fex
Enter number of exit pupil return surface: [5]: 6
 ...chief ray / centroid locations...
 Exit pupil finder results:
   Old f= ...          New f= ...
   (z, psi, Vpt likewise)
Accept the new element? [YES]:
```

Locates the exit pupil from the chief ray plus FOUR differential chief rays (+/-5e-6 rad about two orthonormal axes perpendicular to the chief ray; the crossings are averaged, so the result does not depend on the sign or azimuth of the source frame – 2026-09-08, before that a single probe about xGrid) and refits the chosen Return/Reference element there as the reference sphere for far-field propagation (eElt=0, fElt=|z|, KrElt=-fElt, KcElt=0, psi, Vpt/Rpt updated). Requires STOp first. The EP radius is the chief-ray distance from the EP to the next element (the far-field propagation distance); the legacy previous-element leg is the fallback. When that next element is CURVED the radius runs to its SURFACE, not to its tangent plane (2026-08-28) — on a curved focal surface the plane sits the sag h**2/2R beyond it at image height h and the miss lands in the OFF-AXIS OPD as pure defocus (4.3e-5 -> 7.3e-6 mm at 1' on the JWST OTE deck; identically zero on axis). Flat next elements are bit-identical to the old behaviour, and each curved run prints the element, its Kr and the plane-surface gap. Only the base conic is used, so a grid/FreeForm figure at the next element is ignored (um-class against a sub-mm sag). The sphere AXIS is the CHIEF RAY by default (2026-08-27, all platforms); CENTRoid opts into the beam-centroid axis and CHIefray restores — see the CHIefray entry for why the centroid axis leaves tip/tilt in the OPD. Each run prints the axis it used. Rerun after any PERturb. SMACOS: IARG(1)=EP element, CARG(1)=accept answer. Dispatcher also has siblings SXP (geometrically identical radius — it intersects the chief with the next element's plane whatever its type; FEX adds the telecentric/footprint/Return-order guards) and XPS (pupil solve over the full ray grid). *Related:* STOp, CENTRoid, ORS, FDP.

SXP

Minimum match: `SXP` — *needs:* [Rx]

Set exit pupil at an element.

CLI suite: `STOP_FEX_SXP.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `sxp_fnd`.

Syntax: `SXP [elt] [accept]` — as FEX, with the exit pupil solved on the sphere.

Defaults: elt [nElt−1]; accept [YES].

```
MACOS>sxp
Enter number of exit pupil return surface: [5]: 6
 ...chief ray / centroid locations...
 Exit pupil finder results:
   Old f= ...          New f= ...
   (z, psi, Vpt likewise)
Accept the new element? [YES]:
```

Set eXit Pupil: an FEXit clone (same prompts, same EP-element refit) whose EP radius is the chief-ray distance from the EP to the next element (the focal plane at iElt+1) instead of the legacy previous-element-to-EP leg, making the radius sensitive to focal-plane despace (Tz); lateral FP motion and rotations are still not captured. Requires STOp first; the element must be a Return or Reference surface. The radius math intersects element iElt+1's vertex/normal plane when that element is flat and its actual conic SURFACE when it is curved (2026-08-28, the same rule and the same shared solver as FEX) — type-agnostic either way, so a mask/Reference/Obscuring at iElt+1 is handled correctly despite the "FP" naming. Since the FEX rework made the EP-to-next-element radius FEX's own default, FEX and SXP are geometrically identical on every deck; FEX adds the telecentric/footprint/Return-order guards. The sphere AXIS follows the same CHIefray/CENTRoid selection as FEX (chief-ray default on all platforms, 2026-08-27; each run prints its axis). SMACOS: IARG(1)=EP element, CARG(1)=accept answer. *Related:* FEXit, XPS, STOp, CENTRoid.

XPS

Minimum match: `XPS` — *needs:* [Rx]

Exit-pupil surface fit over the full ray grid.

CLI suite: `XPS.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `xps_cmd`.

Syntax: `XPS [elt]` — the exit-pupil surface evaluator (no element is added).

Defaults: elt [nElt−1].

```
MACOS>xps
Enter number of exit pupil return surface: [5]: 6
 ***** XPS: vertex = ...
 ***** XPS: zp(chief) = ...
 XPS: exit-pupil surface computed; crossings in the ray
 buffer (ray_info_get).
```

FEXit generalized to the full ray grid: traces the grid at the nominal field and at a tiny rotated field, then per ray finds the crossing point of the index-matched exit-ray pair. The cloud of crossings is the exit-pupil SURFACE (the ray-1 crossing = the FEX vertex); it is left in the ray buffer (positions, plus exit directions) for ray_info_get and the bindings. Pure geometry, no OPD; unlike FEXit/SXP no element is modified and there is no accept prompt. Requires STOp first; the element must be a Return or Reference surface. mmacos pupil_quality() fits pupil-surface Zernikes (defocus/astigmatism/coma) from the result. SMACOS: IARG(1)=EP element. *Related:* FEXit, SXP, STOp.

FSR

Minimum match: `FSR` — *needs:* [Rx]

Find source ray nearest an aperture position.

CLI suite: `RRAY_PRAY_TPR_FSR.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — image-plane post-processing (B.2, third).

Syntax: `FSR [x,y]` — the ray's position in the source aperture, normalized.

Defaults: [the last FSR position] — 0,0 (the chief ray) before any.

```
MACOS>fsr
Enter ray position in source aperture: [0.,0.]: 0.5,0
   Ref. ray XY offsets from chief ray: ...,  ID = ...
 Ray id:  1234
```

Find Source Ray — not free spectral range: returns the index of the ray whose source-aperture position best matches the given fractional (x,y) coordinates (1.0 = aperture edge along +x/+y). Candidate rays are test-traced through the system; rays that fail to pass are skipped. *Related:* RRAy, PRAy, RAY, MAP.

CENter

Minimum match: `CENter`

Center beam in system stop.

CLI suite: `BINARY_CENTER.jou`, `CENTER_FFP_PFP_FDP.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — STOP's sibling (center the beam in the stop); no verified api twin.

Syntax: `CENTER [stop] [dx,dy]` — the stop element and the offset of the chief ray at it.

Defaults: stop [1]; offset [0.00000,0.00000]: literals.

```
MACOS>center
Enter system stop element number: [1]: 2
Enter offset vector (dx,dy): [0.,0.]: 0,0
 Beam centered in   2 iterations, error= 3.231486333D-15
```

Iteratively solves for a source position that centers the beam at the element (plus a local x/y offset from its vertex); it changes ChfRayPos, not the field angle. Unlike STOp, CENter actually moves the beam. SMACOS: IARG(1)=element, DARG(1:2)=offset. *Related:* STOp, FFP, PFP.

STOp

Minimum match: `STOp`

Define system stop element.

CLI suite: `CENTER_FFP_PFP_FDP.jou`, `STOP_FEX_SXP.jou`, `XPS.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `stop_info_set`, `stop_obj_set`.

Syntax: `STOP ELT [stop] [dx,dy]` or `STOP OBJ [x,y,z]` — the stop at an element (with an offset) or at a point in object space.

Defaults: mode [OBJ]; the object-space point [0,0,0]; the element [1] with offset [0,0]. STOP keeps the handedness of the source frame; on a Segment the prompt is one line (`stop elt 1 0,0`).

```
MACOS>stop
Stop at ELT or OBJect point? [OBJ]: elt
Enter system stop element number: [1]: 3
Enter offset vector (dx,dy): [0.,0.]: 0,0
  Computed StopPos =  0.0...  0.0...  1.637981902D+01
```

Defines the system stop; it does not center the beam (use CENter). OBJ instead prompts "Enter stop (entrance pupil) position in object space (x,y,z):". ELT re-centers the chief ray on the element and computes the equivalent object-space StopPos. Sets the stop-set flag required by PERturb 0, FFP, PFP and FEXit. Source adjustment follows SAOpt (default: collimated source translates ChfRayPos; point

source re-aims ChfRayDir). If the Rx declared ApStop, the first STOp run substitutes that element/offset for the typed values. Segment elements are accepted (2026-09-08; the chief ray is mapped to that segment for the aiming trace) – note that in segment-class decks every segment's VptElt is the PARENT vertex, so offset 0,0 on any segment is the parent vertex; use an RptElt- based offset for a segment centre. Non-sequential elements (NSReflector/NSRefractor) are refused by name. The OBJ answer may be typed on one line (`stop obj 0 0 0`); a short line is completed from the next prompt. SMACOS: CARG(1)='ELT'|'OBJ'; ELT: IARG(1)=element, DARG(1:2)=offset; OBJ: DARG(1:3)=position. *Related:* CENter, SAOpt, FEXit, FFP, PFP.

CENTRoid

Minimum match: `CENTRoid`

Beam-centroid FEX/SXP axis (opt-in; the chief ray is the default).

CLI suite: `MODE_centroid.jou`, `MODE_load_notice.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — `xp_fnd(mode=0)` / `sxp_fnd(0)` per call; the session axis is read by `modes_get`.

Syntax: `CENTROID`; no prompt (the counterpart of CHIEFRAY).

Defaults: none. A session mode: the FEX/SXP axis follows the beam centroid until CHIEFRAY; survives loads and is named in the LOAD notice.

A session mode (2026-10-11, Dave's ruling): the setting survives every LOAD until it is changed again – set once, before or after a deck is loaded. Each LOAD prints one line naming the session modes not at their defaults, e.g. `LOAD: session modes in force: POLARIZATION, VECTOR, CENTROID, NOLNEG, CIR` (nothing when all are default); `STATUS` shows them too. `RESET` returns four of them to the defaults (no polarization, scalar, the chief-ray FEX axis, LNEG); the CIR/GIR palette is kept.

```
MACOS>centroid
 FEX/SXP pupil-sphere axis: beam CENTROID (opt-in; chief ray is the default)
```

No input. Aims the FEX/SXP pupil-sphere axis at the downstream beam centroid instead of along the chief ray. On obscured, segmented, or aberrated beams the centroid walks off the chief, so this axis TILTS the reference sphere and leaves tip/tilt (and piston, against a chief-type OPD reference) in the OPD — see the CHIefray entry for the full convention table. CHIefray restores the chief-ray default (the default on all platforms since 2026-08-27); the Rx keyword `FEXCentroid= N` vetoes the centroid axis per deck. *Related:* CHIefray, FEXit.

FFP <x,y>

Minimum match: `FFP <x`

Find field point that places chief ray at element coordinate.

CLI suite: `CENTER_FFP_PFP_FDP.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `ffp`.

Syntax: `FFP [elt] [dx,dy] [accept]` — the element, the offset of the chief ray there in global units, and whether to keep the new chief ray.

Defaults: elt [nElt]; offset [0.00000,0.00000]; accept [YES].

```
MACOS>ffp
Enter element number: [6]:
Enter offset vector in global units (dx,dy): [0.,0.]: 1e-3,-2e-3
 Field angle finder results:
 Did    1 iterations, error= 6.936774704D-11
 ...old/new dx, crd, crp table...
Accept the new chief ray? [YES]:
```

Newton search (max 100 iterations, differential chief rays pinned to the stop) for the source field that places the chief ray at the target point on the element — offset from its vertex, resolved on the beam-local axes, in global units. Requires STOp first. Accepting resets ChfRayPos/ChfRayDir. A chief ray that reaches the element but is obscured there is tolerated. SMACOS: IARG(1)=element, DARG(1:2)=offset, CARG(1)=accept answer. *Related:* PFP, STOp, CENter.

PFP <x,y>

Minimum match: `PFP <x`

Find field point that places chief ray at pixel coordinate.

CLI suite: `CENTER_FFP_PFP_FDP.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `pfp`.

Syntax: `PFP [elt] [pixel] [dx,dy] [accept]` — the element, the pixel size, the image position in pixels, and whether to keep the new chief ray.

Defaults: elt [nElt]; pixel [the pixel size in force from PIX or WINDOW, 0 before any]; position [0,0]; accept [YES].

```
MACOS>pfp
Enter element number: [6]: 6
Enter pixel size for placing image on pixel array: [0.]: 15e-4
Enter image position in pixel units (dx,dy): [0.,0.]: 20,-100
 Field angle finder results:
 ...
Accept the new chief ray? [YES]:
```

Same solver as FFP, with the target given in pixels: target = (pixel - EltPix) * pixel size, where EltPix and the default pixel size come from WINdow/PLOCate. Requires STOp first; accepting resets ChfRayPos/ChfRayDir. SMACOS: IARG(1)=element, DARG(1)=pixel size, DARG(2:3)=pixel position, CARG(1)=accept answer. *Related:* FFP, PLOcate, STOp.

FDP

Minimum match: `FDP`

Find detector plane orientation.

CLI suite: `CENTER_FFP_PFP_FDP.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — image-plane post-processing (B.2, third).

Syntax: `FDP [elt] [accept]` — the detector element and whether to keep its new position.

Defaults: elt [nElt]; accept [YES].

```
MACOS>fdp
Enter detector element number: [6]: 6
```

```
 New Detector Position = ...
 New Detector Normal   = ...
Accept the New Detector Position? [YES]:
  --Detector Element Position ( Elt 6) Updated!
```

Finds the focal-plane location/orientation: traces the chief ray plus two differential chief rays and averages their sagittal and tangential crossing points; the outgoing chief-ray direction gives the plane normal. Accepting updates the element's Vpt/Rpt and psi (keeping psi's sign) and, when element-2 is a Return surface linked to the detector, moves it too. SMACOS: IARG(1)=element, CARG(1)=accept answer. *Related:* FEXit, FFP, CENter.

SPCenter

Minimum match: `SPCenter`

Spot-diagram centering option.

CLI suite: `SPCENTER.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `spot_cmd`.

Syntax: `SPCENTER [elt|chfray]` — centre the spot diagram on the element vertex or on the chief ray.

Defaults: [elt]; a session option (`spcOption`, set at allocation), so it survives loads.

```
MACOS>spcenter
Enter spot diagram centering option (elt,chfray): [elt]:
```

Chooses the spot-diagram origin for subsequent SPOt plots: `elt` (default) plots ray positions relative to the element reference point; `chfray` plots them relative to the chief-ray spot. SMACOS: option in CARG(1). *Related:* SPOt, OBS, CENTRoid.

Behavior Toggles

LNEg/NOLNeg

Minimum match: `LNEg/NOLNeg`

Allow/disallow negative pathlengths.

CLI suite: `MODE_load_notice.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — no api setter; read by `modes_get` (a session mode – NOT the linear model, whatever its L).

Syntax: `LNEG` / `NOLNEG`; no prompt.

Defaults: none. A session mode (LNEG is the default): survives loads; NOLNEG is named in the LOAD notice.

A session mode (2026-10-11, Dave's ruling): the setting survives every LOAD until it is changed again – set once, before or after a deck is loaded. Each LOAD prints one line naming the session modes not at their defaults, e.g. `LOAD: session modes in force: POLARIZATION, VECTOR, CENTROID, NOLNEG, CIR` (nothing when all are default); `STATUS` shows them too. `RESET` returns four of them to the defaults (no polarization, scalar, the chief-ray FEX axis, LNEG); the CIR/GIR palette is kept.

```
MACOS>lneg
 Negative-length rays OK at reference surfaces
```

```
MACOS>nolneg
 No negative-length rays allowed
```

No input. Sets/clears the trace flag permitting a ray segment of negative length at a reference surface (the surface lies behind the ray, e.g. a reference sphere placed just past focus or double-pass layouts). Contrary to what “allow/disallow” might suggest, the initialized state is ALLOWED (LNEg); use NOLNeg to forbid negative lengths. *Related:* ORS, SRS, FEXit.

POLarized

Minimum match: `POLarized`

Polarization tracing on.

CLI suite: `HELP_POL.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **SMACOS command** — `pol_set`.

Syntax: `POL [Ex0] [Ey0]` — the input polarization, each a complex pair (real,imag).

Defaults: Ex0 and Ey0 [the pair in force], 0,0 before any — a 0,0 launch is swallowed silently (the polarization plan’s record). A session mode: survives loads, named in the LOAD notice.

A session mode (2026-10-11, Dave’s ruling): the setting survives every LOAD until it is changed again – set once, before or after a deck is loaded. Each LOAD prints one line naming the session modes not at their defaults, e.g. `LOAD: session modes in force: POLARIZATION, VECTOR, CENTROID, NOLNEG, CIR` (nothing when all are default); `STATUS` shows them too. `RESET` returns four of them to the defaults (no polarization, scalar, the chief-ray FEX axis, LNEG); the CIR/GIR palette is kept.

```
MACOS>pol
 Polarization turned on
 Vector diffraction enabled
Enter Ex0 (real,imag): [0.,0.]: 1,0
Enter Ey0 (real,imag): [0.,0.]: 0,0
```

Turns on polarization ray tracing (off by default) and prompts for the complex input field components Ex0/Ey0. When the model supports vector diffraction (mWF>=3, true for all stock model sizes) it also enables vector diffraction (“Vector diffraction enabled”); SCAlar/VECtor toggle that separately. Invalidates propagate state. SMACOS: DARG(1:2)=Ex0, DARG(3:4)=Ey0 (real,imag pairs).

With polarization on, every ray carries a complex 3-vector E-field: rays launch as E = Ex0*xGrid* + *Ey0*yGrid in the source frame (point sources use a per-ray frame that reduces to xGrid/yGrid on the chief ray), and each Reflector/Refractor applies its s/p Fresnel or multilayer-coating coefficients (see the `Coating=`, `IndRef=`/`Extinc=` Rx keywords). Conventions: absorbing index N = n - i*kappa*, *time-harmonic exp(+i*omega*t). A mirror with the standard `IndRef=1, Extinc=1e22` idiom is a perfect conductor (RP=RS=-1): polarization-neutral. The source state can also be carried in the Rx via `Ex0Ey0= ExRe ExIm EyRe EyIm` (state only; on/off remains a command/API decision). Bindings: `macos.polarization` / `pymacos.polarization`; see also `macos.jones_pupil` for polarization-aberration analysis.* Related:* NOPolarization, VECtor, SCAlar.

NOPolarization

Minimum match: `NOPolarization`

Polarization off.

CLI suite: `MODE_load_notice.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `pol_set`.

Syntax: `NOPOL`; no prompt.

Defaults: none. The default state; a session mode.

A session mode (2026-10-11, Dave's ruling): the setting survives every LOAD until it is changed again – set once, before or after a deck is loaded. Each LOAD prints one line naming the session modes not at their defaults, e.g. `LOAD: session modes in force: POLARIZATION, VECTOR, CENTROID, NOLNEG, CIR` (nothing when all are default); `STATUS` shows them too. `RESET` returns four of them to the defaults (no polarization, scalar, the chief-ray FEX axis, LNEG); the CIR/GIR palette is kept.

```
MACOS>nopol
 Polarization turned off
```

No input. Turns polarization ray tracing off (the default state) and disables vector diffraction; invalidates propagate state if polarization was on. *Related:* POLarized, SCAlar, VECtor.

OBS

Minimum match: `OBS`

Spot-diagram obscuration option.

CLI suite: `OBS_NONE.jou`, `SESSION_INFO.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `obs_set`.

Syntax: `OBS [ALL|POSITIVE|NEGATIVE]` — which rays the trace keeps against obscurations.

Defaults: [POSITIVE]. Reset by RESET (back to option 1); `obs_opt_get` reads it from the bindings.

```
MACOS>obs
 Enter ray-trace obscuration option (ALL, POSITIVE, NEGATIVE): [POSITIVE]: all
 All rays plotted on spot diagrams
```

Chooses which rays SPOt and OPD plot (Section 5.2.7): POSITIVE (default) plots only rays unobscured on the current and all previous elements; ALL plots every ray, even obscured ones; NEGATIVE plots only rays that are obscured somewhere. Numeric answers 0/1/2 are also accepted. SMACOS: option in CARG(1). *Related:* SPOt, OPD, MAP, SPCenter.

REGrid/NOREGrid

Minimum match: `REGrid/NOREGrid`

Resample wavefront at element during propagation.

CLI suite: `REGRID.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — a per-element resampling flag.

Syntax: `REGRID [elt]` / `NOREGRID [elt]` — the element whose grid is re-sampled at each trace.

Defaults: elt [0]: the literal 0 (a per-element deck flag, so it is per load).

```
MACOS>regrid
Enter element number: [0]: 5
 Rays will be regridded at element  5
MACOS>noregrid
Enter element number: [0]: 5
 Rays will not be regridded at element  5
```

Per-element toggle marking the element for resampling of the rays/wavefront onto a regular grid during diffraction propagation — aberrated systems can lose ray-grid regularity, especially near focus (Section 6.2). REGrid also clears the wavefront buffer and propagate state. Section 6.3.3 states the regridding feature is “undergoing revision and not currently supported”, but the toggle remains live in the code. SMACOS: LoadStack passes no element argument, so the command is effectively interactive-CLI only. *Related:* PROpagate, OPD.

ORS

Minimum match: `ORS`

Optimize reference surface.

CLI suite: `ORS_SRS.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **SMACOS command** — `ors_run`.

Syntax: `ORS [elt] [accept]` — the element to optimize and whether to keep the new data.

Defaults: elt [1]; accept [YES]. A failed element number aborts the command (“command aborted”) instead of re-prompting.

```
MACOS>ors
Enter number of element to be optimized: [1]: 12
 Reference surface optimization results:
 TTL OPD= ...          nCalls= ...
   Old f= ...          New f= ...
   (z, psi, Vpt likewise)
Accept the new element data? [YES]:
```

Optimize Reference Surface for near-field diffraction (Section 6.3): the element must be a Reference, Obscuring or Return surface. Traces a small test ray grid to the preceding element, then bracket-minimizes the RMS OPD over the reference radius fElt (tolerance lambda/100), re-centering the vertex on the chief ray and aligning psi with it. Accepting updates fElt, KrElt=-(1+e)fElt, zElt, psi, Vpt and Rpt (SAVe to keep); a flat reference (f>=1e21) keeps its f and z. May converge off-optimum in systems with strong odd aberrations (coma); if it misbehaves, the surface may be in backwards — flip psiElt. SMACOS: IARG(1)=element, CARG(1)=accept answer. *Related:* SRS, FEXit, LNEg, SAVe.

SRS

Minimum match: `SRS`

Slave one ref surface to another.

CLI suite: `ORS_SRS.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **SMACOS command** — `srs_run`.

Syntax: `SRS [slave] [master] [accept]` — the element slaved, the element it follows, and whether to keep it.

Defaults: slave [1]; master [1]; accept [YES].

```
MACOS>srs
Enter number of element to be slaved: [1]: 13
Enter number of element to slave to: [1]: 12
 Reference surface recalculation results:
   Old f= ...          New f= ...
   (z, psi, Vpt likewise)
Accept the new element? [YES]:
```

Slave Reference Surface: solves the location, orientation, radius and propagation distance zElt of one reference surface of a diffraction-propagation pair from the other's parameters plus the chief-ray path between them; run ORS on the first surface, then SRS on the second (Section 6.3). Both elements must be FocalPlane, Reference, Obscuring or Return surfaces. Flats keep f=INF regardless of the propagation distance, and a slaved-to FocalPlane's big zElt sentinel is preserved. Return-surface parity between the two elements selects among plane/sphere geometry cases; the asymmetric sphere-slaved-to- plane case is unimplemented (warning). Code bug: the accept test uses .OR. where FEXit/ORS use .AND., so the new element data is applied regardless of the answer. SMACOS: IARG(1)=slaved element, IARG(2)=slave-to element, CARG(1)=accept answer. *Related:* ORS, FEXit, PROpagate.

NONe

Minimum match: `NONe`

Turn off plotting (no PGPLOT calls).

CLI suite: `AMP_PHASE_REAL_PIX.jou`, `ATMOS_DMBUILD.jou`, `ATMOS_placement.jou`, `ATMOS_seed.jou`, `BEAM_GAUSS.jou`, `BEAM_gaussian.jou`, `COAT_legacy.jou`, `COMPOSE_ADD_NOISE.jou`, `DOPD_identical.jou`, `FILTERS_MULTISPEC.jou`, `GRIDFILE_beside.jou`, `IMG.jou`, `LINEAR_MODEL.jou`, `MTF_LOS_ROWCOL.jou`, `NEW.jou`, `OBS_NONE.jou`, `OBS_POLYGON.jou`, `PLOCATE_NOPLOC.jou`, `SESSION_INFO.jou`, `SINT.jou`, `WINDOW_GBS.jou`, `ZABERR_ZRM_DOPD.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — plot style (the bindings return the arrays).

Syntax: `NONE`; no prompt — the plot type that draws nothing.

Defaults: none. A plot type like GRAY / CONTOUR …; RESET does not restore it.

```
MACOS>none
```

No input, no output: sets the surface-plot type to 0, so plotting commands produce no graphics (numeric summaries still print). NULL is an undocumented synonym. This is the plot type of choice for journals/scripts that must run headless. *Related:* GRay, SLIce, CONtour, WIRe, TEXt, BINary.

Surface Data

SINt

Minimum match: `SINt`

Set up interpolated-surface data.

CLI suite: `SINT.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — single-ray tracing (B.2, first).

Syntax: `SINT`; no prompt — reads `<deck>.in.srfN.txt` (or `.bin`) for every Interpolated surface N.

Defaults: none; a missing file prompts "Enter new .srf-file root name or QUIT to exit:" [QUIT].

```
MACOS>sint
 Computing interpolated surface data...
 Compute time was    5.7871 sec
```

Surface INTerpolate: reads the data files for every Interpolated (SrfType=5) surface in the prescription and precomputes the triangular-interpolation coefficients used by the ray trace (Section 4.4). Per element it expects `<deck>.srfN.bin` (binary) with `.txt` fallback, where is the loaded prescription's file name INCLUDING its `.in` (`Cassegrain.in.srf3.txt` for element 3); if neither exists it prompts "Enter new .srf-file root name or QUIT to exit:". File format: header `nDP iComputeDZ`, then one `x y z [dz/dx dz/dy]` row per data point (slopes computed by the DIDES code when iComputeDZ=1); x, y in the element's local frame, z the DEVIATION from the base conic, positive along psiElt (toward the incoming beam) — a uniform +1e-6 shortens the path to the surface by 1e-6. Measured on the suite's f = 1 m paraboloid: a 1e-4 r^2 deviation gives RMS 5.64e-7 / P-V 1.91e-6 at the focal plane (5.8e-7 / 2.0e-6 for twice the deviation). Until SINt runs, interpolated surfaces trace as their base conic. Invalidates trace/build/propagate/perturb state. SMACOS: no arguments — the data files must exist under the loaded Rx root name. *Related:* UDSinit, MODify.

UDSinit

Minimum match: `UDSinit`

Set up user-defined-surface data.

CLI suite: `UDSINIT.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — builder.

Syntax: `UDSINIT [elt] [file]` — the element (0 for the aperture) and its user-defined surface file.

Defaults: elt [0]; file none.

```
MACOS>udsinit
Enter number of element (0=aperture): [0]: 3
 Enter user-defined surface file name  [none]: dm349.coef
```

Initializes UserDefined (SrfType=7) surface data for the element, dispatching on its UDSrfType: the built-in types are deformable-mirror influence-function models (1313-, 349- and 397-actuator arrays plus squared-out, bilinear, radial and damped-sine square-array variants) whose files list actuator-number/coefficient pairs, the Hubble primary and secondary x-y grid maps, and a general grid surface. On success the trace/build/propagate state is invalidated; "Unknown user-defined surface type." otherwise. SMACOS: IARG(1)=element, CARG(1)=file name (an SMACOS caller can also poke UDSrfCoef directly, Section 4.4.10). *Related:* SINt, MODify.

Perturbation

PERturb

Minimum match: `PERturb <elt>` — *needs:* [Rx]

Apply 6-DOF perturbation.

CLI suite: `BINARY_CENTER.jou`, `BWK.jou`, `DOPD_identical.jou`, `METCALC.jou`, `PERTURB.jou`, `SPCENTER.jou`, `ZABERR_ZRM_DOPD.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `prb_elt`, `perturb_src`.

Syntax: `PERTURB [elt] [ELEMENT|GLOBAL] [rx,ry,rz] [tx,ty,tz]` — the element, the frame, the rotation vector and the translation vector.

Defaults: elt [iCurWFElt, the element of the last wavefront command; 0 before any]; frame [GLOBAL]; both vectors [0.00000,0.00000,0.00000].

```
MACOS>perturb
Enter element to be perturbed: [0]: 3
Enter coordinate system for perturbation (ELEMENT or GLOBAL): [GLOBAL]:
Enter rotational perturbation vector (x,y,z): [0.,0.,0.]: 0,0,0
Enter translational perturbation vector (x,y,z): [0.,0.,0.]: 0,0,1e-6
```

Applies a rigid-body perturbation directly to the prescription: rotation as an Euler axis/angle 3-vector (direction=axis, magnitude=angle in radians) about the rotation point RptElt, translation in base units; not limited to small motions. Element 0 perturbs the source (requires STOp unless the Rx set RxNoStopSet; rotates ChfRayDir and the grid axes about StopPos). Answering ELEMENT prompts instead for one vector of length nECoord in element coordinates ("No local DOF available!" when nECoord=0). All attached data move with the element: apertures, HOE points, Mon/grid/FreeForm frames, metrology points, and one linked element (LnkElt). If the Rx declared ApStop at or after the element, the stop is re-set automatically afterwards. To undo, reverse the perturbation or reload the Rx (preferred after multiple rotations). Invalidates trace/build/propagate/grid state. SMACOS: IARG(1)=element; element>0: CARG(1)='YES' (element coords, DARG(1:6)) or 'NO'/'GLObal' (DARG(1:3)=rotation, DARG(4:6)=translation); element 0: DARG(1:6) as rotation+translation. *Related:* GPERturb, PREad, MODify, STOp, BWK, LPErturb.

GPERturb

Minimum match: `GPERturb <grp>` — *needs:* [Rx]

Perturb an element group.

CLI suite: `GPERTURB.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `prb_elt_grp`.

Syntax: `GPERTURB [grp] [ELEMENT|GLOBAL] [rx,ry,rz] [tx,ty,tz]` — the element group (`EltGrp=` in the deck), the frame, the rotation and translation vectors.

Defaults: grp [iCurWFElt]; frame [GLOBAL]; vectors [0,0,0]. An invalid group aborts the command.

```
MACOS>gperturb
Enter element to be perturbed: [0]: 5
Enter coordinate system for perturbation (ELEMENT or GLOBAL): [GLOBAL]:
Enter rotational perturbation vector (x,y,z): [0.,0.,0.]:
Enter translational perturbation vector (x,y,z): [0.,0.,0.]:
```

Group PERturb: rigid-body perturbs every member of the element group defined at the element (the `EltGrp=` list in the Rx), rotating all members about the group element's rotation point so the group moves as one body; prints " No element group defined at element N" otherwise. Same coordinate conventions and prompts as PERturb (element coordinates use the group element's nECoord/TElt). Invalidates trace/build/propagate/ grid state. SMACOS: same argument mapping as PERturb (IARG(1), CARG(1), DARG(1:6)). *Related:* PERturb, PREad.

PREad

Minimum match: `PREad <file>`

Apply perturbations from file.

CLI suite: `PREAD.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — `prb_elt` / `perturb_elt` (the file is read in the caller).

Syntax: `PREAD <file>` — the .pert file of perturbations to apply.

Defaults: the file name has no default.

```
MACOS>pread
Enter new .pert-file name: [Cassegrain.pert]:
 File Cassegrain.pert read.
```

Reads a `.pert` file — one all-numeric 6-vector per line (3 rotations then 3 translations, space-separated, GLOBAL coordinates), line 1 = element 1 and so on; every element through the last must have a row (zeros if unperturbed) — and applies each row exactly as a PERturb of that element (rotation about RptElt, attached frames updated). The default name is the Rx root + `.pert`; if the file is missing, PREad re-prompts (answer `q` to give up). A short or malformed file prints " Input file … not compatible with current system". SMACOS: file name in CARG(1). *Related:* PERturb, LPRead.

Linear Model

BUild

Minimum match: `BUild` — *needs:* [Rx]

Build linear model from chief-ray ray partials.

CLI suite: `BUILD_refused_export.jou`, `LINEAR_MODEL.jou`, `PARTIALS_inputray.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only, legacy** — the internal linear model, superseded by the dw_dx route (harvest through the api, build the model outside); the analytic sensitivity forms cover only some surfaces (Dave 2026-10-10).

Syntax: `BUILD [elt]` — the terminal element of the linear model.

Defaults: elt [1]: the literal 1, so give the element — the C-matrix is built to it.

```
MACOS>build
Enter terminal element number: [1]: 4
 Tracing    150 rays and BUILDing linear model...
 Compute time was    0.4141 sec
```

Traces every ray to the terminal element while assembling the C-matrix: per-ray differential sensitivities of ray direction, position and pathlength at that element to source and per- element 6-DOF perturbations (Section 8). Prerequisite for PARtials, LPErturb, LPRead, LSPot, LOPd, LPIxilate and EXPort; zeroes the linear perturbation vector. The terminal element cannot be a Segment or a non-sequential element ("Invalid element type"); mid-train reference/obscuring surfaces are ignored and return surfaces are not supported (Section 8.5.1). Ray count is limited to bRay ("BUILD limited to N rays. MOD npts and recompute."). SMACOS: IARG(1)=terminal element. *Related:* DMBuild, PARtials, LPErturb, LPRead, EXPort.

DMBuild

Minimum match: `DMBuild` — *needs:* [Rx]

Build with deformable mirror.

CLI suite: `ATMOS_DMBUILD.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only, legacy** — the internal linear model, superseded by the dw_dx route (harvest through the api, build the model outside); the analytic sensitivity forms cover only some surfaces (Dave 2026-10-10).

Syntax: `DMBUILD [elt]` — as BUILD, with the interpolated-surface partials.

Defaults: elt [1].

```
MACOS>dmbuild
Enter terminal element number: [1]: 4
 Tracing    150 rays and BUILDing linear model...
```

Same dialog and trace as BUild, additionally raising the DM-build flag intended to add deformable-mirror (user-defined surface) actuator partials to the linear model — but the code paths that consume the flag are commented out, so DMBuild currently behaves exactly like BUild, consistent with Section 8.5.2: "The DMBuild command is not used in this version." (The summary line above is a generator artifact — the help text's PARtials one-liner is merged into it; see BUild.) SMACOS: IARG(1)=terminal element. *Related:* BUild.

PARtials

Minimum match: `PARtials <i>` — *needs:* [BLD]

Print partial derivs, ray i.

CLI suite: `BUILD_refused_export.jou`, `LINEAR_MODEL.jou`, `PARTIALS_inputray.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only, legacy** — the internal linear model, superseded by the dw_dx route (harvest through the api, build the model outside); the analytic sensitivity forms cover only some surfaces (Dave 2026-10-10).

Syntax: `PARTIALS [i] [actsens] …` — ray numbers until 0 (1 the chief ray), each asked whether to print in actuator/sensor coordinates.

Defaults: i [0]: return ends; actuator/sensor [YES]. Needs a BUILD.

```
MACOS>partials
Enter ray number for partials (0=done, 1=chief ray): [0]: 1
Print partials in actuator/sensor coordinates? [YES]:
 Partial of ray    1 at Element    4 (...) to ray at Element
    0 (...)
 ...7x7 sensitivity block...
 Partial of ray    1 at Element    4 (...) to Element    1
 (...) perturbations
 ...7x6 blocks, one per element the ray reaches...
```

Prints one ray's rows of the BUild C-matrix: the sensitivity of the ray state (direction, position, pathlength) at the BUild terminal element, first to source (Element 0) motion, then to the 6-DOF perturbations of each element the ray reaches. Answering YES compacts the blocks through the Tout output matrix and each element's actuator/sensor transform (only elements with nECoord/=0 print); NO prints the raw global-coordinate partials. Requires BUild ("Must BUILD first."); a bad or lost ray gets "Invalid ray number." Despite the "0=done" wording, control returns to the MACOS prompt after ONE ray — the loop-back is commented out. SMACOS: IARG(1)=ray number; the coordinate question falls back to its default (YES) because LoadStack's generic branch consumes PARTIALS before the dedicated branch that would pass CARG(1) is reached. *Related:* BUild, DMBuild, EXPort, LPErturb.

LPErturb

Minimum match: `LPErturb` — *needs:* [BLD]

Linear-model perturb.

CLI suite: `LINEAR_MODEL.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only, legacy** — the internal linear model, superseded by the dw_dx route (harvest through the api, build the model outside); the analytic sensitivity forms cover only some surfaces (Dave 2026-10-10).

Syntax: `LPERTURB [elt] [eltcoords] [rx,ry,rz] [tx,ty,tz]` — a perturbation applied through the linear model.

Defaults: elt [0]; element coordinates [NO]; vectors [0,0,0]. Needs a BUILD. CLI-only, legacy: the `dw_dx` route supersedes it.

```
MACOS>lperturb
Enter element to be perturbed: [0]: 3
Use element coordinates? [NO]:
Enter rotational perturbation vector (x,y,z): [0.,0.,0.]: 0,0,0
Enter translational perturbation vector (x,y,z): [0.,0.,0.]: 0,0,1e-6

 Element    1 Linear Perturbations:
   Rotational= ...
Translational= ...
```

Sets the linear-model perturbation for an element (the Rx itself is untouched); requires BUild first, and element 0 (the source) is not allowed. The linear model requires nOutCord=5 in the order direction (2) / beamwalk (2) / OPD (1), else "PERTURB expects 5 output coordinates…" (sic). Answering YES reads one 6-vector in element coordinates. The full per-element perturbation table prints afterwards. Two deviations from Section 8.5.3: the entry for the chosen element is REPLACED, not summed (accumulation is only across different elements), and the element-coordinate vector is always read as 6 components regardless of nECoord. Reset with LREset. SMACOS: IARG(1)=element, CARG(1)=YES/NO (element coords), DARG(1:6)=perturbation. *Related:* LPRead, LREset, LSPot, LOPd, BUild, PERturb.

LPRead

Minimum match: `LPRead` — *needs:* [BLD]

Linear perturb from file.

CLI suite: `LINEAR_MODEL.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only, legacy** — the internal linear model, superseded by the dw_dx route (harvest through the api, build the model outside); the analytic sensitivity forms cover only some surfaces (Dave 2026-10-10).

Syntax: `LPREAD <file>` — a .pert file applied through the linear model.

Defaults: the file name has no default.

```
MACOS>lpread
Enter .pert-file name: [Cassegrain.pert]:
 FileCassegrain.pert read.
```

(Missing space after “File” — sic.) Reads a `.pert` file (same all-numeric global-coordinate format as PREad, one row per element up to the BUild terminal element) into the linear-model perturbation vector; the prescription is NOT changed. Requires BUild first (“Must BUILD first.”); on success sets the perturbed flag so LSPot/LOPd/LPIxilate can run. Missing file re-prompts (`q` gives up). SMACOS: file name in CARG(1). *Related:* PREad, LPErturb, LREset, BUild.

LREset

Minimum match: `LREset` — *needs:* [BLD]

Zero linear perturbations.

CLI suite: `LINEAR_MODEL.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **CLI-only, legacy** — the internal linear model, superseded by the dw_dx route (harvest through the api, build the model outside); the analytic sensitivity forms cover only some surfaces (Dave 2026-10-10).

Syntax: `LRESET`; no prompt — clears the linear perturbation state.

Defaults: none.

```
MACOS>lreset
```

No input, no output: zeroes the linear-model perturbation vector for all elements. Use it between LPErturb cases. It does not clear the “perturbed” flag, so a following LSPot/LOPd shows the unperturbed linear result. SMACOS: no arguments. *Related:* LPErturb, LPRead.

LSPot, LOPd

Minimum match: `LSPot` — *needs:* [BLD]

Linear-model spot / OPD.

CLI suite: `LINEAR_MODEL.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **CLI-only, legacy** — the internal linear model, superseded by the dw_dx route (harvest through the api, build the model outside); the analytic sensitivity forms cover only some surfaces (Dave 2026-10-10).

Syntax: `LSPOT` / `LOPD`; no prompt — the spot / OPD predicted by the linear model.

Defaults: none. LSPOT draws regardless of the NONE plot type (1.2 (m)).

```
MACOS>lspot
 Tracing perturbed system
 Average OPD is ...
 RMS OPD including piston is ...
```

Evaluates the linear model — nominal ray state at the BUild element plus C-matrix times the current perturbation vector — and plots the result: LSPot a spot diagram (“Spot Diagram”), LOPd a square OPD map (“LinMod OPD Map”), each preceded by the average/RMS OPD summary above. Both require a prior LPErturb or LPRead (“Must LPERTURB before plotting linear model results.”) and the nOutCord=5 output-coordinate setup (see LPErturb). They parallel the exact SPOt and OPD commands — comparing linear vs exact results over the expected perturbation range is the standard linearity check (Section 8.1). SMACOS: no arguments. *Related:* LPErturb, LPIxilate, SPOt, OPD.

LPIxilate

Minimum match: `LPIxilate` — *needs:* [BLD]

Linear-model pixelated image.

CLI suite: `LINEAR_MODEL.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only, legacy** — the internal linear model, superseded by the dw_dx route (harvest through the api, build the model outside); the analytic sensitivity forms cover only some surfaces (Dave 2026-10-10).

Syntax: `LPIX [npix] [pixel]` — the pixel count per side and the pixel size for the linear-model image.

Defaults: npix [128]; pixel [0.00000] (a zero size reads the grid's).

```
MACOS>lpix
 Tracing perturbed system
 Average OPD is ...
 RMS OPD including piston is ...
 FFT/Point Spread Function Data Summary:
 Wavelength= ...;  Transmission Distance= ...
 u-v plane diam= ...  du= ...
 x-y plane diam= ...  dx= ...
Enter number of pixels per side: [64]:
Enter size of pixel: [...]:
```

Computes the linear-model wavefront at the BUild element, performs a single far-field FFT propagation over transmission distance zElt(iElt), and pixelates the image onto the requested grid — the linear parallel of PIXel (Section 8.5). Requires LPErturb/LPRead first and nOutCord=5. The same dispatcher branch also serves the undocumented siblings LINtensity (point-image intensity) and LLO (log10 intensity). TEXt/BINary plot types write `filnam.lpxN`. SMACOS: LoadStack passes no pixel arguments, so the pixel prompts take their defaults. *Related:* LSPot, LOPd, PIXel, INTensity.

Diffraction

PROpagate

Minimum match: `PROpagate` — *needs:* [DIFF]

Note: NOT a command; propagation runs automatically inside any output command.

CLI suite: `COMPOSE_ADD_NOISE.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — not a command: `int_cmd` / `cfield_cmd` propagate.

Syntax: `PROPAGATE`; no prompt — propagates to the element of the last wavefront command.

Defaults: none.

```
MACOS>propagate
** Unknown command = PROPAGATE
```

Listed in HELp but NOT implemented: the command dispatcher (macos_cmd_loop.inc) has no PROPAGATE branch, so the keyword falls through to the unknown-command handler. Diffraction propagation is instead performed implicitly by the wavefront output commands — INTensity, PIXel, AMPlitude, PHAse, REAl, LOG, MTF, IMG, ADD and friends all call RunProp, which traces the beam and advances the complex-amplitude matrices to the requested element per each element's PropType (" Tracing N rays and propagating M grid points…"). The [DIFF] tag on those commands ("needs

PROpagate") therefore means "needs a propagated wavefront", which they establish themselves; results live in WFElt and re-propagation is skipped when the wavefront is already at the requested element. Discrepancy: help lists PROpagate as a command; the manual (Section 6) describes propagation only via the output commands. *Related:* INTensity, PIXel, BEAm, VECtor, SCAlar.

BEAm

Minimum match: `BEAm`

Set beam type (Gauss, plane, …).

CLI suite: `BEAM_GAUSS.jou`, `BEAM_gaussian.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `beam_set`.

Syntax: `BEAM [type] [params…]` — UNIFORM (no more prompts), GAUSSIAN (x and y waist radii), COS**POWER (radius, exponent), DIPOLE.

Defaults: type [UNIFORM]; the Gaussian waists [1.00000] each; the cosine radius [the aperture radius] and exponent [1.0].

```
MACOS>beam
Enter beam type (UNIFORM, GAUSSIAN, COS**POWER, DIPOLE): [UNIFORM]: gauss
Enter x beam waist radius: [...]: 0.3
Enter y beam waist radius: [...]: 0.3
```

Sets the source-beam amplitude profile used when the wavefront grid is initialized (default UNIFORM). GAUSSIAN prompts for x and y waist radii (default Aperture/4); COS**POWER prompts 'Enter cosine beam radius:' (default Aperture/2) and 'Enter cosine exponent:' (default 1); DIPOLE takes no further input. Types may also be entered as 1-4; anything else prints " Unknown beam type." and leaves the profile unchanged. Clears the propagate state so the next [DIFF] command re-propagates with the new profile. Requires a loaded Rx despite carrying no [Rx] tag in help. Manual Section 6.3.6 calls the Gaussian inputs "diameter … the standard deviation"; the prompts say waist radius. SMACOS: CARG(1)=type; DARG(1:2)=x/y waist radii (GAUSSIAN) or radius/exponent (COSPOWER); omitted for UNIFORM. *Related:* INTensity, POLarization, VECtor, SCAlar.

VECtor, SCAlar

Minimum match: `VECtor`

Vector vs scalar diffraction (with POLarized).

CLI suite: `HELP_POL.jou`, `MODE_scalar.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `vecdif_set`.

Syntax: `VECTOR` / `SCALAR`; no prompt.

Defaults: none. A session mode (SCALAR the default): survives loads, VECTOR is named in the LOAD notice.

A session mode (2026-10-11, Dave's ruling): the setting survives every LOAD until it is changed again – set once, before or after a deck is loaded. Each LOAD prints one line naming the session modes not at their defaults, e.g. `LOAD: session modes in force: POLARIZATION, VECTOR, CENTROID, NOLNEG, CIR` (nothing

when all are default); STATUS shows them too. RESET returns four of them to the defaults (no polarization, scalar, the chief-ray FEX axis, LNEG); the CIR/GIR palette is kept.

```
MACOS>vector
 Vector diffraction enabled
MACOS>scalar
 Scalar diffraction enabled
```

Toggle between vector and scalar diffraction for the propagation commands. VECtor requires polarization ray tracing to be on first (" Must turn on POLarization first"); POLarization itself enables vector diffraction automatically when the build supports it (mWF>=3), so VECtor is mainly for re-enabling after SCAlar. Scalar is the default and the only mode with polarization off (Section 6.3.4/6.3.5). Both commands clear the propagate state so the next [DIFF] command recomputes the wavefront; neither takes arguments. Scope: since Phase 3a Tranche 1 vector diffraction covers EVERY propagation leg – far-field, near-field sphere/plane, plane-to-plane, spatial-filter, Fresnel and the DFT legs – plus the diffraction-grid obscuration apply and the ray-side aperture/apodization masking, so a multi-leg chain preserves the vector field between legs. Two limits: the three wavefront planes are repurposed as Ex/Ey/Ez, so only ONE wavefront can be in flight (no multi-WF / COMPOSE); and between two physical legs the grid field is advanced by a scalar phase, which is exact only when the intervening elements are non-polarizing (Obscuring / Reference / FocalPlane – the coronagraph pupil->FPM->Lyot->focal case). A COATED or reflecting surface BETWEEN legs needs the per-ray running-Jones work of Tranche 2, not yet implemented. In vector mode AMPlitude/PHAse plot X, Y, Z field components and IMG imaging is unavailable. SMACOS: no arguments. *Related:* POLarization, NOPolarization, AMPlitude, PHAse.

COMpose

Minimum match: `COMpose`

Start a multi-object/color image.

CLI suite: `COMPOSE_ADD_NOISE.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `compose_start`.

Syntax: `COMPOSE [elt] [npix] [pixel]` — the element the image is composed at, the pixel count per side, the pixel size.

Defaults: elt [the last composed element, 0 before any]; npix [the diffraction grid size, 256 at model 256]; pixel [1.000000E-03] the first time, then the size in force.

```
MACOS>compose
Enter element where image is to be composed: [16]: 16
Enter number of pixels per side: [512]: 64
Enter size of pixel: [...]: 1.392542E-06
```

Defines a detector pixel grid (npix per side, capped at mPix; pixel size default 1e-3 base units) at the chosen element and zeroes PixArray, arming the accumulate mode: each subsequent ADD propagates the current beam, pixelates its intensity onto this grid and sums it in — the mechanism for multi-object / multi-wavelength (multi-color) composite images (change source, Flux, wavelength, field via MOD/FFP/PFP/PERturb between ADDs). Also the prerequisite for NOIse and GBLur. Segment / non-sequential elements are rejected ("COMPOSE: Invalid element type"); re-issuing prints " Overwriting previous COMPOSED image…" SMACOS: IARG(1)=element, IARG(2)=pixels per side, RARG(1)=pixel size. *Related:* ADD, DAdd, CADD, NOIse, PIXel.

ADD, DAdd

Minimum match: `ADD`

Add to / display composed image.

CLI suite: `COMPOSE_ADD_NOISE.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `compose_add`.

Syntax: `ADD [plot]` / `DADD`: ADD propagates and adds the image to the composite, then asks whether to plot it; DADD displays the composite.

Defaults: plot [YES]. Both need a COMPOSE ("Must COMpose before ADDing images").

```
MACOS>add
 Tracing ... rays and propagating ... grid points...
 Wavefront Propagation Data Summary:
 Wavelength= ...;  Source Flux= ...
 u-v plane diam= ...  du= ...
 x-y plane diam= ...  dx= ...
 Peak intensity= ...; Peak occurs at i= ..., j= ...
 Sum of intensity= ...
Image added. Plot composed image? [YES]:
```

ADD propagates the current beam to the COMpose element (no element prompt), converts the complex amplitude to intensity, pixelates it onto the COMpose grid and accumulates it into PixArray; requires COMpose first ("Must COMpose before ADDing images"). Repeated ADDs under different source/wavelength/ field settings build up composite images (Section 7.2.5). DAdd just re-displays the composed PixArray — e.g. after a STRetch or plot-type change — with no accumulation ("No data to display" if nothing has been pixelated yet). SMACOS: CARG(1)=YES/NO plot flag for ADD; DAdd's LoadStack pushes the same flag but the display path never reads it (the fresh stack per call makes this harmless). *Related:* COMpose, CADD, NOIse, STRetch, PIXel.

CADD

Minimum match: `CADD`

Add current intensity to composed.

CLI suite: `COMPOSE_ADD_NOISE.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — through `compose_*` (B.2, image-plane set).

Syntax: `CADD [elt] [plot]` — adds the complex amplitude at the element to the composite.

Defaults: elt [the last propagation element]; plot [YES].

```
MACOS>cadd
Enter number of element where data is to be generated: [...]: 16
 Tracing ... rays and propagating ... grid points...
 Wavefront Propagation Data Summary: ...
Complex amplitude added. Plot composed image? [YES]:
```

Coherent add — intended to resample and sum complex amplitudes (fields, not intensities) onto a common grid, the coherent counterpart of ADD. It is a STUB in this version: the dialog runs and the beam is propagated to the chosen element, but the regrid/resample-and-accumulate calls (CAReGrid / ResampAddCa) are commented out in the dispatcher, so nothing is added and nothing is plotted; only

internal bookkeeping flags toggle. Confirmed against macos_cmd_loop.inc. Use ADD for incoherent accumulation. SMACOS: no LoadStack entry — prompts take their defaults. *Related:* COMpose, ADD, DAdd.

NOIse, SEEd

Minimum match: `NOIse`

Image noise + RNG seed.

CLI suite: `ATMOS_placement.jou`, `ATMOS_seed.jou`, `COMPOSE_ADD_NOISE.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — image-plane post-processing (B.2, third).

Syntax: `NOISE [type]` — PHOton, BACkground, REAd or GAUssian noise on the composite; `SEED [n]` — the generator seed, 1 to 32 700.

Defaults: type [PHOTON]; seed [12345].

```
MACOS>noise
Enter noise type (PHOton, BACkground, REAd, GAUssian): [PHOTON]:
MACOS>seed
Enter noise seed (any integer 1 to 32,700): [12345]:
```

NOIse adds detector noise to the composed PixArray in place (requires COMpose/ADD first: "Must COMpose before adding noise."). PHOton replaces each pixel by a Poisson deviate of its value; BACkground adds Poisson background ('Enter expected number of background noise photons per pixel:'); REAd adds Gaussian read noise ('Enter standard deviation of noise (photo-electrons):' + 'Enter detector quantum efficiency:'); GAUssian adds zero-mean Gaussian flux noise ('Enter flux standard deviation:'). A fifth type, POIsson (expected photo-electrons + quantum efficiency), is accepted though not offered in the prompt. Repeat NOIse to stack effects. SEEd re-seeds the random-number generator used by the noise deviates (default 12345); re-issuing the same seed reproduces a noise realization. SMACOS: NOISE CARG(1)=type, then GAUSS: DARG(1)=sigma; READ: DARG(1)=sigma, DARG(2)=QE; POISSON: RARG(1)=expected, DARG(1)=QE; BACKGROUND: RARG(1)=expected. SEED: IARG(1)=seed. *Related:* COMpose, ADD, DAdd, PIXel.

STRetch

Minimum match: `STRetch`

LIN / LOG / SQRT display stretch.

CLI suite: `COMPOSE_ADD_NOISE.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — plot style (the bindings return the arrays).

Syntax: `STRETCH [LINEAR|LOG10|SQRT]` — the display stretch of intensity images.

Defaults: [LINEAR]. OPD maps are always linear.

```
MACOS>stretch
Enter image stretch type (LINEAR, LOG10, SQRT): [LINEAR]: log10
```

Sets the display stretch (intensity scaling) applied to intensity-type rasters — INTensity, PIXel and the composed image (ADD/DAdd/BLUr/GBLur) — to expand dynamic range for faint detail such as PSF wings. Plot titles and colorbar wedge labels follow the setting ('LOG10 Intensity', 'SQRT Pixel value',

…). OPD maps are unaffected (always linear), and the LOG command is a fixed log10-intensity display independent of STRetch. Unrecognized input silently selects LINEAR. The setting persists until changed or RESet. SMACOS: CARG(1)=type. *Related:* INTensity, PIXel, LOG, IMGmode.

Outputs (Ray-Trace & Diffraction)

OPD

Minimum match: `OPD` — *needs:* [Rx]

Wavefront error map.

CLI suite: `ATMOS_DMBUILD.jou`, `ATMOS_placement.jou`, `ATMOS_seed.jou`, `BINARY_CENTER.jou`, `COAT_legacy.jou`, `DOPD_identical.jou`, `EXPORT.jou`, `GLASS_mod_save.jou`, `GPERTURB.jou`, `GRIN_unsupported.jou`, `IndRef_stamp.jou`, `JOURNAL_EXE.jou`, `LONG_PATH.jou`, `MOD_spaces.jou`, `MRESET.jou`, `NEW.jou`, `NSRefractor_grid.jou`, `OBS_POLYGON.jou`, `OPD.jou`, `OPD_reference.jou`, `ORS_SRS.jou`, `OUTCORD_missing.jou`, `PERTURB.jou`, `PGD_reopen_units.jou`, `PLOT_TYPES.jou`, `PREAD.jou`, `PROPTYPE_numeric.jou`, `RRAY_PRAY_TPR_FSR.jou`, `SAVE_roundtrip.jou`, `SINT.jou`, `SYSPROP_after_OPD.jou`, `TRACE_keeps_rx.jou`, `UDSINIT.jou`, `WARN_obscured_elt.jou`, `WLENS_SWL.jou`, `ZABERR_ZRM_DOPD.jou`, `ZCOEF_zero_rays.jou`, `ZERN_refused.jou`, `ZernType_default.jou`, `ZernType_extfringe.jou`, `nGridpts_clamp.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `trace_rays`.

Syntax: `OPD [elt]` — the element the wavefront is evaluated at.

Defaults: elt [the element of the last OPD]: 0 before any (0 is refused: "Must load…" / invalid element), then sticky.

```
MACOS>opd
Enter element where OPD is to be evaluated: [0]: 3
 Tracing       150 rays...
 Compute time was    0.1445 sec
 RMS OPD error is ...
 P-V OPD error is ...
 Average OPD is   ...
```

Traces the full ray bundle to the element and plots the beam OPD map: per-ray pathlength minus the CHIEF RAY's pathlength (positive = travelled farther), in BaseUnits. The chief ray is the default reference; the aperture MEAN is the option (OPDREF MEAN per session, `UseChfRay4OPD= N` per prescription, `opd_ref` from the bindings) – see OPDREF for when it matters (segmented pupils) and for the `OPD reference:` line every run prints. The printed RMS is mean-removed and P-V is max-min, so neither depends on the reference; 'Average OPD' is the chief-minus-mean piston itself. The map is always the square aperture ray grid (see MAP) — use SPOt for true ray locations; obscured-ray plotting follows OBS. The map represents system wavefront error only at an exit pupil or proper reference surface; in a caustic, use a diffraction propagation plus PHAse instead (Section 5.2.2). Segment and non-sequential elements are rejected ("Invalid element type"). Raster plots are annotated with RMS/P-V; STRetch does not apply (OPD is always linear). TEXt/BINary/FITs/MAT plot types write `filnam.OPDn`. Prerequisite for ZCOef, ZABerr, ZRM and DOPD. SMACOS: IARG(1)=element; the map is returned in OPDMat. *Related:* OPDREF, SPOt, ZCOef, ZABerr, OBS, MAP.

OPDREF

Minimum match: `OPDREF`

OPD reference: CHIEF or MEAN (the chief ray is the default).

CLI suite: `OPD_reference.jou`, `ZCOEF_zero_rays.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — `opd_ref_set` / `opd_ref_get` (the loaded deck).

Syntax: `OPDREF [CHIEF|MEAN]` — the OPD reference: the chief ray's path or the aperture mean.

Defaults: [the reference in force], CHIEF after start-up; a session option set at allocation, so it survives loads (a deck's `UseChfRay4OPD= N` overrides it per load).

```
MACOS>opdref mean
OPD reference (CHIEF,MEAN): [CHIEF]: mean
 OPD reference: APERTURE MEAN (opt-in; the chief ray is the default)
MACOS>opd
 ...
 OPD reference: APERTURE MEAN (OPDREF MEAN)
 RMS OPD error is ...
```

Selects what every ray's optical path is measured AGAINST when OPD (and the OPD map a diffraction propagation fills) is computed: CHIEF the chief ray's own path (ray 1) – THE DEFAULT (the manual's definition, Section 5.2.2; all front ends since 2026-10-10) MEAN the mean path over every valid ray in the aperture The two maps differ by ONE constant per field (the chief-minus-mean piston), so RMS, P-V and 'Average OPD' as printed do NOT change – the per-ray map does. Use the chief ray on SEGMENTED pupils: under the mean reference, moving one segment shifts the mean and prints a spurious piston on every other segment (16.7 % of the poked segment's peak on the 7-segment e5hex1 deck); under the chief reference the untouched segments read exactly 0. OPDREF is a SESSION option: it survives later loads. Per prescription, the `UseChfRay4OPD= Y|N` keyword overrides it for that deck (SAVE writes `UseChfRay4OPD= N` when the mean is in force, nothing for the default); from the bindings, `macos.opd_ref` / `pymacos.opd_ref` act on the loaded deck. Every OPD run prints the reference it used and where it came from: `(default)`, `(UseChfRay4OPD= Y|N)`, `(OPDREF CHIEF|MEAN)`, `(opd_ref_set)`, `OPDRefRayLen= (Rx)`; if the chief ray is geometrically lost the engine falls back to the mean and prints `APERTURE MEAN (chief ray lost)` – no error. An OBSCURED chief ray still serves (the reference needs the ray's geometry, not its flux). Consequence for consumers: under the chief reference a valid ray whose path equals the chief's reads EXACTLY 0, the same value OPDMat holds where there is no ray – use the validity mask (`macos.opd_mask` / `pymacos.opd_mask`), never `W ~= 0`, to find the pupil. SMACOS: not dispatched; use the api `opd_ref_set`. Note the dispatcher tests OPDREF before OPD (prefix match). *Related:* OPD, CHIefray, SPCenter.

SPOt

Minimum match: `SPOt` — *needs:* [Rx]

Spot diagram.

CLI suite: `ATMOS_placement.jou`, `ATMOS_seed.jou`, `OBS_NONE.jou`, `SPCENTER.jou`, `SPOT_text.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `spot_cmd`.

Syntax: `SPOT [elt] [Tout|Enter|TElt|Beam]` — the element and the output frame of the spot diagram.

Defaults: elt [the element of the last SPOT, 0 before any]; frame [Tout] when the deck has `nOutCord= 5`, else Beam; a TEXT plot type that finds `<deck>.spotN.txt` asks "Overwrite? [YES]". SPCENTER chooses the centring (element vertex or chief ray).

```
MACOS>spot
Enter element where spot diagram is to be computed: [0]: 4
Enter output coordinate option (Tout, Enter, TElt or Beam): [Tout]:
```

```
STOP cmd: Tracing      150 rays...
Compute time was ...
       Chief ray location: x= ... y= ...
 Centroid of spot diagram: x= ... y= ...
```

Traces the full bundle and scatter-plots the ray pierce points projected onto an x-y frame at the element. Coordinate options: Tout (the 5x7 output matrix) and TElt (the element's local frame) are offered only when nOutCord=5 — otherwise the prompt reads '(Enter or Beam)' with default Beam; Enter lets you type xOut/yOut vectors; Beam computes a chief-ray local frame on the fly (aborts if that fails). Spot offsets are referenced to the element vertex or to the chief ray per SPCenter; obscured-ray selection follows OBS. The plot is annotated with the RMS spot radius about the centroid. TEXt/BINary/MAT write `filnam.spotN`. Quirk: the trace banner prints " STOP cmd: Tracing N rays…" (shared with the STOP handler). Segment and non-sequential elements are rejected. SMACOS: IARG(1)=element, CARG(1)=coordinate option; spot data in RaySpot. *Related:* OPD, OBS, SPCenter, MAP, LSPot.

ZABerr

Minimum match: `ZABerr` — *needs:* [Rx]

Zernike OPD aberration.

CLI suite: `DOPD_identical.jou`, `ZABERR_ZRM_DOPD.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — Zernike / analysis set (B.2, second).

Syntax: `ZAB [n] <modes> [save]` — the number of Zernike modes, their indexes (one line), and whether the aberration-removed map becomes the new OPD.

Defaults: n [1]; the indexes have no default; save [no]. Needs an OPD; DOPD after it reads the sentinel −1 (1.2 (k)).

```
MACOS>zaberr
 ** Computing OPD with selected Zernike modes (Noll ordered)
Enter number of Zernike modes: [1]: 3
Enter Zernike indexes: [no default]: 4,5,6
  RMS WFE (after ZAB) = ...
Save new OPD (no,yes): [no]:
```

Extracts the Zernike-aberration content of the current OPD: least-squares fits the OPDMat from the last OPD command ("Must run OPD command first") over the beam mask, sums the selected modes only, plots that modal OPD map and prints its RMS. Modes are Noll-ordered; entering a NEGATIVE count -N instead prompts 'Enter starting Zernike mode index:' and uses N consecutive modes. Answering yes to "Save new OPD" replaces OPDMat with the modal map (and invalidates the trace/build state). Complement of ZRM, which removes the fitted modes. Uses the element's user-defined Zernike frame (center/axes/radius) when the Rx provides one, else the beam grid. Interactive CLI only. *Related:* ZCOef, OPD, ZRM.

ZCOef

Minimum match: `ZCOef` — *needs:* [Rx]

Zernike coefficient extraction.

CLI suite: `ZCOEF_zero_rays.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — Zernike / analysis set (B.2, second).

Syntax: `ZCO [n] <start>` — the number of modes fitted and the first mode index.

Defaults: n [1]; the start index has no default. The fit runs over the valid rays only (`opdValid`), never over the zero-filled ones.

```
MACOS>zcoef
 ** Computing OPD Zernike fit coeffs (Noll ordered) **
Enter number of Zernike modes: [1]: 6
Enter Zernike indexes: [no default]: 1,2,3,4,5,6
 Zernike mode coeffs (OPD best fit):
  ...mode names / indices...
  ...coefficients...
```

Least-squares fits the selected Zernike modes to the current OPDMat (from the last OPD command — "Must run OPD command first") over the beam mask and prints the coefficient table (named low-order modes, numeric indices above); the OPD itself is not modified and nothing is plotted. Same mode-selection dialog as ZABerr: Noll-ordered indices, or a NEGATIVE count -N followed by 'Enter starting Zernike mode index:' for N consecutive modes. Uses the element's user-defined Zernike frame when the Rx provides one, else the beam grid. Interactive CLI only. *Related:* ZABerr, OPD, ZRM.

LOS

Minimum match: `LOS`

Line-of-sight output.

CLI suite: `MTF_LOS_ROWCOL.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — line of sight (the harvests' `compute_los` reads it from spots).

Syntax: `LOS [elt]` — the element the line of sight is evaluated at.

Defaults: elt [the last LOS element].

```
MACOS>los
Enter element where LOS is to be evaluated: [...]:
 Tracing ... rays and propagating ... grid points...
 Compute time was ...
  System Line of Sight (X-tilt, Y-tilt (radian)) = ...
  System Line of Sight (PSF centroid at detector (m ... )) = ...
```

Computes the system line of sight from the wavefront at the chosen element (default nElt-1, nominally the exit pupil; out-of-range input silently reverts to it), per Mahajan's "Line of Sight of an Aberrated Optical System" pupil-function formulation: the amplitude-weighted mean OPD gradient gives the LOS tilt (radians), which scaled by the propagation distance zElt gives the PSF-centroid displacement at the detector (base units). Propagates the beam first if needed (RunProp). Code quirks: the x-gradient loop omits the amplitude weighting the y-loop applies, and the angular line prints its two components in the opposite order (losAng(2), losAng(1)) to the centroid line — take the axis labels with caution. SMACOS: no LoadStack entry — the element prompt takes its default. *Related:* OPD, INTensity, FEXit, CENTroid.

METcalc

Minimum match: `METcalc`

Metrology beam output.

CLI suite: `METCALC.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — `met_calc`, `met_get`, `met_geom_get`.

Syntax: `METCALC`; no prompt — the metrology beams of the deck's `Met*` keys.

Defaults: none. The beam length does not follow a despace of its target element (1.2 (n), open).

```
MACOS>metcalc
 ** All metrology beam lengths updated
```

Computes the lengths of all defined metrology beams: the straight-line distances between metrology points on each instrumented surface and the corresponding points on its target surface, as defined in the Rx (per-element nMetPos= point list plus tMetSrf= target element). Results go to the metMeasBuf array for programmatic retrieval (GMI/bindings). With `ShowMetData= YES` in the Rx each beam prints individually (' Met beam I length from Elt .. Pos .. to Elt .. Pos .. = …'); otherwise only the summary line above. Run after PERturb to simulate a metrology-gauge reading of element motions. No arguments; requires only a loaded Rx. *Related:* PERturb, SUMmarize.

INTensity

Minimum match: `INTensity` — *needs:* [DIFF]

Wavefront intensity.

CLI suite: `BEAM_GAUSS.jou`, `BEAM_gaussian.jou`, `BINARY_CENTER.jou`, `EFL_SYSPROP.jou`, `IMG.jou`, `INT_LOG.jou`, `ORS_SRS.jou`, `SYSPROP_after_OPD.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `int_cmd`, `cfield_cmd`.

Syntax: `INT [elt]` — the element the field is propagated to.

Defaults: elt [the last propagation element, nElt after a load].

```
MACOS>int
Enter number of element where data is to be generated: [...]: 6
 Tracing ... rays and propagating ... grid points...
 Compute time was ...
 Wavefront Propagation Data Summary:
 Wavelength= ...;  Source Flux= ...
 u-v plane diam= ...  du= ...
 x-y plane diam= ...  dx= ...
 Peak intensity= ...; Peak occurs at i= ..., j= ...
 Sum of intensity= ...
```

Propagates the beam to the element — combined ray trace plus diffraction per each element's PropType, re-using the stored wavefront when it is already at or upstream of the target — and plots the intensity |CA|^2. There is no separate propagate step: INTensity (like all [DIFF] commands) performs the propagation itself via RunProp. The summary gives start (u-v) and end (x-y) grid diameter/spacing; the Fresnel propagators also print per-leg "lin=" ray-grid linearity percentages. Segment and non-sequential elements are rejected. STRetch selects linear/log10/sqrt display, IMGmode the raster polarity; the plot is annotated with the peak pixel. TEXt/BINary/FITs/MAT write `filnam.intN`. SMACOS: IARG(1)=element; intensity available programmatically. *Related:* PIXel, LOG, AMPlitude, PHAse, STRetch, BEAm.

PIXel

Minimum match: `PIXel` — *needs:* [DIFF]

Pixelated intensity output.

CLI suite: `AMP_PHASE_REAL_PIX.jou`, `PLOCATE_NOPLOC.jou`, `WINDOW_GBS.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — the INT block: `int_cmd` / `int_get`.

Syntax: `PIX [elt] [npix] [pixel]` — the element, pixels per side, the pixel size.

Defaults: elt [the last propagation element]; npix [the diffraction grid size, 256 at model 256]; pixel [the propagated field's own sample, printed]; an existing TEXT file asks "Overwrite? [YES]".

```
MACOS>pix
Enter number of element where data is to be generated: [...]: 6
 ...ray-trace / propagation summary as for INTensity...
Enter number of pixels per side: [...]: 64
Enter size of pixel: [...]:
 Peak intensity= ...; Peak occurs at i= ..., j= ...
 Sum of intensity= ...
```

Simulates a pixel-array detector (CCD): propagates like INTensity, then area-averages the intensity onto an npix-by-npix grid of square pixels. Default pixel size is the wavefront grid spacing dx at the element ("WARNING: PIXELS TOO SMALL" below dx/2); npix is capped at mPix. The pixel grid is centered on the chief ray unless WINdow / PLOcate has fixed it in detector coordinates. The result replaces PixArray and clears any COMposed image (" Overwriting previous COMPOSED image…"). The plot is annotated with the peak pixel; STRetch applies. TEXt/BINary write `filnam.pixN`. SMACOS (LoadStack keyword PIXILATE): IARG(1)=element, IARG(2)=pixels per side, RARG(1)=pixel size. *Related:* INTensity, COMpose, ADD, WINdow, STRetch, LPIxilate.

AMPlitude

Minimum match: `AMPlitude` — *needs:* [DIFF]

Amplitude output.

CLI suite: `AMP_PHASE_REAL_PIX.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — the INT block: `cfield_cmd` / `cfield_get`.

Syntax: `AMP [elt]` — the element whose field amplitude is shown.

Defaults: elt [the last propagation element].

```
MACOS>amp
Enter number of element where data is to be generated: [...]: 5
 ...propagation summary...
 Peak amplitude= ...; Peak occurs at i= ..., j= ...
```

Plots the modulus of the complex amplitude at the element, propagating there first if needed (see INTensity for the shared dialog and summary). In vector-diffraction mode the X, Y and Z field components are plotted in turn (file tags .ampX/.ampY/.ampZ); scalar mode gives one plot (.amp). TEXt/BINary/FITs/MAT write `filnam.ampN`. SMACOS: IARG(1)=element. *Related:* PHAse, REAl, INTensity, VECtor.

PHAse, PH

Minimum match: `PHAse` — *needs:* [DIFF]

Phase output.

CLI suite: `AMP_PHASE_REAL_PIX.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — the INT block: `cfield_cmd` / `cfield_get`.

Syntax: `PHASE [elt]` (also `PH`) — the field phase at the element.

Defaults: elt [the last propagation element].

```
MACOS>phase
Enter number of element where data is to be generated: [...]: 5
 ...propagation summary...
```

Plots the phase (argument) of the complex amplitude at the element, propagating there first if needed. In vector-diffraction mode the X, Y and Z components plot in turn (.phaseX/.phaseY/.phaseZ); scalar mode one plot (.phase). Phase at a well-chosen reference surface after a diffraction propagation is the recommended alternative to a geometric OPD map in caustic regions (Section 5.2.2 note). SMACOS: no LoadStack entry — the element prompt takes its default (the current wavefront element, else the last element). *Related:* AMPlitude, REAl, OPD, VECtor.

REAl

Minimum match: `REAl` — *needs:* [DIFF]

Real & imaginary wavefront.

CLI suite: `AMP_PHASE_REAL_PIX.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — the INT block: `cfield_cmd` / `cfield_get`.

Syntax: `REAL [elt]` — the real part of the field at the element.

Defaults: elt [the last propagation element].

```
MACOS>real
Enter number of element where data is to be generated: [...]: 5
 ...propagation summary...
```

Plots the real part and then the imaginary part of the complex amplitude at the element (two plots per invocation; file tags .real and .imag), propagating there first if needed. The manual (Section 7.2.12) refers to a separate IMAGINARY command, but no such command exists in the dispatcher — REAl produces both components. SMACOS: IARG(1)=element. *Related:* AMPlitude, PHAse, INTensity.

LOG

Minimum match: `LOG` — *needs:* [DIFF]

Log10 intensity.

CLI suite: `INT_LOG.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **direct setter** — the INT block: `int_cmd` / `int_get`.

Syntax: `LOG [elt]` — the log10 intensity at the element (not the journal; that is JOU).

Defaults: elt [the last propagation element].

```
MACOS>log
Enter number of element where data is to be generated: [...]: 16
 ...propagation summary...
 Peak intensity= ...; Peak occurs at i= ..., j= ...
 Sum of intensity= ...
```

Log10-intensity display of the wavefront at the element (title ‘Log10 Wavefront Intensity’), propagating there first if needed — a fixed log stretch independent of the STRetch setting, useful for PSF wings and dark zones. Do not read LOG as “start logging”: the session transcript command is JOUrnal. TEXt/BINary write `filnam.logN`. The linear-model counterpart is the undocumented LLO command (see LPIxilate). SMACOS: IARG(1)=element. *Related:* INTensity, STRetch, JOUrnal.

MTF, CMTF

Minimum match: `MTF` — *needs:* [DIFF]

Modulation transfer function.

CLI suite: `MTF_LOS_ROWCOL.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **missing** — image-plane post-processing (B.2, third).

Syntax: `MTF [elt]` / `CMTF [elt]` — the modulation transfer function at the element (CMTF the cumulative).

Defaults: elt [the last propagation element].

```
MACOS>mtf
Enter number of element where data is to be generated: [...]: 16
 ...propagation summary...
Use ROW or COL command to plot 1D MTF in each dimension
```

MTF propagates to the element, computes the intensity and Fourier-transforms it into the modulation transfer function, plotted against spatial frequency in cycles per BaseUnit. A GRay plot type is switched to SLIce for the display (no gray image for MTF); use ROW/COLumn for 1-D cuts. The CLI branch also derives the system EFL/focal ratio for sampling checks. CMTF (all four characters required) computes the 2-D MTF the other way — autocorrelating the complex field at the LAST element (no element prompt, ” Computing 2D MTF …“) into the MTFMat2 array — and produces no plot or file; retrieval is programmatic. SMACOS: MTF has no LoadStack entry (element prompt takes its default) and its plot stage is CLI-only — the MTF data is computed into the shared display buffer. *Related:* INTensity, ROW, COLumn, EFL.

IMG

Minimum match: `IMG` — *needs:* [DIFF]

Simulate imaging of the ObjFile object map.

CLI suite: `IMG.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **direct setter** — the INT block: `int_cmd` / `int_get`.

Syntax: `IMG [elt] [INC|COH]` — the element (must be the last) and incoherent or coherent imaging of the `ObjFile=` map.

Defaults: elt [the last propagation element]; mode [INC]. Refuses without an `ObjFile=` and at any element but the last. Both modes are flux-normalized: a one-pixel object images to the PSF itself (the same peak and sum in INC and COH), and an extended object’s coherent flux lies between 1 and N times its incoherent flux for N in-phase pixels (`IMG.jou`: 24.7 for an 8 × 8 square on the Cassegrain; fixed 2026-10-18, 1.2 (s)).

```
MACOS>img
Enter number of element where data is to be generated: [...]: 16
 ...propagation summary...
```

```
 Simulating incoherent or coherent imaging ...
INCoherent or COHerent imaging? [INC]:
```

Simulates extended-object imaging: combines the propagated focal-plane field/PSF with a user-supplied object intensity map, incoherently (default) or coherently, and displays the result (‘Incoherent Imaging’ / ‘Coherent Imaging’, file tag .intensity). The object map is loaded via the `ObjFile=` Rx keyword — without it “* Object intensity input not available”. Must be evaluated at the LAST element (“Imaging must be on focal plane!”) and is unavailable in vector-diffraction mode. The object map is a text matrix of the diffraction grid’s size (model size squared), read at LOAD. Measured (Cassegrain, an 8 x 8 square of ones at the center of a 256 x 256 map): the incoherent image is the PSF convolved with the object — its sum is the PSF’s sum times the 64 object pixels (2.1204e10 = 64 x 3.3131e8), peak 2.24e8; the coherent image of the same unit-amplitude object has peak 5.3e19 and sum 2.2e21, 1e9 times the incoherent — an unnormalized transform in the coherent branch of SysImging, recorded as PLAN_CONSOLIDATION 1.2 (s). Discrepancy: the help one-liner (“load saved image data”) does not describe what the command does. SMACOS: no LoadStack entry — prompts take their defaults (last element, incoherent). *Related:* INTensity, PIXel, COMpose.

Plot Style

GRay

Minimum match: `GRay`

Gray-scale surface plot.

CLI suite: `PLOT_TYPES.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **CLI-only** — plot style (the bindings return the arrays).

Syntax: `GRAY`; no prompt — selects the plot type for the next map or image (“Plot type set to GRAY”).

Defaults: none. The plot type is a session setting that RESET does not restore (1.2 (i)); COLUMN and ROW ask which column / row when the next map is drawn.

```
MACOS>gray
 Plot type set to GRAY
```

No further dialog. Selects gray-scale raster rendering for subsequent surface-data plots (OPD, INTensity, SPOt, …) — the startup default plot type. IMGmode sets the raster polarity, STRetch the intensity scaling, and CIR/GIR the color-vs-gray palette. The setting persists until another plot type (or NONe) is chosen. The minimum match is the two-letter `GR`. *Related:* WIRe, SLIce, CONtour, NONe, IMGmode, CIR.

WIRe

Minimum match: `WIRe`

Wireframe plot.

CLI suite: `PLOT_TYPES.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **CLI-only** — plot style (the bindings return the arrays).

Syntax: `WIRE`; no prompt — selects the plot type for the next map or image (“Plot type set to WIRE”).

Defaults: none. The plot type is a session setting that RESET does not restore (1.2 (i)); COLUMN and ROW ask which column / row when the next map is drawn.

```
MACOS>wire
 Plot type set to WIRE
```

Selects the wireframe plot type: the data array is plotted as a 3-D surface — nearest-neighbor grid points joined, perspective added, hidden lines removed (manual Section 7.4). There is no axis scale; use SLIce for a scaled surface view. Applies to subsequent surface-data plots until changed. *Related:* SLIce, GRay, CONtour, COLumn.

SLIce

Minimum match: `SLIce`

Slice surface plot.

CLI suite: `PLOT_TYPES.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — plot style (the bindings return the arrays).

Syntax: `SLICE`; no prompt — selects the plot type for the next map or image ("Plot type set to SLICE").

Defaults: none. The plot type is a session setting that RESET does not restore (1.2 (i)); COLUMN and ROW ask which column / row when the next map is drawn.

```
MACOS>slice
 Plot type set to SLICE
```

Selects the slice plot type: a 3-D surface rendering like WIRe but with axis scales, so data values can be read directly from the plot (manual Section 7.4). Applies to subsequent surface-data plots until changed. *Related:* WIRe, GRay, CONtour, ROW.

COLumn

Minimum match: `COLumn`

Column line plot.

CLI suite: `MTF_LOS_ROWCOL.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — plot style (the bindings return the arrays).

Syntax: `COLUMN`; no prompt — selects the plot type for the next map or image ("Plot type set to COLUMN").

Defaults: none. The plot type is a session setting that RESET does not restore (1.2 (i)); COLUMN and ROW ask which column / row when the next map is drawn.

```
MACOS>column
 Plot type set to COLUMN
MACOS>opd
 ...
Enter number of column to be plotted: [...]:
```

Selects the column plot type: a line plot along one column of the data matrix. The column number is prompted when a plot is actually produced (default = the center column); out-of-range input skips the plot. SMACOS: LoadStack queues IARG(1) and a literal 0, but the command itself reads nothing — the column prompt happens at plot time. *Related:* ROW, SLIce, MTF.

CONtour

Minimum match: `CONtour`

Contour surface plot.

CLI suite: `PLOT_TYPES.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — plot style (the bindings return the arrays).

Syntax: `CONTOUR`; no prompt — selects the plot type for the next map or image ("Plot type set to CONTOUR").

Defaults: none. The plot type is a session setting that RESET does not restore (1.2 (i)); COLUMN and ROW ask which column / row when the next map is drawn.

```
MACOS>contour
 Plot type set to CONTOUR
```

Selects the contour-map plot type: 16 contour levels connect points of equal value, an elevation-map view of the data (manual Section 7.4). Applies to subsequent surface-data plots until changed. *Related:* GRay, WIRe, SLIce.

IMGmode

Minimum match: `IMGmode`

Toggle image polarity (NEG=astro, POS=conventional).

CLI suite: `IMG.jou`, `MTF_LOS_ROWCOL.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — plot style (the bindings return the arrays).

Syntax: `IMGMODE [NEG|POS]` — raster polarity: NEG (astronomy, large values dark) or POS (conventional).

Defaults: [NEG] — a session setting (`ifAstroMode` in plot_mode_mod) that survives loads. Dispatched before IMG (the 3-character match would otherwise take it).

```
MACOS>imgmode
Raster polarity (NEG=astro/POS=conventional): [POS]: neg
 IMGMODE = NEG (astronomy: large -> dark)
```

Sets the polarity of raster (GRay-type) image displays: NEG (or ASTRO) renders large values dark, astronomy-style; POS (or CONV) renders large values bright. The prompt default is the current setting. Non-raster plot types (SLIce, WIRe, CONtour, COLumn, ROW, SPOt) are unaffected, and unrecognized input changes nothing ("IMGMODE: unrecognized option, no change"). SMACOS: no LoadStack entry — the prompt takes its default, leaving the mode as-is. *Related:* GRay, STRetch, CIR.

CIR, GIR

Minimum match: `CIR`

Color / gray image rendering.

CLI suite: `MODE_cir.jou`, `MODE_load_notice.jou`, `ZABERR_ZRM_DOPD.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — plot style (the bindings return the arrays).

Syntax: `CIR` / `GIR`; no prompt — colour or gray image rendering.

Defaults: none. A session mode (GIR the default): survives loads; CIR is named in the LOAD notice.

A session mode (2026-10-11, Dave's ruling): the setting survives every LOAD until it is changed again – set once, before or after a deck is loaded. Each LOAD prints one line naming the session modes not at their defaults, e.g. `LOAD: session modes in force: POLARIZATION, VECTOR, CENTROID, NOLNEG, CIR` (nothing when all are default); `STATUS` shows them too. `RESET` returns four of them to the defaults (no polarization, scalar, the chief-ray FEX axis, LNEG); the CIR/GIR palette is kept.

No dialog. CIR turns color image rendering on (" Color image rendering is on"); GIR reverts to gray-scale (" Gray image rendering is on"). Affects raster image displays (GRay plot type). Interactive CLI only. *Related:* GRay, IMGmode, PGP.

PGP

Minimum match: `PGP`

PGPLOT panel layout (1,2,3,4,9).

CLI suite: `MTF_LOS_ROWCOL.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — plot style (the bindings return the arrays).

Syntax: `PGP [panels]` — 1, 2, 3 or 4 display panels per page.

Defaults: [1].

```
MACOS>pgp
Enter PGPLOT display panel option (1,2,3 or 4): [1]: 4
  Use 2 x 2 PGPLOT panels
```

Splits the graphics window into panels: 1 (single), 2 (1x2), 3 (1x3), 4 (2x2) or 9 (3x3) — 9 is accepted although the prompt text doesn't offer it (the HELP line does). Invalid input reverts to a single panel. Forces the graphics device to reinitialize on the next plot. Interactive CLI only. *Related:* PGD, CIR, GRay.

File Output

TEXt

Minimum match: `TEXt`

ASCII print output.

CLI suite: `ATMOS_placement.jou`, `ATMOS_seed.jou`, `BINARY_CENTER.jou`, `BUILD_refused_export.jou`, `EXPORT.jou`, `IMG.jou`, `PLOCATE_NOPLOC.jou`, `PLOT_TYPES.jou`, `SPCENTER.jou`, `SPOT_text.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — CLI file-output mode (the bindings return the arrays).

Syntax: `TEXT`; no prompt — the plot type that writes a formatted text array instead of drawing.

Defaults: none. The file is `<deck>.<map>N.<ext>` beside the deck (e.g. `Cassegrain.in.OPD3.txt`); an existing file asks "Overwrite? [YES]".

```
MACOS>text
 Plot type set to TEXT
MACOS>intensity
 ...
 Writing Wavefront Intensity, Elt=5
 FORMATTED File=CassWithExitPupil.int5.txt
```

Redirects subsequent data outputs to formatted ASCII files instead of screen plots: single-precision values, one line per grid row, named `filnam.<tag><elt>.txt` where the tag identifies the data (.int, .amp, .phase, .real/.imag, .spot, .OPD, …). An existing file prompts “Image file … exists. Overwrite?” (default YES; answering no prompts ‘Enter new file name:’). Persists until another plot type is chosen. *Related:* BINary, FITs, MAT, NONe, GRay.

BINary

Minimum match: `BINary`

Binary image file.

CLI suite: `BINARY_CENTER.jou`, `PLOT_TYPES.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **CLI-only** — CLI file-output mode (the bindings return the arrays).

Syntax: `BINARY`; no prompt — the plot type that writes a binary array instead of drawing.

Defaults: none. The file is `<deck>.<map>N.<ext>` beside the deck (e.g. `Cassegrain.in.OPD3.txt`); an existing file asks “Overwrite? [YES]”.

```
MACOS>binary
 Plot type set to BINARY
```

Redirects subsequent data outputs to flat binary files (direct-access records, one per grid row, 4 bytes per value) named `filnam.<tag><elt>` with no added extension, for import into other image/signal-processing software. Same overwrite dialog as TEXt. Persists until another plot type is chosen. *Related:* TEXt, FITs, MAT.

FITs, WFIts

Minimum match: `FITs`

FITS / wide-FITS image file.

CLI suite: `PLOT_TYPES.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **CLI-only** — CLI file-output mode (the bindings return the arrays).

Syntax: `FITS` / `WFITS`; no prompt — the plot type that writes a FITS file.

Defaults: none. `<deck>.<map>N.fit`; long deck paths are no longer refused (the 18-character FILENAME card limit is gone).

```
MACOS>fits
 Plot type set to FITS
MACOS>wfits
Save OPD or Intensity (OPD,INT): [OPD]:
  Dumping current OPD into FITS file Opd.fits
```

FITs redirects subsequent data outputs to FITS files named `filnam.<tag><elt>.fit`. WFIts (interactive CLI only) is instead a one-shot dump of the CURRENT data: OPD writes OPDMat to `Opd.fits`, INT writes the intensity to `Int.fits`; it requires a prior OPD or INt command (“OPD data is not available, run OPD command”). *Related:* TEXt, BINary, MAT, OPD, INTensity.

MAT

Minimum match: `MAT`

MATLAB .mat file.

CLI suite: `PLOT_TYPES.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — CLI file-output mode (the bindings return the arrays).

Syntax: `MAT`; no prompt — the plot type that writes a MATLAB file — "MAT output is not built into this macos" on this build instead of drawing.

Defaults: none. The file is `<deck>.<map>N.<ext>` beside the deck (e.g. `Cassegrain.in.OPD3.txt`); an existing file asks "Overwrite? [YES]".

```
MACOS>mat
 Plot type set to MAT
```

Redirects subsequent data outputs to MATLAB-readable files named `filnam.<tag><elt>.mat` (the manual's ".dat" is stale — the code appends ".mat"). Same overwrite dialog as TEXt. Persists until another plot type is chosen. *Related:* TEXt, BINary, FITs, GETMatlabmatrix.

GETMatlabmatrix

Minimum match: `GETMatlabmatrix`

(currently disabled: handler is a no-op stub) (smacos only).

CLI suite: `GETMATLABMATRIX.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — disabled (a no-op stub); the bindings return arrays.

Syntax: `GETMATLABMATRIX`; no prompt — SMACOS builds only.

Defaults: none. The CLI answers "Command not understood … Unknown command".

SMACOS builds only (#ifdef SMACOS_CMD) — the CLI does not dispatch it ("Command not understood … ** Unknown command = getmatlabmatrix", pinned by the suite). The handler body is entirely commented out, so the command is currently a no-op stub: it matches, does nothing, and returns to the prompt. Discrepancy: the help line promises fetching a named matrix into MATLAB; no such transfer is implemented — data exchange happens through the SMACOS call-line arguments instead. *Related:* MAT, EXPort.

Window & Post-Processing

PLOcate

Minimum match: `PLOcate`

Set plot pixel-array location.

CLI suite: `PLOCATE_NOPLOC.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — plot pixel-array location.

Syntax: `PLOC [frame] [pixel] [vx,vy] [wx,wy]` (also `WINDOW`) — the output frame (Tout, Enter or Beam), the pixel size, the element vertex in pixels, the window location in pixels.

Defaults: frame [Tout] when the deck has `nOutCord= 5`, else Beam; pixel [0.00000] (zero clears the offsets); vertex and window [0.00000,0.00000].

```
MACOS>window
Enter output coordinate option (Tout, Enter or Beam): [Tout]:
Enter pixel size for placing window: [0.]: 1.392542d-6
Enter pixel coords of element vertex (x,y): [0.,0.]:
Enter window location in pixel coords (x,y): [0.,0.]:
```

PLOcate and WINdow are the same command (one dispatcher branch). Turns on the pixel-locate option: the PIXel / COMposed detector array is fixed in detector coordinates (element-vertex origin, window center, pixel size), so field-angle changes move the image across the array instead of the image always landing centered. The Tout option is offered only when nOutCord=5 — otherwise the prompt reads '(Enter or Beam)' with default Beam; Enter prompts for explicit 'Enter xOut (x,y,z)' / 'Enter yOut (x,y,z)' axis vectors, Beam derives a chief-ray local frame at the last element. A pixel size of 0 (the default) zeroes all the offsets. Measured (Cassegrain, 64 x 64 x 2e-6 PIXel array, window at pixel 10,10): the PSF peak lands at row 22, column 22 of the array and the central pixel reads 2.87e4; after NOPLOC the peak is at (32,32), the central pixel 2.41e7. The propagation's own "Peak occurs at i= 129, j= 129" is the diffraction grid and does not move. Cleared by NOPLOC. SMACOS (keyword PLOCATE or WINDOW): CARG(1)=coordinate option, DARG(1)=pixel size, DARG(2:3)=element vertex, DARG(4:5)=window center. *Related:* NOPLOC, PIXel, COMpose, SPOt, FEXit.

NOPLOC

Minimum match: `NOPLOC`

Clear plot location.

CLI suite: `PLOCATE_NOPLOC.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `window_off`.

Syntax: `NOPLOC`; no prompt.

Defaults: none.

```
MACOS>noploc
 Images will be placed at center of pixel array
```

No dialog. Turns the pixel-locate option back off, undoing PLOcate / WINdow. *Related:* PLOcate, PIXel, COMpose.

PWIn

Minimum match: `PWIn`

Launch external he.py plot GUI.

CLI suite: `PWIN.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — external plot GUI.

Syntax: `PWIN`; no prompt.

Defaults: none; says whether he.py was found on the PATH.

```
MACOS>pwin
 PWIN: he.py not found on PATH; nothing launched
```

Interactive CLI only. No dialog: checks the PATH for the external Python/Tcl-Tk GUI script "he.py" and says what it found; when it is there, launches `he.py &` in the background ("PWIN: launching he.py in the background") — a vestigial hook; nothing useful happens without an executable he.py on the PATH.

Before 2026-10-17 the shell was handed the literal `!he.py&` (never launched anything) and its “not found” arrived after the next prompt (PLAN_CONSOLIDATION 1.2 (q)). Discrepancy: the help line “plot window definition” describes the PLOcate/WINdow dialog, not this command. *Related:* PLOcate, PGD.

SZCo

Minimum match: `SZCo`

Select ZCOef Zernike basis (Cir / Ann / Hex).

CLI suite: `ZABERR_ZRM_DOPD.jou`, `ZERN_refused.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **missing** — Zernike / analysis set (B.2, second).

Syntax: `SZCO [Cir|Ann|Hex]` — the Zernike basis ZCO fits on.

Defaults: [Cir]. A per-load setting (`zcoType`), since the basis needs the deck’s aperture.

```
MACOS>szco
Choose Zernike type for ZCO command (Cir,Ann,Hex): [Cir]:
 -- Set to using NormNoll
```

Needs a loaded Rx. Selects the Zernike basis family used by the ZCOef fit: Cir (Noll-normalized circular), Ann (annular) or Hex (hexagonal), confirmed as NormNoll / NormAnnular / NormHex. Unknown input: “** Unknown Zernike type”. Discrepancy: the help line calls SZCo “sizing / coordinate options”; the code sets the ZCOef Zernike type. SMACOS: no LoadStack entry. *Related:* ZCOef, ZABerr.

GBS

Minimum match: `GBS` — *needs:* [Rx]

Geometric beam footprint at an element.

CLI suite: `WINDOW_GBS.jou` (`cli_tests/cmd/`; each journal is the entry’s example and its regression test).

Binding: **missing** — field / beam set.

Syntax: `GBS [first] [last]` — the element range whose beam sizes go to beamSize.txt.

Defaults: first [1]; last [1]: literals. The file lands in the working directory.

```
MACOS>gbs
Enter first element number: [1]: 0
Enter last element number: [1]: 4
 iElt =  0, Source aperture = ...
 iElt = 1 (PM, Reflector)  Beam diameter = ...
 ==> Result is saved in file beamSize.txt
```

Reports the geometric beam footprint at element surfaces: traces the bundle and takes twice the largest ray distance from the surviving-ray centroid as the beam diameter (element 0 = the source aperture). Equal first/last element prints a single “Optical beam diameter” line to the screen; a range also writes name/type/diameter per element to `beamSize.txt`. Despite the help line, nothing Gaussian is involved. SMACOS: LoadStack queues IARG(1) plus a literal 0 for the last-element prompt, so only element 0 passes the first<=last check — ranges are effectively CLI-only. *Related:* SUMmarize, SHOw, FEXit.

BLUr, GBLur

Minimum match: `BLUr`

Blur / Gaussian blur.

CLI suite: `COMPOSE_ADD_NOISE.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — image-plane post-processing (B.2, third).

Syntax: `BLUR [peak]` — a blur of the composite with a kernel of the given peak; `GBLUR [wx,wy] [plot]` — a Gaussian blur of the given widths in base units, added to the composite.

Defaults: BLUR peak [1.00000]; GBLUR widths [0.00000,0.00000]; plot [YES]. GBLUR ADDS its result to the composite, so COMPOSE again before a second kernel.

```
MACOS>blur
Enter blur kernel peak (1e0): [1.]:
MACOS>gblur
 ...propagation summary...
Enter Gaussian blur kernel width in base units (0,0): [0.,0.]: 300d-6,300d-6
```

Image-blur post-processing on the COMposed pixel array. BLUr convolves the existing PixArray with a fixed kernel scaled by the requested peak and re-displays it ("No data to display" without a composed image). GBLur simulates long-exposure jitter and similar smooth image spreading: propagates the beam to the COMpose element, convolves the intensity with a Gaussian of the given x/y widths at the wavefront sampling density (generally more accurate than blurring after pixelization), adds the result to PixArray and displays it. GBLur requires COMpose first ("Must COMpose before GBLURing images") and warns "WARNING: PIXELS TOO SMALL" below half the WF grid spacing. Plot titles follow STRetch. SMACOS: BLUR RARG(1)=kernel peak; GBLUR RARG(1:2)=x,y widths; CARG(1)='NO' suppresses the plot. *Related:* COMpose, ADD, NOIse, PIXel, STRetch.

GAIn

Minimum match: `GAIn`

Apply gain map to intensity.

CLI suite: `AMP_PHASE_REAL_PIX.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — image-plane post-processing (B.2, third).

Syntax: `GAIN [elt]` — the element where the far-field gain over an isotropic radiator is computed.

Defaults: elt [the last propagation element].

```
MACOS>gain
Enter number of element where data is to be generated: [...]: 16
 ...propagation summary...
 (2)Peak intensity= ...; Peak occurs at i= ..., j= ...
 Sum of intensity= ...
 Maximum gain= ...
```

Far-field gain map: propagates the beam to the element (as INTensity) and converts the intensity to gain — the ratio of the diffracted intensity to a uniformly radiating point source of the same power — plotting it (title 'Gain', file tag .gain) and printing the maximum. Used mainly to evaluate radio-frequency

antennas (manual Section 7.2.14, Luneberg lens example). SMACOS: IARG(1)=element. *Related:* INTensity, LOG, BEAm.

ODRaw, PGD

Minimum match: `ODRaw`

Outline drawing / PGPLOT diagnostics.

CLI suite: `PGD_ODRAW.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only** — plot style (the bindings return the arrays).

Syntax: `ODRAW [first] [last] [XZ|YZ|XY]` — overdraw the layout and a reduced ray fan on the open page; `DRAW` the same on a new page; `PGD` opens the graphics device.

Defaults: first [0]; last [nElt]; plane [XZ]. PGD asks the device ("Graphics device/type", default /xw) once; later PGDs are silent.

```
MACOS>odraw
Enter first element to include: [0]:
Enter last element to include: [nElt]:
Enter drawing plane (XZ, YZ or XY in source coords): [XZ]:
```

ODRaw is overdraw: the same handler as DRAW (which HELP does not list) — trace a reduced ray fan and draw the system layout plus rays projected onto the chosen source-coordinate plane — but drawn on top of the existing plot without erasing it, for before/after geometry comparisons. Invalidates the trace/build/propagate state. PGD opens the graphics device: the first use prompts "Graphics device/type (? to see list, default /xw):" (`/null`, `/png`, `/xw` …); a later PGD is silent, the device is kept (GRAINI caches it) — so it does not re-prompt when the plot window was closed. Measured with `/png` (giza writes a page to giza_NNNN.png when the next page opens): each DRAW opens a page, ODRAW adds to the open page and writes no file. PGD is interactive CLI only. SMACOS: no LoadStack entry for ODRaw (only DRAW has one: IARG(1:3), CARG(1)). *Related:* PGP, SPOt.

ROW

Minimum match: `ROW`

Extract / plot row from image.

CLI suite: `MTF_LOS_ROWCOL.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — image-plane post-processing (B.2, third).

Syntax: `ROW`; no prompt — the plot type that draws one row of the next map (asked when drawn).

Defaults: none.

```
MACOS>row
 Plot type set to ROW
MACOS>intensity
 ...
Enter number of row to be plotted: [...]:
```

Selects the row plot type — the COLumn line plot taken along a row of the data matrix instead of a column. The row number is prompted when a plot is actually produced (default = the center row); out-of-range input skips the plot. MTF recommends exactly this for 1-D MTF cuts. SMACOS: LoadStack queues IARG(1), but the command itself reads nothing — the row prompt happens at plot time. *Related:* COLumn, SLIce, MTF.

System Optimization

AVAR

Minimum match: `AVAR <elt>`

Add a variable element.

CLI suite: `AVAR_VARS_MVAR_DVAR.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only, legacy** — differential FD inside the CLI; the api route (dw_dx and family over perturb_elt + opd_val) does it with the step, reference and mask under the caller's control (Dave 2026-10-10).

Syntax: `AVAR [elt] [dof] [zrange]` — the element made variable, its degree-of-freedom mask (eight flags: six rigid-body, then Zernike and aspheric), and the Zernike range.

Defaults: elt [0]; mask [0,0,0,0,0,0,0,0]; range [1,45]. An invalid element aborts the command.

```
MACOS>avar
Enter variable element id for optimization: [...]: 1
Enter variable degree of freedom: [0,0,0,0,0,0,0,0]: 0,0,0,0,0,0,1,1
Enter range of Zernike terms: [1,45]: 0,0
```

Declares an element variable for CALib. The 8-integer mask (nonzero = vary) follows the DOF_NameList order: TIP, TILT, CLOCK, DX, DY, PIST, ROC (radius of curvature), CONIC. The Zernike range additionally frees a run of surface Zernike terms (1-45); answer 0,0 for none. Element 0 makes the SOURCE variable (its position DOFs require a STOP). An already-variable element is refused ("Variable element N already defined!"). Constraint bounds for the constrained (SLSQP/NPSOL) path come from the Rx Opt* keywords, not this dialog. SMACOS: IARG(1)=element, IARG(2:9)=DOF mask; the Zernike-range step is skipped (no Zernike terms). *Related:* MVAR, DVAR, VARS, CALib, SETC.

MVAR

Minimum match: `MVAR <elt>`

Modify a variable element.

CLI suite: `AVAR_VARS_MVAR_DVAR.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only, legacy** — differential FD inside the CLI; the api route (dw_dx and family over perturb_elt + opd_val) does it with the step, reference and mask under the caller's control (Dave 2026-10-10).

Syntax: `MVAR <elt> [dof]` — modify a variable element's mask.

Defaults: elt has no default; mask [the element's current mask].

```
MACOS>mvar
Enter variable element ID for optimization: [...]: 1
Enter variable degree of freedom: [...]:
```

Redefines the 8-DOF mask of an element previously declared with AVAR ("Element N is not a variable" otherwise); the Zernike term range is not re-prompted — DVAR then AVAR again to change that. SMACOS: no LoadStack entry. *Related:* AVAR, DVAR, VARS, CALib.

DVAR

Minimum match: `DVAR <elt>`

Delete a variable element.

CLI suite: `AVAR_VARS_MVAR_DVAR.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only, legacy** — differential FD inside the CLI; the api route (dw_dx and family over perturb_elt + opd_val) does it with the step, reference and mask under the caller's control (Dave 2026-10-10).

Syntax: `DVAR <elt>` — remove an element from the variable set.

Defaults: elt has no default.

```
MACOS>dvar
Enter element ID to remove from optimization: [...]: 1
```

Removes an element from the CALib variable set, clearing its DOF mask and Zernike terms ("Element N is not a variable element!" otherwise). SMACOS: IARG(1)=element. *Related:* AVAR, MVAR, VARS.

VARS

Minimum match: `VARS`

List all variable elements.

CLI suite: `AVAR_VARS_MVAR_DVAR.jou`, `CALIB_save_roundtrip.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — lists the CALIB variables; `calib_set_var_elt` sets them, no getter.

Syntax: `VARS`; no prompt — lists the variable elements and their masks.

Defaults: none.

```
MACOS>vars
 Variable elements defined for optimization:
 Elt Name= PM, Elt ID= 1, Variable DOF= ROC CONIC ...
```

No dialog. Lists each variable element: name, element ID (plus the IDs of any linked elements that move with it, per LnkElt), the variable DOF names and any Zernike terms; " No Variable Element for Optimization!" when the set is empty. *Related:* AVAR, FOVS, WLENS, CALib, ELTS.

AFOV

Minimum match: `AFOV <fov>`

Add a field of view.

CLI suite: `FOV_SETC_SAOPT.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — field / beam set.

Syntax: `AFOV <dir> <pos>` — add a field: the chief-ray direction and position.

Defaults: no defaults.

```
MACOS>afov
 Add FOV for design optimization ...
Enter Chief Ray Direction [...]:
Enter Chief Ray Position [...]:
```

Appends a field of view — a chief-ray direction/position pair — to the optimization FOV list. FOVs can also be declared in the Rx by repeating the ChfRayDir/ChfRayPos (or OptChfRayDir/OptChfRayPos) keywords; the first pair is the nominal field. Per-FOV weights are set via SETC. Code quirk: the position prompt's default is the direction vector just entered. SMACOS: no LoadStack entry. *Related:* DFOV, FOVS, SFOV, SETC, CALib.

DFOV

Minimum match: `DFOV <fov>`

Delete a field of view.

CLI suite: `FOV_SETC_SAOPT.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only, legacy** — differential FD inside the CLI; the api route (dw_dx and family over perturb_elt + opd_val) does it with the step, reference and mask under the caller's control (Dave 2026-10-10).

Syntax: `DFOV <id>` — delete a field by id.

Defaults: no default.

```
MACOS>dfov
Enter ID of FOV to delete [...]: 2
```

Deletes the given field of view from the optimization list, shifting later entries down ("DFOV: Invalid FOV ID" when out of range). SMACOS: no LoadStack entry. *Related:* AFOV, FOVS, CALib.

FOVS

Minimum match: `FOVS`

List all fields of view.

CLI suite: `CALIB_fov_weight.jou`, `FOV_SETC_SAOPT.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — field / beam set.

Syntax: `FOVS`; no prompt — lists the fields with their weights.

Defaults: none. Weights are 1 after a load unless the deck gives `OptFOVWt=` (they were 0 before 2026-10-17, 1.2 (a)).

```
MACOS>fovs
 Number of field of view used for design optimization = 2
 FOV ID = 1, Weight = ...
    ChfRayPos = [ ... ]
    ChfRayDir = [ ... ]
```

No dialog. Lists the optimization fields of view with their weights. Code quirk: the two vector labels are swapped — the line labelled ChfRayPos prints the chief-ray DIRECTION and vice versa (AFOV and SFOV store and read the array consistently; only this listing mislabels it). *Related:* AFOV, DFOV, SFOV, WLENS, VARS.

CALib

Minimum match: `CALib`

Run system optimization.

CLI suite: `CALIB_SPOT_asph.jou`, `CALIB_SPOT_mixed.jou`, `CALIB_SPOT_rigid.jou`, `CALIB_SPOT_zern.jou`, `CALIB_WFE_asph.jou`, `CALIB_WFE_mixed.jou`, `CALIB_WFE_rigid.jou`, `CALIB_WFE_zern.jou`, `CALIB_ZWF_asph.jou`, `CALIB_ZWF_mixed.jou`, `CALIB_ZWF_rigid.jou`, `CALIB_ZWF_zern.jou`, `CALIB_beam_under_spot.jou`, `CALIB_failed_trials.jou`, `CALIB_fov_weight.jou`, `CALIB_history.jou`, `CALIB_keywords.jou`, `CALIB_setup.jou`, `CALIB_stalled.jou`, `OptAsph_order.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `calib_run`.

Syntax: `CALIB [summary] [save]` — run the optimization, then whether to print the summary and whether to keep the optimized state.

Defaults: summary [yes]; save [yes]. The controls come from the deck's Opt* keys (Table 13 of the manual) or SETC.

```
MACOS>calib
 ...optimizer iteration output...
Display optimization summary (yes,no): [yes]:
     ================================================
     ***    MACOS System Optimization Summary    ***
     ================================================
  Optimization Scheme: Nonlinear Unconstrained
  Optimization Target =  RMS Wavefront Error at Element ...
  * RMS WFE at FOVs BEFORE Optimization: ...
  * RMS WFE at FOVs AFTER  Optimization: ...
```

Runs the native multi-field design optimizer over the AVAR variable elements and the FOV list. Unconstrained problems use the Levenberg-Marquardt least-squares driver; when the Rx declares Opt* variable constraints the run dispatches to the bound-constrained SLSQP driver (NPSOL in USE_NPSOL builds). Target per SETC or the Rx OptTarget keyword: WFE (default), ZWF (WFE Zernike modes), Beam, and SPOT from the Rx or the bindings; OPL was removed and is refused at load. The whole keyword family (defaults, per-load reset, not written by SAVE), the row/unit conventions and the stopping rule (converged / floor / cap / failed / stalled, printed as `Stopped: ...`) are Manual Section 4, Table 13 "Design Optimization (CALIB) Keywords". Optional CLI arguments restrict a run to a subset of the variables (`calib 2 3`; each is read back with 'Enter element to be calibrated:' and must already be AVAR'd). Preconditions: at least one variable element and one FOV; STOP must be set when optimizing with FEX or moving the source. The summary reports scheme, target, rays / FOVs / wavelengths and per-wavelength before/after figures. SMACOS: no LoadStack arguments; the summary always prints. Suite: the `cli_tests/cmd/CALIB_*.jou` journals (the 12-cell target x DOF matrix, keywords, stopping, history, failed and stalled trials). *Related:* AVAR, VARS, AFOV, FOVS, SETC, SAOpt, STOp.

SFOV

Minimum match: `SFOV`

Select field of view.

CLI suite: `FOV_SETC_SAOPT.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — field / beam set.

Syntax: `SFOV [id]` — the field to trace.

Defaults: id [the field in force], 1 after a load.

```
MACOS>sfov
 Total FOV available:  3
Enter ID of FOV to use: [1]: 2
   Set ChfRayDir = ...
   Set ChfRayPos = ...
```

Switches the active chief ray to one of the fields declared in the Rx (repeated ChfRayDir/ChfRayPos keyword pairs — the count is the Rx pair count, not fields added later with AFOV) and invalidates the grid/trace/build/propagate state so subsequent analyses run at the new field. Interactive CLI only. *Related:* AFOV, FOVS, SWL, CHIefray.

Misc / Debug

SRAy, SRT, SRTrace

Minimum match: `SRAy`

Older single-ray variants (use RAY).

CLI suite: `SRAY_SRT.jou`, `ZERN_normhex.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **SMACOS command** — `srt_trace`.

Syntax: `SRAY [i] … 0 [first] [last]` — single rays by number until 0, then the element range; `SRT [last] [x,y]` (also `SRTRACE`) — one ray from a normalized source-aperture point traced to an element.

Defaults: SRAY ray [0] (return ends), first [0], last [0]; SRT last [nElt], position [0.00000,0.00000] (the chief ray). SRT crashed on every ray before 5bb836f.

```
MACOS>sray
Enter number of ray (1=chief ray, 0=quit): [0]: 1
Enter starting element: [...]:
Enter end element: [...]:
```

SRAy prints one ray's per-surface trace over an element range, silently pre-tracing from the source when the range starts downstream. The source flags it "buggy, doesn't work" (it also prints a stray 'iElt,iCurRayElt =' debug line) — prefer RAY. SRT crashed the CLI on every ray until 2026-10-17 (its ray-state buffers were never allocated); fixed, gate `SRAY_SRT.jou`. SMACOS: IARG(1)=ray, IARG(2)=start, IARG(3)=end element. SRT (interactive CLI only) traces a single ray launched from a designated NORMALIZED source-aperture position:

```
MACOS>srt
Enter last element to trace to: [nElt]:
Enter ray position in normalized source aperture: [...]: 0.25,0
```

positions outside +/-0.5 are refused. SRT's LoadStack mapping (IARG(1), DARG(1:2)) is vestigial — the SMACOS build does not dispatch SRT. SRTrace exists only as a compiled-out (#if 0) test hook for srtrace.F and is not available. *Related:* RAY, SEGRAYTrace, PRAy.

DOPD, DOPL, ZRM

Minimum match: `DOPD`

Debug / diagnostic prints.

CLI suite: `DOPD_identical.jou`, `ZABERR_ZRM_DOPD.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **CLI-only, legacy** — differential FD inside the CLI; the api route (dw_dx and family over perturb_elt + opd_val) does it with the step, reference and mask under the caller's control (Dave 2026-10-10).

Syntax: `DOPD` / `DOPL`; no prompt — the rms difference of the last two OPDs; `ZRM [n] <modes> [save]` — remove Zernike modes from the OPD.

Defaults: ZRM n [1]; the indexes have no default; save [no]. DOPD reads the sentinel −1 right after ZAB / ZRM (1.2 (k)).

DOPD (no dialog; interactive CLI only) plots a DIFFERENTIAL OPD map — the current OPDMat minus the previous OPD run — and prints its RMS and P-V. It requires OPD run at least twice ("Before run DOPD command, load Rx and run OPD command at least twice!") and warns "**Warning: OPDs are not on same element!" when the two runs targeted different elements. ZRM (interactive CLI only) removes fitted Zernike modes from the current OPD:

```
MACOS>zrm
 ** Removing Zernike modes from OPD (Noll ordered) **
Enter number of Zernike modes: [1]: 3
Enter Zernike indexes: [...]: 1,2,3
  RMS WFE (after ZRM) = ...
Save new OPD (no,yes): [no]:
```

same mode dialog as ZCOef/ZABerr (a NEGATIVE count -N prompts 'Enter starting Zernike mode index:' for N consecutive modes; basis family per SZCo); answering yes replaces OPDMat and invalidates the trace/build state. DOPL (differential OPL between two prescriptions) is commented out of the dispatcher and NOT available. *Related:* OPD, ZCOef, ZABerr, SZCo.

JWST_v3d, Vis3d

Minimum match: `JWST_v3d`

3D rendering demo.

CLI suite: `EMACS_VI.jou`, `VIS3D.jou` (`cli_tests/cmd/`; each journal is the entry's example and its regression test).

Binding: **missing** — builder; the Vis3d ray history is `ray_hist_set` / `ray_pos_hist_get`.

Syntax: `JWST_V3D` / `VIS3D`; no prompt.

Defaults: none; prints the refusal unless the build has VIS3D_DEMO.

No dialog. Starts the JWST 3-D rendering demo as a child process — available only in builds compiled with VIS3D_DEMO; standard builds print " **JWST 3D rendering demo not included in MACOS!". *Related:* ODRaw.

Part II — mmacos / pymacos function catalog

Each entry shows the MATLAB (mmacos) and Python (pymacos) form of the same operation where both exist; both wrap the same engine routine in `macos_api_mod.F90`, so arguments, units and behavior match. An entry with only one language is a coverage gap in the other (noted).

Session, prescription and model I/O

Session

- **mmacos:** `classdef Session < handle`

- **pymacos:** *not available*

OO veneer over the +macos function package. m = macos.Session(model_size) initializes the engine and returns a handle. Every method delegates to the same-named +macos package function – the only state in the class is the loaded Rx path (cached so it's discoverable via m.rx_path).

cbm

- **mmacos:** `c = macos.cbm()`
- **pymacos:** *not available*

Base-units-to-metres conversion factor (CBM). c = macos.cbm() returns the SI-metres value of one BaseUnit in the currently loaded Rx (e.g. 1e-3 for an Rx in mm). Used internally when callers want to feed/read distances in SI metres while the prescription is in its native units.

has_rx

- **mmacos:** `tf = macos.has_rx()`
- **pymacos:** `pymacos.has_rx()`

True iff a prescription is loaded. Detects via `num_elt > 0`; if init hasn't run yet, mmacos errors internally and we return false.

init

- **mmacos:** `macos.init(model_size)`
- **pymacos:** `pymacos.init(model_size)`
- **engine API:** `macos_api_mod::init`

Initialize / resize the macos engine. macos.init(MODEL_SIZE) allocates the engine for prescriptions up to MODEL_SIZE elements / rays / etc. Must be called once per MATLAB session before any other macos.* call. MODEL_SIZE must be one of macos.model_sizes() – param_mod.F rejects

load_rx

- **mmacos:** `nElt = macos.load_rx(rx_path)`
- **pymacos:** `pymacos.load(macos_rx)`
- **engine API:** `macos_api_mod::load_rx`

Load a prescription file (.in) into the macos engine. N = macos.load_rx(RX_PATH) loads the prescription at RX_PATH and returns the number of elements N (>0). RX_PATH may be supplied with or without the trailing '.in' — macos's OLD command appends '.in' unconditionally, so we strip a trailing '.in' if present (otherwise macos tries to open 'foo.in.in').

Session modes survive load_rx (2026-10-11): POLARIZATION, VECTOR/SCALAR, CENTROID, LNEG and CIR are session modes, set once in the engine and kept across every load – in the CLI and the bindings alike (one semantics; Dave's ruling on the CLI modes, extended to the bindings). `load_rx` warns when any is in force – MATLAB `warning('mmacos:modesInForce', 'load_rx: session modes in force: ...')`, Python `RuntimeWarning` – and is silent at the defaults; `macos.modes()` / `pymacos.modes()` read them. Set them per deck with `macos.polarization` / `vecdif_set` / the `xp_fnd` mode.

modify

- **mmacos:** `macos.modify()`
- **pymacos:** `pymacos.modify()`

Reset ray-trace-dependent state (the SMACOS MODIFY cmd). Call after any direct prescription mutation (perturb, set_elt_*) before re-running a trace, so cached intermediates don't carry stale geometry.

num_elt

- **mmacos:** `n = macos.num_elt()`
- **pymacos:** `pymacos.num_elt()`

Number of elements in the currently loaded Rx.

save_rx

- **mmacos:** `macos.save_rx(rx_path)`
- **pymacos:** `pymacos.save(rx, overwrite)`
- **engine API:** `macos_api_mod::save_rx`

Write the current macos state to a prescription file.

sys_units

- **mmacos:** `s = macos.sys_units()`
- **pymacos:** `pymacos.sys_units()`
- **engine API:** `macos_api_mod::sys_units`

Return a struct of unit identifiers for the loaded Rx. s = macos.sys_units() returns: .base_unit_id numeric id of the prescription's BaseUnits .wave_unit_id numeric id of the WaveUnits See macos's BaseUnits / WaveUnits enums for the mapping.

Source and wavelength

dwdx_for_current_source

- **mmacos:** `[dwdx, w_nom_2d, w_nom_vec, indx, names, dcdx, spot_pos, spot_neg, spot_nom, spot_pert] = ...`
- **pymacos:** *not available*

TODO: expand — dialog details, behavior notes, related commands.

dwdz_for_current_source

- **mmacos:** `[dwdz, w_nom_2d, w_nom_vec, indx, names, dcdx, spot_pos, spot_neg, spot_nom, spot_pert] = ...`
- **pymacos:** *not available*

TODO: expand — dialog details, behavior notes, related commands.

is_point_source

- **mmacos:** `tf = macos.is_point_source()`
- **pymacos:** *not available*

True iff the loaded Rx has a finite (point) source. tf = macos.is_point_source() returns true for a point (finite-distance) source, false for a collimated source. See also: macos.get_src_size.

perturb_src

- **mmacos:** `macos.perturb_src(opts)`
- **pymacos:** `pymacos.perturb_src(rotation_rad, translation_m)`
- **engine API:** `macos_api_mod::perturb_src`

Rigid-body perturbation of the source (iElt=0). Name-value pairs: ‘rotation’ [3x1] x,y,z rotation (rad). Default zeros. ‘translation’ [3x1] x,y,z translation in SI metres. Converted internally to BaseUnits via 1/CBM. Default zeros.

src_csys

- **mmacos:** `s = macos.get_src_csys()`
- **pymacos:** `pymacos.src_csys(x_dir, y_dir, z_rot, threshold)`
- **engine API:** `macos_api_mod::get_src_csys`

Source coordinate-frame axes. s = macos.get_src_csys() returns a struct: .xDir 3×1 source frame x-axis (unit vector) .yDir 3×1 source frame y-axis .zDir 3×1 source frame z-axis (chief-ray / propagation direction)

src_finite

- **pymacos:** `pymacos.src_finite()`
- **mmacos:** *not available*
- **engine API:** `macos_api_mod::src_finite`

Returns if Source Position is finite

src_flux

- **mmacos:** `flux = macos.get_src_flux()`
- **mmacos:** `macos.set_src_flux(flux)`
- **pymacos:** `pymacos.src_flux(flux)`
- **engine API:** `macos_api_mod::src_flux`

Source flux (propagated intensity scales with it). Implementation: api src_flux is intent(inout) – caller provides a placeholder value, gets the engine’s value back in getter mode (setter=0).

src_fov

- **mmacos:** `fov = macos.get_src_fov()`
- **mmacos:** `macos.set_src_fov(opts)`
- **pymacos:** `pymacos.src_fov(src_pos, src_dir, src_dist)`
- **engine API:** `macos_api_mod::get_src_fov`

Read the source field-of-view state. fov = macos.get_src_fov() returns a struct with fields: .zSrc scalar — source distance (BaseUnits) .src_pos 3×1 — ChfRayPos .src_dir 3×1 — ChfRayDir (unit vector along chief ray) .src_finite logical — true for finite point source, false for

src_info

- **pymacos:** `pymacos.src_info()`
- **mmacos:** *not available*
- **engine API:** `macos_api_mod::src_info`

Retrieve Source def. information: shape, position & wavelength

src_sampling

- **mmacos:** `n = macos.get_src_sampling()`
- **mmacos:** `macos.set_src_sampling(n)`
- **pymacos:** `pymacos.src_sampling(n_gridpts)`

- **engine API:** `macos_api_mod::get_src_sampling`

Source-grid sampling (nGridPts).

src_seg

- **pymacos:** `pymacos.src_seg()`
- **mmacos:** *not available*

The source segmentation tiling (the truth mmacos `view_rx` draws Segment tiles from): `grid_type` (1 Circular, 2 Square, 3 Hex, 4 Pie, 5 Flower, 6 UDG), `n_seg`, `width` (flat-to-flat), `gap`.

src_size

- **mmacos:** `s = macos.get_src_size()`
- **mmacos:** `macos.set_src_size(aperture, obscuration)`
- **pymacos:** `pymacos.src_size(ape, obs)`
- **engine API:** `macos_api_mod::get_src_size`

Source aperture and obscuration. s = macos.get_src_size() returns a struct: .aperture scalar beam N.A. (point source) or diameter (collimated) .obscuration scalar central obscuration, same units as .aperture Collimated diameters are in BaseUnits; point-source values are numerical

src_wvl

- **mmacos:** `wvl = macos.get_src_wvl()`
- **mmacos:** `macos.set_src_wvl(wvl)`
- **pymacos:** `pymacos.src_wvl(wvl)`
- **engine API:** `macos_api_mod::src_wvl`

Source wavelength (WaveUnits). Returns the currently-configured source wavelength in the prescription's WaveUnits (typically µm). Multiply by macos.cbm() * wave-unit-ratio for SI if needed. Implementation: api src_wvl is intent(inout) — caller provides a

sxp

- **mmacos:** `xp = macos.sxp(mode)`
- **pymacos:** `pymacos.sxp(mode)`

Set / find the system exit pupil from the current chief ray. xp = macos.sxp() re-derives the exit-pupil pose and radius from the current chief-ray geometry and returns a 7-element row: [xp_pos_x, xp_pos_y, xp_pos_z, % EP centre xp_dir_x, xp_dir_y, xp_dir_z, % EP axis direction

Element geometry and properties

del_elt_grp

- **mmacos:** `macos.del_elt_grp(iElt)`
- **pymacos:** `pymacos.elt_grp_rm(srf)`

Remove the EltGrp from element(s) IELT. macos.del_elt_grp(IELT) clears the EltGrp(0:N, iElt) array on IELT (or each entry of the column vector IELT). Companion to macos.set_elt_grp.

elt_asph

- **mmacos:** `c = macos.get_elt_asph(srf, n)`
- **pymacos:** `pymacos.get_elt_asph(srf, n)`

Even-radial aspheric coefficients (AsphCoef: h^4, h^6, …) of an element. c = macos.get_elt_asph(srf) returns the first 4 coefficients (the Rx default count); c = macos.get_elt_asph(srf, n) the first n (n <= 9). What CALIB's OptAsph= DOFs leave on the element (engine 2026-10-04).

elt_csys

- **mmacos:** `s = macos.get_elt_csys(srfs)`
- **mmacos:** `macos.set_elt_csys(srf, xDir, yDir, zDir, opts)`
- **pymacos:** `pymacos.elt_csys(srf, xdir, ydir, zdir, upd, glb)`

Local coordinate-system matrices for elements. s = macos.get_elt_csys(SRFS) returns a struct with fields: .csys 6×6×N double — TElt frame matrix for each element .csys_lcs N×1 logical — true if a local CSYS is defined .csys_upd N×1 logical — true if the LCS updates with perturbations

elt_ff_any

- **mmacos:** `tf = macos.elt_ff_any()`
- **pymacos:** *not available*

True iff the loaded Rx has any FreeForm (SrfType 14) surfaces. See also: macos.find_freeform_elts, macos.get_elt_ff_zrn_coef.

elt_ff_zrn_coef

- **mmacos:** `coefs = macos.get_elt_ff_zrn_coef(iElt, modes)`
- **mmacos:** `macos.set_elt_ff_zrn_coef(iElt, modes, coefs, opts)`
- **pymacos:** *not available*

Read FFZern coefficients on a FreeForm element. coefs = macos.get_elt_ff_zrn_coef(IELT, MODES) returns COEFS(k) = FFZernCoef(MODES(k)) on element IELT (must be FreeForm SrfType=14), as a column vector. See also: macos.set_elt_ff_zrn_coef.

elt_grating_any

- **mmacos:** `tf = macos.elt_grating_any()`
- **pymacos:** `pymacos.elt_grating_any()`

True iff the loaded Rx has any grating elements.

elt_grating_dir

- **mmacos:** `d = macos.get_elt_grating_dir(srf)`
- **mmacos:** `macos.set_elt_grating_dir(srf, d)`
- **pymacos:** `pymacos.elt_grating_dir(srf, rule_dir)`

h1HOE ruling-direction vector (column 3-vec). Perpendicular to the ruling direction and to psiElt. Always returned as a unit vector.

elt_grating_fnd

- **mmacos:** `s = macos.elt_grating_fnd(srfs)`
- **pymacos:** `pymacos.elt_grating_fnd(srf)`

Find grating-bearing elements. s = macos.elt_grating_fnd() queries all elements. s = macos.elt_grating_fnd(SRF) queries the listed elements only. Returns a struct with fields: .srfs integer column vector of element ids that carry a grating

elt_grating_order

- **mmacos:** `order = macos.get_elt_grating_order(srf)`
- **mmacos:** `macos.set_elt_grating_order(srf, order)`
- **pymacos:** `pymacos.elt_grating_order(srf, diff_order)`

Diffraction order at element SRF.

elt_grating_params

- **mmacos:** `p = macos.get_elt_grating_params(srf)`
- **mmacos:** `macos.set_elt_grating_params(srf, params)`
- **pymacos:** `pymacos.elt_grating_params(srf)`

Read grating parameters at element SRF. Returns a struct with fields: .reflective logical: true=reflective, false=transmissive .rule_width double: spacing along projected flat plane .diff_order integer: diffraction order .rule_dir 3×1 double: h1HOE — perpendicular to ruling + psi

elt_grating_rulewidth

- **mmacos:** `w = macos.get_elt_grating_rulewidth(srf)`
- **mmacos:** `macos.set_elt_grating_rulewidth(srf, w)`
- **pymacos:** `pymacos.elt_grating_rulewidth(srf, rule_width)`

Ruling spacing at element SRF.

elt_grating_type

- **mmacos:** `refl = macos.get_elt_grating_type(srf)`
- **mmacos:** `macos.set_elt_grating_type(srf, refl)`
- **pymacos:** `pymacos.elt_grating_type(srf, reflective)`

Reflective (true) vs transmissive (false).

elt_grid

- **mmacos:** `s = macos.get_elt_grid(srf)`
- **mmacos:** `macos.set_elt_grid(srf, dx, mat)`
- **pymacos:** `pymacos.elt_grid(srf, sampling_spacing, grid_dz)`

Read the grid-data surface figure at element SRF. s = macos.get_elt_grid(SRF) returns a struct: .size scalar grid dimension nGridMat (square) .dx scalar node spacing GridSrfdx (BaseUnits, dx==dy) .mat size×size grid displacements from the nominal shape (BaseUnits)

elt_grid_add

- **mmacos:** `macos.elt_grid_add(srf, grid_dz)`
- **pymacos:** `pymacos.elt_grid_add(srf, grid_dz)`

Add a z-displacement grid to a grid surface. macos.elt_grid_add(SRF, GRID_DZ) ADDS the [N x N] displacement matrix GRID_DZ (along the surface's local z-axis) to the existing grid-surface data on element SRF. Use it to perturb an optic through its grid-data description (e.g. a measured/figure-error map or a DM influence sum).

elt_grid_any

- **mmacos:** `tf = macos.elt_grid_any()`
- **pymacos:** `pymacos.elt_grid_any()`

True iff the loaded Rx has any grid-data surfaces. Covers every GridData-bearing SrfType (GridData(9), AsGrData(11), MonGrData(12), ZrnGridData(13), FreeForm(14) – the engine's GridTypeAll set). See also: macos.find_grid_elts, macos.get_elt_grid.

elt_grid_dx

- **pymacos:** `pymacos.elt_grid_dx(srf, sampling_spacing)`
- **mmacos:** *not available*

Set/Get element grid surface grid sampling spacing (dx==dy).

elt_grid_npts

- **pymacos:** `pymacos.elt_grid_npts(srf)`
- **mmacos:** *not available*

Get element grid surface grid sampling N (N == Nx == Ny).

elt_grid_npts_max

- **pymacos:** `pymacos.elt_grid_npts_max()`
- **mmacos:** *not available*

Get Max. surface grid sampling Size N (N == Nx == Ny).

elt_grid_scale

- **pymacos:** `pymacos.elt_grid_scale(srf, scalar)`
- **mmacos:** *not available*

Scales element surface grid data.

elt_grid_size

- **mmacos:** `n = macos.get_elt_grid_size(srf)`
- **pymacos:** *not available*

Grid sampling (nGridMat) at element SRF. n = macos.get_elt_grid_size(SRF) returns the square grid dimension for a grid-bearing surface, or -1 if SRF carries no grid. See also: macos.get_elt_grid, macos.find_grid_elts.

elt_grid_spacing

- **mmacos:** `dx = macos.get_elt_grid_spacing(srf)`
- **mmacos:** `macos.set_elt_grid_spacing(srf, dx)`
- **pymacos:** *not available*

Grid node spacing GridSrfdx (dx==dy) at elt SRF. dx = macos.get_elt_grid_spacing(SRF) returns the grid sampling spacing in BaseUnits. The grid spans (nGridMat-1)*dx centred on the surface. Errors if SRF is not a grid surface. See also: macos.set_elt_grid_spacing.

elt_grp

- **mmacos:** `members = macos.get_elt_grp(iElt)`
- **mmacos:** `macos.set_elt_grp(iElt, members)`
- **pymacos:** `pymacos.elt_grp(srf, srfs_in_grp)`

Read the EltGrp member list on a reference element. members = macos.get_elt_grp(IELT) returns a column vector of 1-based element ids comprising IELT's EltGrp. Empty column vector if no group is installed on IELT. Companion to macos.set_elt_grp / macos.del_elt_grp / macos.prb_grp.

elt_grp_any

- **pymacos:** `pymacos.elt_grp_any()`
- **mmacos:** *not available*
- **engine API:** `macos_api_mod::elt_grp_any`

Check if Rx has any Elt Grp. defined

elt_grp_fnd

- **pymacos:** `pymacos.elt_grp_fnd(srf)`
- **mmacos:** *not available*
- **engine API:** `macos_api_mod::elt_grp_fnd`

Find all elements where element Grps. are defined.

elt_grp_max_size

- **pymacos:** `pymacos.elt_grp_max_size(srf)`
- **mmacos:** *not available*

TODO: expand — dialog details, behavior notes, related commands.

elt_grp_wipe

- **pymacos:** `pymacos.elt_grp_wipe()`
- **mmacos:** *not available*

Wipes out all Element-Grp. Settings from Rx

elt_info

- **mmacos:** `info = macos.get_elt_info(k)`
- **pymacos:** `pymacos.elt_info(srf)`

Element metadata for layout drawing. info = macos.get_elt_info(K) queries element K of the loaded Rx: .elt_id engine EltID code (see .type for the name) .type element type name ('Reflector', 'Refractor', …) .ap_type aperture type code (0 None, 1 Circular, 2 Elliptical, 6 Hexagonal, 7 Polygonal, 8 Tapered_Polygonal, …)

elt_jones

- **mmacos:** `J = macos.elt_jones(srf)`
- **pymacos:** `pymacos.elt_jones(srf)`

The 2x2 Jones matrix a polarizing element applied. J = macos.elt_jones(SRF) returns the complex 2x2 Jones matrix that the polarizing element at SRF applied during the last trace, expressed in the element's OWN transverse eigenbasis: column/row 1 is the declared axis projected into the ray's transverse plane, 2 is its orthogonal partner (rhat x ahat).

Reads a formerly dead array. `JmatElt(2,2,mElt)` was allocated, zeroed and deallocated by `elt_mod` and never otherwise touched. Phase 3 fills it in `PolElt` and exposes it here.

Per element, not per ray – and that is exact. The coefficients of an ideal polarizer or retarder in its OWN eigenbasis do not depend on the ray; all the ray dependence lives in the basis (the declared axis projected into each ray's transverse plane). So this is not a substitute for `jones_pupil`, which is the pupil-referenced Jones and is what any polarization-aberration analysis wants.

Diagonal by construction: `diag(1,0)` for an ideal polarizer, `diag(1, exp(-i*2*pi*R))` for a waveplate. Returns zeros if no polarized trace has run.

elt_kc

- **mmacos:** `kc = macos.get_elt_kc(srf)`
- **mmacos:** `macos.set_elt_kc(srf, kc)`
- **pymacos:** `pymacos.elt_kc(srf, conic_constant)`
- **engine API:** `macos_api_mod::elt_kc`

Conic constant Kc of element SRF (dimensionless). Kc = 0 sphere, -1 paraboloid, <-1 hyperboloid, -1<Kc<0 prolate ellipsoid, Kc>0 oblate ellipsoid.

elt_kr

- **mmacos:** `kr = macos.get_elt_kr(srf)`
- **mmacos:** `macos.set_elt_kr(srf, kr)`
- **pymacos:** `pymacos.elt_kr(srf, radii)`
- **engine API:** `macos_api_mod::elt_kr`

Radius of curvature Kr of element SRF (BaseUnits). Flat / FocalPlane elements report a big-magnitude sentinel (~1e22), not a physical radius.

elt_mon_zrn_coef

- **mmacos:** `coefs = macos.get_elt_mon_zrn_coef(iElt, modes)`
- **mmacos:** `macos.set_elt_mon_zrn_coef(iElt, modes, coefs, opts)`
- **pymacos:** *not available*

Read MonZern coefficients on a FreeForm element. coefs = macos.get_elt_mon_zrn_coef(IELT, MODES) returns COEFS(k) = MonZernCoef(MODES(k)) on element IELT (must be FreeForm SrfType=14), as a column vector. See also: macos.set_elt_mon_zrn_coef.

elt_obs

- **mmacos:** `obs = macos.get_elt_obs(k)`
- **pymacos:** `pymacos.elt_obs(srf)`

Obscuration declarations of element K (loaded Rx). obs = macos.get_elt_obs(K) returns the element's declared obscurations (a perforated primary's central hole, a coronagraph mask, a spider blade, …): .n declared count (engine nObs; may exceed the returned N) .type 1xN ObsType codes as stored (+/-1 Circle/NegCircle,

elt_psi

- **mmacos:** `psi = macos.get_elt_psi(srf)`
- **mmacos:** `macos.set_elt_psi(srf, psi)`
- **pymacos:** `pymacos.elt_psi(srf, psi)`
- **engine API:** `macos_api_mod::elt_psi`

Vertex surface normal of element SRF (3-vector).

elt_rpt

- **mmacos:** `rpt = macos.get_elt_rpt(srf)`
- **mmacos:** `macos.set_elt_rpt(srf, rpt)`
- **pymacos:** `pymacos.elt_rpt(srf, rpt)`
- **engine API:** `macos_api_mod::elt_rpt`

Reference point of element SRF (3-vector, BaseUnits).

elt_srf_csys

- **mmacos:** `s = macos.get_elt_srf_csys(srfs)`
- **mmacos:** `macos.set_elt_srf_csys(srf, pMon, xMon, yMon, zMon)`
- **pymacos:** `pymacos.elt_srf_csys(srf, origin, xdir, ydir, zdir)`
- **engine API:** `macos_api_mod::elt_srf_csys`

Surface (monomial/Zernike/grid) coordinate frame. s = macos.get_elt_srf_csys(SRFS) returns the surface-figure coordinate frame (pMon/xMon/yMon/zMon) for each element in SRFS – the frame in which monomial, Zernike and grid figures are defined: .pMon 3×N frame origin (vertex position) .xMon 3×N x-axis (each column a unit vector)

elt_vpt

- **mmacos:** `vpt = macos.get_elt_vpt(srf)`
- **mmacos:** `macos.set_elt_vpt(srf, vpt)`
- **pymacos:** `pymacos.elt_vpt(srf, vpt)`
- **engine API:** `macos_api_mod::elt_vpt`

Vertex point of element SRF (3-vector, BaseUnits).

elt_z

- **mmacos:** `z = macos.get_elt_z(srf)`
- **pymacos:** `pymacos.elt_z(srf, z)`
- **engine API:** `macos_api_mod::elt_z`

Propagation distance zElt at element SRF. For Flat / FocalPlane elements this is a big-magnitude sentinel (~1e19–1e22) meaning “propagate to the next defined plane”, not a physical distance. Curved elements report the actual prescribed propagation distance from the previous surface.

elt_zrn

- **mmacos:** `s = macos.get_elt_zrn(srf)`
- **pymacos:** *not available*

Read the Zernike-surface definition at element SRF. s = macos.get_elt_zrn(SRF) returns a struct for a SrfType=Zernike surface: .norm_radius scalar Zernike normalisation radius (lMon, BaseUnits) .type scalar ZernType id (1 ANSI, 2 BornWolf, 3 Fringe, 4 NormANSI, 5 NormBornWolf, 6 NormFringe, 7 NormHex (45 modes), 8 NormNoll,

elt_zrn_any

- **mmacos:** `tf = macos.elt_zrn_any()`
- **pymacos:** `pymacos.elt_zrn_any()`

True iff the loaded Rx has any Zernike (SrfType 8) surfaces. See also: macos.get_elt_zrn, macos.find_zern_elts.

elt_zrn_coef

- **mmacos:** `coefs = macos.get_elt_zrn_coef(iElt, modes)`
- **mmacos:** `macos.set_elt_zrn_coef(iElt, modes, coefs, opts)`
- **pymacos:** `pymacos.elt_zrn_coef(srf, mode, coefs, reset)`

Read Zernike coefficients on a Zern-typed element. coefs = macos.get_elt_zrn_coef(IELT, MODES) returns COEFS(k) = ZernCoef(MODES(k)) on element IELT, as a column vector matching the shape of MODES. IELT must declare Surface=Zernike or Surface=ZrnGridData in the Rx.

elt_zrn_norm_rad

- **pymacos:** `pymacos.elt_zrn_norm_rad(srf, norm_rad)`
- **mmacos:** *not available*

Set/Get Zernike Normalisation Radius for specified Surface.

elt_zrn_norm_radius

- **mmacos:** `r = macos.get_elt_zrn_norm_radius(srf)`
- **mmacos:** `macos.set_elt_zrn_norm_radius(srf, norm_radius)`
- **pymacos:** *not available*

Zernike normalisation radius (lMon) at SRF. r = macos.get_elt_zrn_norm_radius(SRF) returns the Zernike normalisation radius in BaseUnits, or -1 if SRF is not a Zernike surface. See also: macos.set_elt_zrn_norm_radius, macos.get_elt_zrn.

elt_zrn_type

- **mmacos:** `t = macos.get_elt_zrn_type(srf)`
- **mmacos:** `macos.set_elt_zrn_type(srf, zern_type, opts)`
- **pymacos:** `pymacos.elt_zrn_type(srf, zrn_type, reset)`

Zernike normalisation type id at element SRF. t = macos.get_elt_zrn_type(SRF) returns the ZernType id (1 ANSI, 2 BornWolf, 3 Fringe, 4 NormANSI, 5 NormBornWolf, 6 NormFringe, 7 NormHex (45 modes), 8 NormNoll, 9 NormAnnularNoll (refused at load), 10 Noll, 11 ExtFringe), or -1 if SRF is not a Zernike surface. The orderings: MACOS Manual Appendix E. See also: macos.set_elt_zrn_type, macos.get_elt_zrn.

find_freeform_elts

- **mmacos:** `ff = macos.find_freeform_elts()`
- **pymacos:** `pymacos.findFreeFormElts()`

Indices of FreeForm-typed elements. ff = macos.find_freeform_elts() returns a column vector of 1-based element ids for every FreeForm (SrfType=14) surface in the loaded prescription, in element order. Empty array if no FreeForm surfaces are present.

find_grid_elts

- **mmacos:** `g = macos.find_grid_elts()`
- **pymacos:** `pymacos.elt_grid_fnd(srf, grid_srf_type)`

Indices of grid-bearing elements (ANY GridData type). g = macos.find_grid_elts() returns a column vector of 1-based element ids for every surface that carries a grid-data component (grid size > 0) in the loaded prescription, in element order. Empty if none. Eligibility is discovered by grid SIZE (nGridMat>0), NOT by SrfType, so

find_powered_elts

- **mmacos:** `[pe, kinds] = macos.find_powered_elts(session, rx_path, opts)`
- **pymacos:** *not available*

Indices of POWERED optics in the beam train. pe = macos.find_powered_elts(SESSION, RX_PATH) returns a column vector of 1-based element ids for every POWERED optic in the LOADED Rx: an element of a powered-capable Element= kind whose base radius is real (|Kr| << the flat sentinel 1e22).

find_zern_elts

- **mmacos:** `ze = macos.find_zern_elts(rx_path)`
- **pymacos:** `pymacos.elt_zrn_fnd(srf)`

Indices of Zernike-typed elements (SrfType=8/13). ze = macos.find_zern_elts(RX_PATH) returns a column vector of 1-based element ids for every element declared as Surface=Zernike or Surface=ZrnGridData / ZrnGrData in the Rx text file. Empty array if no such surfaces are present.

prb_grp

- **mmacos:** `macos.prb_grp(iElt, prb, ifGlobal)`
- **pymacos:** `pymacos.prb_grp(srf, prb, glb_csys)`

GPERTURB: rigid-body perturb a group as a single unit. macos.prb_grp(IELT, PRB) applies the 6-DOF rigid-body perturbation vector PRB to the group whose reference element is IELT (the element whose EltGrp array enumerates the members). Inputs:

Ray trace and outputs

apodize

- **mmacos:** `macos.apodize(srf, mask)`
- **pymacos:** `pymacos.apodize(srf, mask)`

Multiply WFElt(:,:,iEltToiWF(srf)) by a real NxN mask. Companion to PROPER's prop_multiply – pass the same numpy/MATLAB array to both engines and the apodisation is bit-identical. Caller must already have propagated to SRF (e.g. via macos.trace or macos.complex_field with default reset_trace=true). Subsequent

apodize_complex

- **mmacos:** `macos.apodize_complex(srf, mask)`
- **pymacos:** `pymacos.apodize_complex(srf, mask)`

Multiply WFElt(:,:, iEltToiWF(srf)) by a complex mask. Companion to macos.apodize for the complex case — used by the DM phase-imprint tests where the mask is `exp(i * 2*pi * OPD / lambda)` (unit-magnitude, pure phase). Same caveat as macos.apodize:

complex_field

- **mmacos:** `cf = macos.complex_field(srf, opts)`
- **pymacos:** `pymacos.complex_field(srf, reset_trace, plane)`

WFElt at element SRF (N x N complex double). cf = macos.complex_field(SRF) propagates to SRF and returns the complex wavefront on the diffraction grid. |cf|^2 matches macos.intensity(SRF) to numerical precision. cf = macos.complex_field(SRF, 'plane', K) returns a single Cartesian

`plane` selects a single Cartesian FIELD COMPONENT. In vector-diffraction mode the three wavefront storage planes are repurposed as Ex/Ey/Ez of one wavefront, so `plane` = 1, 2, 3 returns Ex, Ey, Ez. `plane` = 0 (the default) returns the element's own wavefront and is bit-identical to omitting the option.

This is the only way to see how the three components contribute to a propagated intensity — `intensity` sums them. The planes add in INTENSITY, not amplitude: `sum_k |complex_field(s, plane=k)|^2 == intensity(s)`.

Requesting `plane` 1..3 with vector diffraction OFF is an ERROR, not a silent fallback: in scalar mode plane k is an unrelated wavefront, not a field component, and returning it would look plausible and be wrong. Enable with `polarization('on')` + `vector_diffraction(true)`. *Related:* intensity, vector_diffraction, ray_field.

compose

- **mmacos:** `I = macos.compose(srf, wavelengths, npix, dx, opts)`
- **pymacos:** `pymacos.compose(srf, wavelengths, npix, dx, dx_unit)`

Multi-wavelength PSF on a fixed pixel grid (COMPOSE+ADD). I = macos.compose(SRF, WAVELENGTHS, NPIX, DX) assembles a composite image at element SRF: for each wavelength in WAVELENGTHS the source wavelength is set, the field is propagated to SRF, and its intensity is accumulated onto a single NPIX x NPIX detector grid of pitch DX. The result is the INCOHERENT sum – a broadband PSF.

draw_rays

- **mmacos:** `b = macos.draw_rays(plane, iStart, iEnd)`
- **pymacos:** `pymacos.draw_rays(plane, i_start, i_end)`

Real DRAW ray bundle as data (no graphics device). b = macos.draw_rays(PLANE, ISTART, IEND) runs the engine DRAW command in DATA-ONLY mode and returns its traced ray fan projected onto PLANE ('YZ' | 'XZ' | 'XY', source coords), for elements ISTART..IEND (ISTART = 0 => from the source). No window is opened, nothing renders.

draw_rays3d

- **mmacos:** `b = macos.draw_rays3d(plane, iStart, iEnd)`
- **pymacos:** `pymacos.draw_rays3d(plane, i_start, i_end)`

Real DRAW ray fan as 3-D data (no graphics device). b = macos.draw_rays3d(PLANE, ISTART, IEND) runs the engine DRAW command in DATA-ONLY mode and returns the traced fan's surface crossings in GLOBAL 3-D coordinates (BaseUnits) — the unprojected companion of macos.draw_rays. PLANE ('YZ' | 'XZ') selects WHICH meridian fan the engine traces (DRAW draws the middle fan of that plane); the returned

dx_at

- **mmacos:** `dx = macos.dx_at(srf, unit)`
- **pymacos:** `pymacos.dx_at(srf, unit)`

Per-element diffraction-grid pitch. dx = macos.dx_at(SRF) returns the pitch at element SRF in SI metres. dx = macos.dx_at(SRF, UNIT) returns it in the requested UNIT: 'm' SI metres (default). 'mm' millimetres. 'cm' centimetres.

fex

- **mmacos:** `s = macos.fex(mode)`

- **pymacos:** `pymacos.fex(mode)`

Find the Exit Pupil (FEX), return its geometry. s = macos.fex(MODE) runs the exit-pupil finder (placing the XP at element nElt-1) and returns: .vpt 3×1 pupil vertex (BaseUnits, global frame) .psi 3×1 pupil-surface normal (global) .rad reference-sphere radius (BaseUnits; = Kr at the XP)

ffp

- **mmacos:** `macos.ffp(place_elt, offset)`
- **pymacos:** `pymacos.ffp(place_elt, offset)`
- **engine API:** `macos_api_mod::ffp`

Place an off-axis field point by DIRECTION COSINES (angle on sky). macos.ffp(PLACE_ELT, OFFSET) tilts the source so the image at element PLACE_ELT lands at the off-axis field point OFFSET = [dx dy], given as DIRECTION COSINES (normalized; ~= field angle in radians for small angles). This is the “angle on the sky” way to place a planet – contrast with macos.pfp, which positions in focal-plane PIXELS. The

first_order_properties

- **mmacos:** `p = macos.first_order_properties(srf)`
- **pymacos:** `pymacos.first_order_properties(srf)`

First-order system properties (SYSPROP). p = macos.first_order_properties() returns a struct of the system’s first-order / diffraction properties at the focal plane (default = last element). p = macos.first_order_properties(SRF) uses element SRF. This is the SAME engine computation the interactive SYSPROP command prints and the design layer can target (e.g. an RC with

intensity

- **mmacos:** `I = macos.intensity(srf, opts)`
- **pymacos:** `pymacos.intensity(srf, reset_trace)`

|WFElt|^2 at element SRF (the SMACOS INT command). I = macos.intensity(SRF) runs INT at SRF and returns the (mdttl x mdttl) intensity buffer. Name-value pairs: ‘reset_trace’ true (default) — issue MODIFY + re-trace before

opd

- **mmacos:** `W = macos.opd(opts)`
- **pymacos:** `pymacos.opd(orient, sign)` (alias: opd_val)

OPD matrix from the most recent trace (N x N). W = macos.opd() returns the engine OPDMat (N = source-grid sampling). Call macos.trace() first. Conventions (see mmacos/doc/opd_conventions.md for the full story): * raw array is OPD(i,j) with FIRST index i = global X, SECOND

opd_mask

- **mmacos:** `M = macos.opd_mask(opts)`
- **pymacos:** `pymacos.opd_mask(orient)`

Which pixels of the last OPD map hold a ray (N x N logical). M = macos.opd_mask() is TRUE exactly where the engine’s last OPD wrote the map (engine 2026-10-10, api opd_mask_get), in the same (i,j) = (X,Y) layout as macos.opd(). Use it – NOT W ~= 0 – to find the pupil: under the chief-ray reference (the default) a valid ray whose path equals the chief’s reads EXACTLY 0, the value of an empty pixel.

opd_psf

- **mmacos:** `out = macos.opd_psf(rx_path, opts)`
- **pymacos:** *not available*

Load a prescription, trace, and display/save its OPD (+ PSF). OUT = macos.opd_psf(RX_PATH, …) is a simple, reusable front-end for eyeballing or capturing the wavefront of ANY .in file: it loads RX_PATH, (optionally) cleans up the exit pupil with FEX, traces to the wavefront element, and returns / displays / saves the OPD. Optionally it also runs INTENSITY to produce a PSF.

opd_ref

- **mmacos:** `out = macos.opd_ref(mode)`
- **pymacos:** `pymacos.opd_ref(mode)`

Choose (or read) the OPD map's reference. macos.opd_ref('chief') reference every ray to the CHIEF ray's own OPL. macos.opd_ref('mean') reference to the whole-aperture mean OPL. r = macos.opd_ref() read the current setting ('chief' | 'mean'). The engine (tracesub.F, SUBROUTINE OPD) fills OPDMat one of two ways:

pfp

- **mmacos:** `macos.pfp(place_elt, pix_size, offset)`
- **pymacos:** `pymacos.pfp(place_elt, pix_size, offset)`
- **engine API:** `macos_api_mod::pfp`

Place an off-axis field point in focal-plane PIXELS. macos.pfp(PLACE_ELT, PIX_SIZE, OFFSET) tilts the source so the image at element PLACE_ELT lands at pixel position OFFSET = [dx dy] on a grid of pitch PIX_SIZE (BaseUnits). This is the focal-plane-pixel sibling of macos.ffp (which places by direction cosines / sky angle); the two inputs differ by the plate scale (first_order_properties.plate_px_rad,

pupil_quality

- **mmacos:** `pq = macos.pupil_quality(ep_elt, opts)`
- **pymacos:** *not available*

Exit-pupil SURFACE quality of the loaded system. pq = macos.pupil_quality(EP_ELT) runs the engine XPS command (the field-differential exit-pupil finder – FEX generalized to the whole ray grid), reads the per-ray crossing cloud, and fits a low-order Zernike model of the pupil SURFACE in the FLAT entrance-pupil coordinate. Pure geometry, no OPD.

spot

- **mmacos:** `s = macos.spot(srf, opts)`
- **pymacos:** `pymacos.spot(srf, vpt_center, beam_csys, reset_trace)`

Ray-surface intersection (spot) at element SRF. s = macos.spot(SRF) returns a struct: .pts nSpot×2 intersection points in the spot coord frame .centroid 1×2 centroid (x,y) .shift 1×4 shift from the projected reference position .csys 3×3 spot coordinate-frame orientation

The `'ref'` option selects the coordinate frame the spot is expressed in — `'beam'` (default), `'tout'`, or `'telt'`; `'at'` selects the reference position (`'chief'` ray, default, or element `'elt'` vertex). The `'beam'` frame is derived from a chief-ray trace, so it is well-defined **even when the chief ray is vignetted** — e.g. the on-axis chief ray of a centrally- obscured telescope, which the secondary/central obscuration blocks. (Obscuration sets only the flux flag, not the geometric intersection, so the beam frame is still valid; earlier engine versions aborted the beam frame in that case, fixed in `tracesub.F LocalCoord`.) `'tout'` / `'telt'` instead use the element's stored output / element frame and never trace the chief ray.

srs

- **mmacos:** `macos.srs(iSlv1, iSlv2, opts)`
- **pymacos:** `pymacos.srs(slave, master, link)`

Slave an element to another via the SRS (Set Reference Surface) cmd. macos.srs(IS1, IS2) moves IS1 onto IS2 in the sense of macos's interactive SRS command – IS1's pose (vpt / psi / rpt) is recomputed from the chief-ray geometry as it crosses IS2's plane. macos.srs(IS1, IS2, 'link', true) creates a permanent linked-element

stop

- **mmacos:** `macos.stop(iElt, vpt_offset)`
- **pymacos:** `pymacos.stop(srf, offset)`

Set the system aperture stop to an element. macos.stop(IELT) declares element IELT as the system Stop. The chief ray is then aimed to pass through the centre of that element (VptElt(IELT) + offset). macos.stop(IELT, [DX DY]) places the stop at a local-frame offset

stop_info

- **mmacos:** `s = macos.get_stop_info()`
- **pymacos:** *not available*

Element and vertex offset of the system stop. s = macos.get_stop_info() returns a struct: .elt scalar element id at which the aperture stop is defined .offset 1×2 [dx,dy] offset from that surface's vertex Errors (mmacos raises) if no stop is currently set. See also:

stop_obj

- **mmacos:** `macos.stop_obj(x, y, z)`
- **pymacos:** `pymacos.stop_obj(x, y, z)`

Set the system aperture stop to an object-space point. macos.stop_obj(X, Y, Z) places the system Stop at the global coordinate (X, Y, Z) in BaseUnits. The chief ray is then aimed to pass through this point. Use this for Rx files that don't declare an ApStop= element-id

trace

- **mmacos:** `s = macos.trace(srf)`
- **pymacos:** `pymacos.trace_rays(srf)` (alias: traceWavefront)

Run a full ray trace through element SRF. s = macos.trace(SRF) returns a struct with fields: .nRays number of successful rays .rmsWFE RMS wavefront error (WaveUnits) SRF defaults to num_elt() (last element / image plane).

window

- **mmacos:** `macos.window(frame, sizPix, opts)`
- **pymacos:** `pymacos.window(frame, siz_pix, elt_pix, win_cen)`

Place diffraction images at their TRUE offset (pixel-loc ON). macos.window(FRAME, SIZPIX) turns on the WINDOW pixel-location option so PIX / COMPOSE place each source's image at its real sky offset (via the chief-ray projection) instead of re-centred on the grid – required to COMPOSE an off-axis source (e.g. a planet) at its true position relative to an on-axis star. Needs a prior trace.

window_off

- **mmacos:** `macos.window_off()`

- **pymacos:** `pymacos.window_off()`
- **engine API:** `macos_api_mod::window_off`

Turn the WINDOW pixel-location option back off. macos.window_off() cancels macos.window(…) so PIX / COMPOSE go back to re-centring each source's image on its own grid (the default). See also: macos.window, macos.compose.

xps

- **mmacos:** `macos.xps(iElt)`
- **pymacos:** `pymacos.xps(srf)`

eXit-Pupil Surface – per-ray exit-pupil crossing cloud. macos.xps(IELT) runs the engine XPS command (FEX generalized to the full ray grid): it crosses each ray's nominal exit ray with its field-differential partner, leaving the per-ray exit-pupil crossing point in the ray buffer. Read the cloud IMMEDIATELY afterwards with macos.get_ray_info(N) – the .pos field is the exit-pupil SURFACE,

Perturbation

perturb

- **mmacos:** `macos.perturb(srf, opts)`
- **pymacos:** `pymacos.perturb(srf, rotation_rad, translation_m, in_local_coords)`

Rigid-body perturbation of a single element. macos.perturb(SRF, …) applies a rotation + translation to element SRF via macos's CPERTURB_PROG (the programmatic sibling of the interactive CPERTURB). Unlike a raw set_elt_vpt, this performs the full bookkeeping: position + orientation, the element's local TElt frame matrix, aperture vector, HOE points,

perturb_many

- **mmacos:** `macos.perturb_many(srf_vec, prb, is_global)`
- **pymacos:** *not available*

Bulk rigid-body PERTURB of multiple elements. macos.perturb_many(SRFS, PRB, IS_GLOBAL) applies the array form (api prb_elt). Inputs: SRFS length-N vector of element ids. PRB 6xN matrix; column k = [Rx Ry Rz Tx Ty Tz]' for SRFS(k). Rotations in radians, translations in the Rx's

prb_elt

- **pymacos:** `pymacos.prb_elt(srf, prb, glb_csys)`
- **mmacos:** *not available*
- **engine API:** `macos_api_mod::prb_elt`

Apply 6-DoF rigid body perturbations to elements defined by srf

Jacobians (sensitivity channels)

dw_dgrid

- **mmacos:** `out = macos.dw_dgrid(session, rx_path, opts)`
- **pymacos:** *not available*

Finite-difference dW/d(grid-data) sensitivity Jacobian. out = macos.dw_dgrid(SESSION, RX_PATH) perturbs each grid-bearing surface (ANY GridData-enabled SrfType – GridData/AsGrData/MonGrData/ZrnGridData/ FreeForm) by a set of influence-function maps and returns the OPD-wavefront Jacobian dW/d(map amplitude), one column per (element, influence map).

dw_dgrid_multi

- **mmacos:** `out = macos.dw_dgrid_multi(session, rx_path, opts)`
- **pymacos:** *not available*

Multi-field dw/d(grid-data) sensitivity supervisor. out = macos.dw_dgrid_multi(SESSION, RX_PATH, …) loads RX_PATH on SESSION, snapshots the nominal source FoV, then loops field points – each iteration absolutely sets ChfRayDir via set_src_fov and runs dw_dgrid at the new field. The per-field results are tiled into one big OPDall canvas and scattered into one big dwdgall (canonical

dw_dsurf

- **mmacos:** `out = macos.dw_dsurf(session, rx_path, opts)`
- **pymacos:** *not available*

Single-field dw/dKr + dw/dKc sensitivity driver. out = macos.dw_dsurf(SESSION, RX_PATH) loads RX_PATH on SESSION, discovers the POWERED optics (Element= Reflector / Refractor / NSReflector / NSRefractor / Segment with |Kr| << the flat sentinel 1e22 – engine-queried, see macos.find_powered_elts), builds one channel per (optic, param)

dw_dsurf_multi

- **mmacos:** `out = macos.dw_dsurf_multi(session, rx_path, opts)`
- **pymacos:** *not available*

Multi-field dw/dKr + dw/dKc supervisor. out = macos.dw_dsurf_multi(SESSION, RX_PATH, …) loads RX_PATH on SESSION, snapshots the nominal source FoV, then loops field points – each iteration absolutely sets ChfRayDir via set_src_fov and runs dw_dsurf at the new field. The per-field results are tiled into one big OPDall canvas and scattered into one big dwdsall (canonical

dw_dx

- **mmacos:** `out = macos.dw_dx(session, rx_path, opts)`
- **pymacos:** *not available*

Single-field dw/dx rigid-body Jacobian driver. out = macos.dw_dx(SESSION, RX_PATH, …) loads RX_PATH on SESSION (optional – pass reload_rx=false to skip), builds rigid-body channels for every actual-optic element (and optionally source channels), runs the FD sweep at the current source state, and returns a struct of the result.

dw_dx_multi

- **mmacos:** `out = macos.dw_dx_multi(session, rx_path, opts)`
- **pymacos:** *not available*

Multi-field dw/dx rigid-body supervisor. Mirror of macos.dw_dz_zernike_multi but for the rigid-body Jacobian. Loads Rx once, builds source / per-element channels once, then loops field points – each iteration absolutely sets ChfRayDir via set_src_fov + modify(), runs dw_dx with reload_rx=false, and stacks per-field dwdx into the canonical

dw_dz_zernike

- **mmacos:** `out = macos.dw_dz_zernike(session, rx_path, opts)`
- **pymacos:** *not available*

Single-field dw/dz_Zernike sensitivity driver. out = macos.dw_dz_zernike(SESSION, RX_PATH) loads RX_PATH on SESSION (an open macos.Session), discovers MonZern + Zern eligible elements from the

Rx, builds one channel per (element, mode) pair, runs the finite-difference sweep at the current source state, and returns a struct of the result.

dw_dz_zernike_multi

- **mmacos:** `out = macos.dw_dz_zernike_multi(session, rx_path, opts)`
- **pymacos:** *not available*

Multi-field dw/dz_Zernike supervisor. out = macos.dw_dz_zernike_multi(SESSION, RX_PATH, …) loads RX_PATH on SESSION, snapshots the nominal source FoV, then loops field points – each iteration absolutely sets ChfRayDir via set_src_fov and runs dw_dz_zernike at the new field. The per-field results are tiled into one big OPDall canvas and scattered into one big

dwdx_trans_per_metre

- **mmacos:** `ox = macos.dwdx_trans_per_metre(ox)`
- **pymacos:** *not available*

Put a dw_dx harvest's translation columns per SI metre. OX = macos.dwdx_trans_per_metre(OX) takes a macos.dw_dx / dw_dx_multi result (live, or loaded from a jac .mat) and returns it with every TRANSLATION column (dof_idx 3..5) and the matching dcdx row expressed per SI METRE of translation, the convention the design runners (run_compare, run_simulator, run_met) and jacobian_check work in.

CALIB (native optimizer)

calib

- **mmacos:** `result = macos.calib()`
- **pymacos:** `pymacos.calib()`

Run the macos design optimizer (CALIB). result = macos.calib() invokes the SMACOS CALIB command using the current optimization configuration (variable elements, FOVs, wavelengths, target, iteration cap, tolerance). The config can be baked into the Rx file (OptTarget=, OptMxItrs=, VarDOF=, etc.) or set programmatically via the macos.calib_set_* family.

The result also carries `n_iter` (LM iterations taken), `merit` (the final chisq), `stop_reason` (`converged` / `floor` / `cap` / `stalled`; see calib_set_tol for the rule) and `n_rejected` / `n_failed`. `Telescope.optimize` hands `.stop` and `.n_iter` on in its result. `macos.calib_stats()` / `pymacos.calib_stats()` read the same three after a FAILED run, which `calib` reports as an error (`failed`, the iterations completed, the merit at the last accepted step). The count no longer depends on any earlier CALIB in the process (before 2026-10-11 a T <= 4 run exited on a flag the previous run had left). The CLI prints the same three lines after `Optimization iterations =`.

CALIB in a journal (the CLI): CALIB asks two questions when it finishes, `Display optimization summary (yes,no)` and `Save optimized optical state (yes,no)`, both defaulting to yes. A journal should answer both. If it does not – the next line is a command, e.g. `quit` – each prompt prints `CALIB: "quit" is not yes/no -- taking the default [yes]; the token goes back to the command stream`, takes the default (so the optimized state IS saved into the session, as an interactive Enter would) and the command then runs. Before 2026-10-11 the command was swallowed as an answer and the run hung on the terminal. (PLAN_CONSOLIDATION 3.3; journal `cli_tests/cmd/CALIB_journal_prompts.jou`.)

Aspheric DOFs in a prescription: the canonical keyword is `OptAsph= n t1 .. tn` (n terms, then the 1-based term indices of `AsphCoef`), written in the element block AFTER that element's `VarElt=` line (the parser keys it on the element being a variable; an `OptAsph=` above its `VarElt=` is ignored with no message today). `OptAsphCoef=` is accepted too – the parser matches the first 7 characters – and appears in one legacy

deck; new decks and the design layer's emitter use `OptAsph=` (2026-10-11, PLAN_CONSOLIDATION 3.2).

calib_clear_var_elts

- **mmacos:** `macos.calib_clear_var_elts()`
- **pymacos:** `pymacos.calib_clear_var_elts()`
- **engine API:** `macos_api_mod::calib_clear_var_elts`

Wipe all CALIB variable-element state. Use before defining a fresh variable-element list with macos.calib_set_var_elt; otherwise subsequent set calls accumulate on top of whatever the prescription's VarDOF= keywords already set.

calib_set_beam

- **mmacos:** `macos.calib_set_beam(kind, srf, target, varargin)`
- **pymacos:** `pymacos.calib_set_beam(kind, srf, target, on)`
- **engine API:** `macos_api_mod::calib_set_beam`

Beam rows for the next CALIB, on ANY target. macos.calib_set_beam('dir', srf, n) – the chief ray's direction at element srf is driven to the unit vector n (a telecentric image: n = the detector normal). macos.calib_set_beam('pos', srf, p) – the beam position at srf (chief ray, or the ray centroid with 'centroid' in

calib_set_beam_pos_fov

- **mmacos:** `macos.calib_set_beam_pos_fov(pos)`
- **pymacos:** `pymacos.calib_set_beam_pos_fov(pos)`
- **engine API:** `macos_api_mod::calib_set_beam_pos_fov`

Per-field beam position targets for CALIB. macos.calib_set_beam_pos_fov(POS), POS 3 x n (or n x 3): the target position of the beam (see calib_set_beam 'pos') for CALIB field 1..n in the order of the deck's OptChfRayDir= / OptChfRayPos= list (n <= 12). Fields beyond n use the calib_set_beam 'pos' target. macos.calib_set_beam_pos_fov([]) clears the table.

calib_set_beam_wt

- **mmacos:** `macos.calib_set_beam_wt(wt, centroid)`
- **pymacos:** `pymacos.calib_set_beam_wt(wt, centroid)`
- **engine API:** `macos_api_mod::calib_set_beam_wt`

Weight of the CALIB beam rows, and the centroid option. macos.calib_set_beam_wt(WT) weights every beam row (calib_set_beam) against the WFE / SPOT rows: the row's sigma is divided by sqrt(WT), so WT = 1e6 makes a 1 um position error count as a 1 mm one. Mind the units: WFE rows are in the target's units, SPOT rows and beam rows in base units.

calib_set_iter

- **mmacos:** `macos.calib_set_iter(n_iter)`
- **pymacos:** `pymacos.calib_set_iter(n_iter)`
- **engine API:** `macos_api_mod::calib_set_iter`

Set CALIB iteration cap (nitrs_dopt).

calib_set_target

- **mmacos:** `macos.calib_set_target(target, varargin)`

- **pymacos:** `pymacos.calib_set_target(target, wf_zern_modes)`
- **engine API:** `macos_api_mod::calib_set_target`

Set the CALIB optimization target. macos.calib_set_target(NAME) selects one of: ‘WFE’ – RMS wavefront error ‘WFE_ZMODE’ – specific Zernike modes (alias ‘ZWF’) ‘BEAM’ – beam waist / position / divergence ‘SPOT’ – RMS spot size

calib_set_tol

- **mmacos:** `macos.calib_set_tol(tol)`
- **pymacos:** `pymacos.calib_set_tol(tol)`
- **engine API:** `macos_api_mod::calib_set_tol`

Set CALIB convergence tolerance (dopt_tol).

What the tolerance means (2026-10-11, PLAN_CONSOLIDATION 3.3). `tol` is OptTol: an ABSOLUTE change of the rms merit, `sqrt(chisq/rows)` in the target’s own units (WFE: the rms wavefront error; SPOT / BEAM: the rms of their residual rows, base units). Default 1e-12. In a prescription: `OptTol= 1e-9` (2026-10-11). It is a per-load value: every load resets it to 1e-12, the deck’s `OptTol=` sets it, and `calib_set_tol` after the load overrides it for that deck. Like every `Opt*` key it is not written by SAVE. The iteration cap is `OptMaxItrs=` (`OptMxItrs=` is accepted too; before 2026-10-11 it was silently ignored). The CLI’s CALIB banner prints both values in force (`LM stop: OptTol (abs rms) = ...`). The LM (the default `NonLin` algorithm) stops for one of three reasons, reported as `stop_reason` by `calib` / `calib_stats`:

reason	condition
`converged`	two consecutive ACCEPTED steps, each moving the rms merit by no more than `tol` AND lowering chisq by no more than `1e-6` of it
`floor`	ten consecutive REJECTED trials since the last accepted step (lambda up 1e10, no decrease: the round-off floor) – after progress
`cap`	the iteration cap (`calib_set_iter`, Rx `OptMaxItrs=` / `OptMxItrs=`)
`stalled`	NO trial step was accepted and none failed: the start is a stationary point of the merit (typically already optimal). Not an error; `converged` is true; `n_iter == n_rejected`
`failed`	the LM failed (a singular step, e.g. a DOF the merit cannot see), the initial evaluation failed, or NO trial step was accepted and some could not be evaluated (‘no trial step was accepted – N of K trials lost the chief ray’). The bindings raise a catchable error; the optics are restored to the last accepted state

A trial whose evaluation FAILS (a trace inside it lost the chief ray, e.g. a step that swung a mirror far off) is a REJECTED trial: the damping grows and a smaller step follows. `calib_stats` reports `n_rejected` and, of those, `n_failed` (2026-10-11, PLAN_CONSOLIDATION 3.4).

The `1e-6` relative term is a fixed engine constant (MINPACK’s `ftol` idea), not a knob: on a merit of a few nm the first steps of an LM still ramping its damping move the rms by ~1e-12 and would otherwise read

as converged. `tol` is the only user setting; a coarser `tol` stops sooner, never before the relative condition holds. Neither test runs before iteration 2. The linear (SVD) algorithm keeps its own schedule.

calib_set_var_elt

- **mmacos:** `macos.calib_set_var_elt(srf, varargin)`
- **pymacos:** `pymacos.calib_set_var_elt(srf, dofs, zern_modes)`
- **engine API:** `macos_api_mod::calib_set_var_elt`

Mark an element as a CALIB variable. macos.calib_set_var_elt(SRF, dof_name1, dof_name2, …) marks element SRF as a variable with the named DOFs free. Valid names (case-insensitive): TIP, TILT, CLOCK, DX, DY, PIST, ROC, CONIC. macos.calib_set_var_elt(SRF, mask) accepts an 8-int positional

calib_stats

- **mmacos:** `st = macos.calib_stats()`
- **pymacos:** `pymacos.calib_stats()`

Iteration count and final merit of the last CALIB. st = macos.calib_stats() returns a struct: .n_iter (double) LM iterations the last CALIB took; -1 when none ran (no CALIB yet, or it stopped before the solve); the linear (SVD) path reports its count .merit (double) the final LM merit – the chisq the solve minimises,

Segment and grid bases

segment_grid_basis

- **mmacos:** `out = macos.segment_grid_basis(session, rx_path, opts)`
- **pymacos:** *not available*

Per-segment Zernike grid (GridMat) generator. OUT = macos.segment_grid_basis(SESSION, RX_PATH, …) steps through every grid-bearing segment of a SEGMENTED prescription and builds, IN THAT SEGMENT'S OWN clocked (xData,yData) frame, a bespoke aperture mask plus a stack of Zernike figure modes sampled on the segment's N x N grid – i.e. the per-segment GridMat content.

write_grid_file

- **mmacos:** `macos.write_grid_file(path, M)`
- **pymacos:** *not available*

Write an N x N grid to a MACOS GridFile (ASCII). macos.write_grid_file(PATH, M) writes the [N x N] matrix M to a formatted grid-data file that the engine's GridInit (surfsub.F) reads back into GridMat with NO transpose. The reader is DO j=1,N: READ (GridMat(i,j), i=1,N) i.e. file line j is GridMat column j. fprintf serialises M column-major,

zernike_grid_basis

- **mmacos:** `B = macos.zernike_grid_basis(N, modes, ap_frac, convention)`
- **pymacos:** *not available*

Engine-exact Zernike maps sampled on an N×N grid. B = macos.zernike_grid_basis(N, MODES) returns an [N×N×K] array of K Zernike-mode shapes sampled on a surface's N×N grid, using MACOS's OWN convention so that a grid poke and the matching MonZern coefficient produce the IDENTICAL sag – left/right pairs overlay.

zrn_freeform

- **mmacos:** `out = macos.zrn_freeform(srf, opts)`
- **pymacos:** `pymacos.zrn_freeform(srf, zrn_1, zrn_2, grid)`

Get or set the composite description of a FreeForm element. out = macos.zrn_freeform(SRF) returns a struct with fields: .ff [] or struct with fields zern_type, modes, coefs, norm_rad, csys (csys: struct with pos, x, y, z 3-vectors) .mon [] or same shape as .ff (the MonZern overlay) .grid [] or struct with fields dx, mat, csys

Other utilities

beam

- **mmacos:** `out = macos.beam(type, opts)`
- **pymacos:** `pymacos.beam(kind, waist, radius, power)`

Shape the source amplitude (apodization) profile. macos.beam(TYPE, …) sets the source beam profile that macos applies to the aperture amplitude before tracing. Setting the beam resets the trace, so re-trace afterwards. TYPE (case-insensitive):

coating

- **mmacos:** `out = macos.coating(srf, opts)`
- **pymacos:** `pymacos.coating(srf, index, extinc, thickness)`

Set or query the thin-film coating stack on an element. macos.coating(SRF, 'index', N, 'extinc', K, 'thickness', T) sets a multilayer coating on element SRF (the polarization-path coating, active only when macos.polarization('on') is set). Layers are ordered OUTERMOST -> INNERMOST (matching the 'Coating=' Rx keyword). N, K, T are equal-length vectors (one entry per layer):

This drives the polarization-path ("Model A") coating stack: `EltCoat/IndRefArr/ExtincArr/EltCoatThk`, applied by the s/p Fresnel recursion in Reflector/Refractor when polarization is on. Thickness here is PHYSICAL, in the Rx BaseUnits – unlike the `Coating=` Rx keyword, whose per-layer thickness is in WAVES at the Wavelen current at parse time (converted to physical at load). `coating(srf)` with no layer args queries; set followed by query round-trips exactly. The incident medium is the medium the ray actually travels in (CurIndRef), not the previous element's substrate. An optically thick absorbing layer (thickness >> skin depth) reproduces the bare metal's Fresnel coefficients regardless of substrate – the standard way to model a bare-metal mirror. Query returns `n_layer=0` for an uncoated element; `load_rx` clears all coating state. *Related:* polarization, jones_pupil; Rx keywords `Coating=`, `IndRef=`, `Extinc=`.

config_canvas

- **mmacos:** `[OPDall, indxall] = macos.config_canvas(canvases, tile_rc)`
- **pymacos:** *not available*

Lay per-configuration canvases out, and index them. [OPDall, indxall] = macos.config_canvas(CANVASES) concatenates the per-configuration field canvases in CANVASES (a 1xNc cell of equal-size 2-D maps, each one configuration's own field-tiled canvas) left to right, and returns the combined canvas with the m2v-style index struct that addresses it.

dark_zone_metrics

- **mmacos:** `m = macos.dark_zone_metrics(I, peak_unaberrated, lam_over_D_px, ...`
- **pymacos:** *not available*

TODO: expand — dialog details, behavior notes, related commands.

deadband_notes

- **pymacos:** `pymacos.deadband_notes()`
- **mmacos:** *not available*

Source-frame / stop-aim updates the round-off dead band suppressed beyond the first one the engine printed (0 on every deck measured 2026-10-02).

ffcut

- **mmacos:** `varargout = macos.ffcut(on)`
- **pymacos:** `pymacos.ffcut(on)`

The far-field evanescent cut (engine 2026-10-03, dyson5 addendum 15). macos.ffcut(true) makes every far-field leg zero the output pixels with x^2 + y^2 > dz^2 – |sin theta| > 1, spatial frequencies above 1/lambda, which carry no propagating energy but which a wide output window (a pupil sampled finer than lambda/2) otherwise hands to an energy-fraction metric. macos.ffcut(false) restores the default (off).

first_airy_null

- **mmacos:** `r_null = macos.first_airy_null(intensity, search_min_px, search_max_px, ...`
- **pymacos:** *not available*

TODO: expand — dialog details, behavior notes, related commands.

fwd_root_flips

- **pymacos:** `pymacos.fwd_root_flips()`
- **mmacos:** *not available*

Surface intersections in the last trace where the forward-crossing root pick differed from the legacy metric (reset at every trace start).

getEltSrfZernMode

- **pymacos:** `pymacos.setEltSrfZernMode(iElt, izernMode, zernCoef)`
- **mmacos:** *not available*

ToDo

grid_dropped

- **mmacos:** `n = macos.grid_dropped()`
- **pymacos:** `pymacos.grid_dropped()`

Grid files the last load declined (engine 2026-10-10). n = macos.grid_dropped() counts the GridFile= grids larger than this model’s mGridMat (macos_param.txt): they cannot be stored, so the load declined them by name and those surfaces trace WITHOUT their figure. Nonzero means a harvest at this model is missing figure data – load at a model whose mGridMat fits (e.g. a 256-point grid needs model 256).

grid_idx_ovf

- **mmacos:** `n = macos.grid_idx_ovf()`
- **pymacos:** `pymacos.grid_idx_ovf()`

Grid-index overflow samples rejected by the last trace. n = macos.grid_idx_ovf() is the engine’s nGridIdxOvf (macos db9236b, api grid_idx_ovf_get): the number of grid-term samples whose pixel index xi/yj was non-finite or >= 2e9 and was therefore treated as OFF the grid (fh = 0) instead of indexing GridMat out of bounds – before that guard an INT32 wrap passed the bounds test and killed the

grid_size_max

- **mmacos:** `n = macos.grid_size_max()`
- **pymacos:** *not available*

Max permitted grid sampling for the current model size. n = macos.grid_size_max() returns mGridMat – the largest square grid a surface may carry at the initialised model size. A grid larger than this cannot be loaded/set (see macos.set_elt_grid).

gs_zernike_segment_basis

- **mmacos:** `[B, mask, info] = macos.gs_zernike_segment_basis(session, rx_path, opts)`
- **pymacos:** *not available*

Gram-Schmidt-orthonormalized Zernike basis over a segment's TRUE (irregular) aperture – a grid-poke influence basis that respects the segment shape. Circular Zernikes cross-talk on a hex / edge segment; manufacturers spec figure over the segment aperture, so the modes are orthonormalized over that aperture (MGS).

jones_pupil

- **mmacos:** `jp = macos.jones_pupil(srf, opts)`
- **pymacos:** `pymacos.jones_pupil(srf, basis, axis, xref)`

2x2 Jones pupil at an element from two polarized traces. jp = macos.jones_pupil(SRF) traces the loaded prescription twice with orthogonal source polarization states, (Ex0,Ey0) = (1,0) and (0,1), harvests the per-ray vector E-field at element SRF (macos.ray_field), and assembles the 2x2 complex Jones matrix at every ray-grid point:

The INPUT basis is the engine launch frame: collimated sources launch every ray with E = S*(Ex0*xGrid + Ey0*yGrid) (uniform over the grid); point sources use a per-ray frame yray = unit(RayDir x xGrid), xray = yray x RayDir, which reduces to the global frame on the chief ray. S is the engine flux normalization (~1/sqrt(nRays)), a common real scalar carried by J – ratios and all pol_maps metrics are unaffected. The OUTPUT basis options: 'double-pole' (default; Chipman double-pole coordinates, smooth over any physical pupil – use for budget numbers), 'local-sp' (per-ray s/p about the exit axis; coordinate-singular on axis, DIAGNOSTIC ONLY – the singularity imprints spurious tilt/astig-like retardance), and 'global' (project onto a fixed pair; near-collimated diagnostics). Vignetted points are NaN, never zero-filled. The two traces are geometry-identical by construction (asserted); the pre-call polarization state is restored on exit. A stock conductor-mirror Rx (IndRef=1, Extinc=1e22) yields a unitary Jones pupil – a free end-to-end sanity gate.* Related:* pol_maps, ray_field, polarization, coating.

lambda_over_D_pixels

- **mmacos:** `lamD = macos.lambda_over_D_pixels(unaberrated_psf, search_min_px, search_max_px)`
- **pymacos:** *not available*

lambda/D in pixels, derived from an Airy PSF. LAMD = LAMBDA_OVER_D_PIXELS(PSF) returns lambda/D in pixel units by locating the first Airy null of a centred un-coronagraphed PSF (first null at 1.22 lambda/D for a circular pupil). This is the EMPIRICAL conversion that avoids having to compute the effective pupil diameter at the science focal plane analytically (it depends

m2v

- **mmacos:** `[vec, jndx] = macos.m2v(mat, indx)`
- **pymacos:** *not available*

Vectorise a 2D OPD matrix preserving non-zero index map. [vec, indx] = macos.m2v(mat) first call – find non-zeros + cache. vec = macos.m2v(mat, indx) reuse cached index struct. indx is a struct with fields: .i Nnz×1 row indices of non-zero positions

messages

- **mmacos:** `lines = macos.messages(clear)`
- **pymacos:** `pymacos.messages(clear)`

The engine’s messages since the last read. lines = macos.messages() returns the engine’s kept messages as an n×1 string array, oldest first, trailing blanks trimmed, and empties the buffer. macos.messages(false) reads without emptying it. The load path’s notes, warnings and refusals, CALIB’s cause lines and the SMACOS loop’s

met

- **mmacos:** `out = macos.met(unit)`
- **pymacos:** `pymacos.met(unit)`

Laser-metrology beam lengths (engine METcalc). out = macos.met() runs the engine metrology compute over the loaded Rx’s declared met beams (the nMetPos / tMetElt / metBeamFlg element keywords) and returns out.l n-by-1 beam lengths, SI metres (default) out.n system beam count (0 when the Rx declares no metrology)

met_geom

- **mmacos:** `g = macos.met_geom()`
- **pymacos:** `pymacos.met_geom()`

Geometry of the loaded Rx’s laser-metrology beams. g = macos.met_geom() runs the engine metrology compute (METcalc) and returns the global endpoints of every gauge beam declared by the prescription’s nMetPos / tMetElt / metBeamFlg keywords: g.src_pts 3 x n gauge source points (launchers), BaseUnits g.tgt_pts 3 x n gauge target points (fiducials), BaseUnits

mimg

- **mmacos:** `macos.mimg(data,stat,crop,ndec,cmap)`
- **pymacos:** *not available*

Display a 2D map (OPD, Jacobian column, mask) with statistics. macos.mimg(DATA) macos.mimg(DATA, STAT, CROP, NDEC, CMAP) STAT = 0 no statistics (DEFAULT) = +/-1 display RMS and PV

model_size

- **pymacos:** `pymacos.model_size()`
- **mmacos:** *not available*

Returns the MACOS model size

model_size_min

- **mmacos:** `n = macos.model_size_min()`
- **pymacos:** *not available*

Smallest model size the engine accepts (128). See macos.model_sizes for the full set.

model_sizes

- **mmacos:** `s = macos.model_sizes()`
- **pymacos:** *not available*

The model sizes the engine accepts. s = macos.model_sizes() returns [128 256 512 1024 2048 4096 8192] – the values param_mod.F's param_mod_init will accept. ANY OTHER VALUE MAKES THE ENGINE CALL `stop`, which terminates the host MATLAB process; macos.init screens against this list so a typo raises a MATLAB error instead.

modes

- **mmacos:** `md = macos.modes()`
- **pymacos:** `pymacos.modes()`

The five SESSION modes as the engine holds them. md = macos.modes() returns a struct with logical fields .pol (polarization), .vecdif (vector diffraction), .centroid (the FEX/SXP centroid axis), .lneg (negative ray lengths allowed; default TRUE), .cir (colour image rendering; always false in the bindings' engine), and .in_force, a cellstr of the modes NOT at their defaults, named as the CLI's LOAD notice names them

mon_zrn_max_modes

- **mmacos:** `n = macos.mon_zrn_max_modes()`
- **pymacos:** *not available*

Number of monomial-Zernike coefficient slots per elt. n = macos.mon_zrn_max_modes() returns mMonCoef – the max Mon-Zernike mode index a FreeForm surface can carry. Use to size mode arrays without hard-coding. See also: macos.get_elt_mon_zrn_coef.

noll_mode

- **mmacos:** `[zern, m, n] = macos.noll_mode(pupil, imode)`
- **pymacos:** *not available*

NOLL_MODE Noll Zernike mode over an arbitrary pupil support (self-contained). [Z, M, N] = NOLL_MODE(PUPIL, IMODE) evaluates Noll mode IMODE (1-based; Noll 1976 ordering: even index -> cosine, odd -> sine) on the support of PUPIL (any nonzero = inside), normalized sqrt(n+1) (x sqrt(2) for m>0) over the unit disk that circumscribes the support.

obs

- **mmacos:** `out = macos.obs(opt)`
- **pymacos:** *not available*

Query or set the ray-trace obscuration option (OBS). OPT = macos.obs() returns the option in force (api obs_opt_get): 'all' (every ray), 'positive' (unobscured rays only: the engine default) or 'negative' (obscured rays only). macos.obs(OPT) sets it (OBS). Session state: persists until changed, across loads. Twin of pymacos obs_set (Dave 2026-10-16).

obs_set

- **pymacos:** `pymacos.obs_set(option)`
- **mmacos:** *not available*
- **engine API:** `macos_api_mod::obs_set`

OBS – set the ray-trace obscuration option for spot diagrams.

ors

- **pymacos:** `pymacos.ors(srf)`
- **mmacos:** *not available*

ORS – Optimize Reference Surface.

pad

- **mmacos:** `[out] = macos.pad(in, npix_rows, npix_cols)`
- **pymacos:** *not available*

Zero-pad (or centre-crop) a matrix to a target size. OUT = macos.pad(IN, NPIX) square NPIX-by-NPIX result OUT = macos.pad(IN, NPIX_ROWS, NPIX_COLS) The input is centred on the output using the floor(n/2+1) centre convention; a target smaller than the input centre-crops instead.

pol_contrast_floor

- **mmacos:** `out = macos.pol_contrast_floor(pupil, det, opts)`
- **pymacos:** *not available*

Polarization-limited contrast floor at a detector. OUT = macos.pol_contrast_floor(PUPIL, DET) propagates the loaded prescription with vector diffraction on and splits the detector field into CO-polarized, CROSS-polarized and LONGITUDINAL channels. The cross-polarized channel is the part of the light no scalar DM control can touch, so its peak-normalized level IS the polarization contrast

The split is taken AT THE DETECTOR, on the engine's own Ex/Ey/Ez component planes (`complex_field(det,'plane',k)`), not through a pupil multiplier. That is legitimate because the chain is linear in the input Jones state and Tranche 1 propagates all three planes with the identical scalar kernel, so a spatially uniform analyzer commutes with propagation. It also sidesteps the reason the two earlier designs failed: the Jones pupil is assembled from `RayE` and carries the accumulated OPL phase, so it can never be used as a `WFElt` multiplier.

"Co-polarized" is referenced to the MEAN OUTPUT state, never to the input state. A train can rotate polarization geometrically with zero diattenuation and zero retardance; billing that uniform rotation as cross-polarized light reports an aberration where there is none. The analyzer is the dominant eigenvector of the 2x2 pupil COHERENCY matrix `C_ij = sum E_i conj(E_j)`, which is phase-insensitive (the common wavefront cancels in the product) and by construction minimizes cross-polarized power. `.per_state.dop` reports how polarized the pupil actually is; below `dop_min` no analyzer is well defined and the run is flagged.

`input='unpolarized'` is TWO traces, x-in and y-in, summed in intensity – the second state is never synthesized from the first – and each run gets its own analyzer.

SCOPE. Tranche 1 seeds the component planes from `RayE` at the FIRST physical-optics leg and thereafter applies only a common scalar phase, so a polarizing surface AFTER that leg transforms the rays but not the diffraction grid. On such a chain the floor is a LOWER BOUND. This is measured rather than assumed: `.scope` compares the pupil cross-polarized fraction computed from the grid planes against the same quantity from `RayE`, per input state, and a `macos:pol_contrast_floor:tranche1` warning fires when they disagree. Rx_Cass_FarField (two mirrors, then one far-field hop) carries the full train; Rx_Coro carries 0.84 of it bare and 0.57 coated, and there its coating sensitivity even comes out with the wrong sign. Closing that is Tranche 2.

`frac_cross` NaN-masks pixels whose denominator falls below `floor_tol` – zero-filling would seed statistics with a ring of perfect nulls. Coating sets in a `coatings` sweep must all cover the same elements (a coating can be overwritten but not cleared) and the sweep leaves the last set applied. *Related:* jones_pupil, pol_maps, complex_field, vector_diffraction, coating.

pol_maps

- **mmacos:** `pm = macos.pol_maps(jp)`

- **pymacos:** `pymacos.pol_maps(jp)`

Polarization-aberration decomposition of a Jones pupil. pm = macos.pol_maps(JP) decomposes the Jones pupil JP (the struct returned by macos.jones_pupil, or any struct with fields .J [N x N x 2 x 2 complex, NaN where invalid] and .mask [N x N logical]) into the standard polarization-aberration maps via the per-point polar decomposition J = H * W (H hermitian >= 0, W unitary).

All algebra is closed-form 2x2 Pauli (vectorized; no per-point SVD). Pauli ordering: s1 = 0/90 linear, s2 = +/-45 linear, s3 = circular. Key outputs: `T` intensity transmission (carries the source normalization – ratios only), `D`/`Dvec` diattenuation, `ret`/`retvec` retardance (canonical [0, pi]; points within 0.2 rad of pi are flagged `ambiguous` – the branch is unresolved there, never silently chosen), `phase` (unitary global phase, mod pi). The pupil MEAN and the RMS VARIATION are reported separately and must not be conflated: uniform retardance/diattenuation is a state change (and, after fold mirrors, includes the system's geometric rotation), not an aberration – only the variation drives a contrast floor or a PSI systematic. `D` and `T` are singular-value invariants (identical in any output basis); `ret`/`retvec` are basis-dependent – exactly what the double-pole basis exists to make artifact-free. *Related:* jones_pupil.

pol_zernike

- **mmacos:** `pz = macos.pol_zernike(pm, opts)`
- **pymacos:** `pymacos.pol_zernike(pm, modes, center, radius, orthonormalize)`

Low-order Zernike expansion of polarization-aberration maps. pz = macos.pol_zernike(PM) expands the diattenuation and retardance maps in PM (the struct returned by macos.pol_maps) onto a Zernike basis over the unvignetted pupil, giving the standard polarization-aberration terms – piston, tilt, defocus, astigmatism and up – in each Pauli component.

Turns the maps from `pol_maps` into the aberration terms the polarization-aberration literature is written in, so a MACOS result can be compared term-by-term instead of map-shape by map-shape.

Mode indices are the MACOS ANSI 1-based numbers you write in `MonZernModes=` (1 piston, 2 tilt-y, 3 tilt-x, 4 astig45, 5 defocus, 6 astig0, 13 spherical, …) – the same convention as `zernike_grid_basis`, deliberately, and the two share one evaluator so they cannot drift apart. Default `modes` is 1..15.

The fit is LEAST SQUARES, not a projection: circular Zernikes are not orthogonal over an obscured (annular) pupil, so a projection would cross-talk between terms. `.cond` reports the conditioning over the actual mask. `orthonormalize=true` Gram-Schmidts the basis over that mask first – coefficients then become mutually independent but are no longer standard Zernike coefficients, so use it for energy bookkeeping and NOT for literature comparison.

Mode 1 (piston) is the pupil MEAN; everything else is variation about it. Keep them apart – a uniform diattenuation or retardance is a state change (after folds it also absorbs the system's geometric rotation), not an aberration, and only the variation drives a contrast floor or a PSI systematic.

Expected answer for an on-axis rotationally symmetric two-mirror system: POLARIZATION ASTIGMATISM and nothing else – astig0 in the s1 component, astig45 in s2, equal magnitude, no circular (s3) content, no defocus. If you get piston, tilt, coma or a circular term above round-off on such a system, suspect the reference frame before the optics.

Retardance caveat: points flagged `pm.ambiguous` (retardance within 0.2 rad of pi, where the branch is unresolved) are EXCLUDED from the retardance fits; compare `.npts_ret` against `.npts` before reading a retardance expansion. *Related:* pol_maps, jones_pupil, zernike_grid_basis.

polarization

- **mmacos:** `out = macos.polarization(state, opts)`
- **pymacos:** `pymacos.polarization(state, Ex, Ey)`

Turn polarized ray tracing on/off + set source state. macos.polarization('on', …) enables polarized ray tracing (the engine POLARIZATION command): rays carry a complex 3-vector E-field, surface coatings become active, and vector diffraction is enabled when the model supports it (mWF>=3, true for all stock model sizes). macos.polarization('off') restores scalar tracing (NOPOLARIZATION).

Query form (`macos.polarization()` with no args / `state=None`) returns the current state: `.on`, `.vector`, `.Ex`, `.Ey`. Setting the state dirties the cached trace (the engine re-seeds every ray's E-field on the next trace) – re-trace before harvesting. Enabling requires mWF>=3 (all stock model sizes); the call errors rather than silently degrading. *Related:* vector_diffraction, coating, ray_field, jones_pupil; CLI POLarized/NOPolarization.

polarizer

- **mmacos:** `out = macos.polarizer(srf, opts)`
- **pymacos:** `pymacos.polarizer(srf, axis)`

Set or query an ideal linear polarizer element. macos.polarizer(SRF, 'axis', A) sets the TRANSMISSION axis of the TrPolarizer element at SRF. A is a 3-vector in GLOBAL coordinates; it need not be unit length, and it need not lie in the element's surface. The engine projects it into each ray's transverse plane and normalizes (see PolElt in elemsub.F).

Element type. Requires the element at `srf` to be declared `Element= TrPolarizer` (EltID 15) in the prescription; any other type is refused rather than silently ignored. Before PLAN_POLARIZATION Phase 3 this EltID existed in the name table only and had no trace dispatch at all, so a prescription naming it loaded cleanly and did nothing.

Ideal, and one-port. Transmission is 1 along the axis and 0 across it – no Fresnel loss at the plate faces, no substrate, no wavelength dependence. The REJECTED component is discarded, not emitted, so a polarizing beamsplitter needs two traces. At 45 degrees a coated `Reflector` is the better model: there the s/p basis is physically meaningful and the coating recursion is already gated against the Fresnel closed form.

Requires `polarization('on')`. With polarization off the element is a plain geometric surface and the axis is inert – a trace through it is bit-identical to the same train with a `Reference` surface in its place (gated by `tPolElement/test_unpolarized_bit_identical_to_reference_twin`).

Off normal, the MATERIAL axis is what gets projected. The declared axis has to be brought into the plane transverse to the ray, and orthographic projection does not preserve orthogonality, so projecting the declared PASS axis is not the same operation as projecting the BLOCK axis and transmitting the complement – the two differ by `acos(2cos a/(1+cos^2 a))`, 3.56 degrees of transmitted-axis orientation at 20 degrees AOI, and identically zero at normal incidence or when the axis lies in, or perpendicular to, the plane of incidence. The engine projects the material (absorbing) direction `psi x axis`, extinguishes it, and transmits its orthogonal partner: the dipole rule, measured for a tilted dichroic polarizer by Korger et al., *Opt. Express* **21**, 27032 (2013). The keyword still declares the PASS axis; only the component perpendicular to the element normal matters, and an axis parallel to the normal extinguishes. Gated at 20 degrees AOI on both dispatch chains by `tPolElement` section F. `RfPolarizer` (EltID 14) remains a stub, now for a different reason: a reflective wire grid carries grid efficiency and substrate s/p physics beyond the axis rule.

Axis storage. The API stores the axis as given, so a query returns what was written; the Rx parser UNITIZES on load (matching `psiElt=`), so a non-unit axis comes back normalized after a save/reload round trip. Either way it is a direction – `PolElt` normalizes per ray.

pupil_zone_map

- **mmacos:** `z = macos.pupil_zone_map(pupil_elt, image_elt, opts)`
- **pymacos:** *not available*

Pupil-resolved imaging quality (zone -> image spot). z = macos.pupil_zone_map(PUPIL_ELT, IMAGE_ELT) partitions the ray grid into zones by each ray's TRANSVERSE position at PUPIL_ELT, and for each zone measures the RMS transverse spread of those same rays at IMAGE_ELT. Conceptually: a cone of rays leaves each patch of the pupil; a perfectly imaging system maps every patch to the SAME image

radial_contrast

- **mmacos:** `[r_lamD, contrast] = macos.radial_contrast(intensity, peak_unaberrated, ...`
- **pymacos:** *not available*

TODO: expand — dialog details, behavior notes, related commands.

radial_profile

- **mmacos:** `[centers, means, stds, ns] = macos.radial_profile(img, center, max_radius, bin_size)`
- **pymacos:** *not available*

Azimuthally-averaged radial profile of a 2D image. [centers, means, stds, ns] = RADIAL_PROFILE(IMG) returns the bin centres, per-bin mean, per-bin std, and per-bin pixel count of the azimuthal average of IMG about its array centre. MATLAB port of contrast.py:radial_profile (pymacos

ray

- **mmacos:** `r = macos.ray(iRay)`
- **pymacos:** `pymacos.ray(i_ray)`

Trace ray iRay of the source grid (1 = the chief ray) – the CLI's RAY. r = macos.ray(iRay) iRay in 1..nRays of the current grid (macos.trace().nRays) Traces to the last element (or the deck's ray-target element). Fields as macos.srt (column k = element k, BaseUnits / global frame): r.pos 3 x nElt the crossing at element k; r.dir 3 x nElt the direction leaving it r.n_prev, r.n_cur 1 x nElt the index before / after

ray_field

- **mmacos:** `rf = macos.ray_field(srf)`
- **pymacos:** `pymacos.ray_field(srf)`

Per-ray complex E-field + geometry at an element. rf = macos.ray_field(SRF) returns, on the N x N ray grid at element SRF (N = model size), the per-ray complex electric field RayE(3,:) plus the ray direction cosines, the element surface normal, and the per-ray status. Requires a polarized trace: call macos.polarization('on') and trace before this.

Returns NaN-free zero fields with `status~=0` at vignetted / failed grid points – always mask on `status==0` before statistics. The surface normal returned is the per-element `psiElt` broadcast to the grid (exact for flats; for curved elements it is the nominal element normal, not the local surface normal at each intersection). The field at an element is the field AFTER that element's reflection/refraction, including the

propagation phase of the leg arriving at it. mmacos returns separate `.Ex/.Ey/.Ez/.kx../.nx..` arrays; pymacos stacks them as `E/k/n` with shape (N, N, 3). *Related:* polarization, jones_pupil, trace_rays.

ray_hist

- **mmacos:** `out = macos.ray_hist(arg)`
- **pymacos:** `pymacos.ray_hist(state)`

Per-trace ray-position history (engine RayPosHist). The engine can record EVERY ray's global 3-D crossing at EVERY element during a trace (traceutil_mod RayPosHist/LRayOKHist – the Vis3D substrate). Capture is off by default: macos.ray_hist('on') enable capture (then run macos.trace)

ray_info

- **mmacos:** `r = macos.get_ray_info(n_rays)`
- **pymacos:** `pymacos.getRayInfo(nRays)`

Read per-ray data from the most recent trace. r = macos.get_ray_info(N) returns a struct with fields: .pos 3×N double — intersection position with current surface .dir 3×N double — incoming direction at that surface .opl N×1 double — cumulative OPL from source .ok_trace N×1 logical — true if the ray traced successfully

ray_pos_hist

- **pymacos:** `pymacos.ray_pos_hist(n_rays)`
- **mmacos:** *not available*

The ray-position history of the last trace (mmacos `ray_hist(nRays)`): `P` (3 x n_rays x nElt+1; column 0 = the source plane) and `ok` (n_rays x nElt+1, True where the ray reached that element).

ray_status

- **mmacos:** `r = macos.get_ray_status(n_rays)`
- **pymacos:** `pymacos.ray_status(n_rays)`

Per-category ray-status codes after the most recent trace. r = macos.get_ray_status(N) returns a struct with fields: .status N×1 int32 — per-ray RayStat_* code: 0 = OK, 1 = Obscured, 2 = Miss, 3 = Bracket, 4 = MaxIter, 5 = Undef .fail_elt N×1 int32 — element index where failure was first

read_grid_file

- **mmacos:** `M = macos.read_grid_file(path)`
- **pymacos:** *not available*

Read a MACOS GridFile (ASCII) into GridMat orientation. M = macos.read_grid_file(PATH) reads an N x N grid-data file the SAME way the engine's GridInit (surfsub.F) does – text line j fills GridMat COLUMN j – and returns M in the engine's GridMat(i,j) convention (first index = +x, second index = +y). It is the exact inverse of macos.write_grid_file, and pairs with macos.elt_grid_add:

ref_rays

- **mmacos:** `r = macos.ref_rays(kind)`
- **pymacos:** `pymacos.ref_rays(kind)`
- **engine API:** `macos_api_mod::ref_rays`

Find the reference rays of the source grid – the CLI's RRAY / PRAY. r = macos.ref_rays('rray') the chief ray + the four rays nearest the aperture edge at +x, +y, -x, -y (5 slots) r = macos.ref_rays('pray') the chief

ray + the eight rays nearest 0.9 of the aperture at 45-degree steps from +x (9 slots) Each candidate must pass the system. Fields (slot 1 = the chief ray):

rm_elt_csys

- **mmacos:** `macos.rm_elt_csys(srfs)`
- **pymacos:** *not available*

Remove the element output local coordinate frame (TElt). macos.rm_elt_csys(SRFS) clears any local output coordinate system on the given elements (resets TElt to identity, nECoord=-6), so their output reverts to the global/beam frame. See also: macos.set_elt_csys, macos.get_elt_csys.

rm_srf_from_rx

- **pymacos:** `pymacos.rm_srf_from_rx(rx_in, srf_idx, rx_out)`
- **mmacos:** *not available*

Eliminate surface(s) from Rx.

rx_modified

- **pymacos:** `pymacos.rx_modified()`
- **mmacos:** *not available*

Reset ray-tracing state

scale_elt_grid

- **mmacos:** `macos.scale_elt_grid(srf, factor)`
- **pymacos:** *not available*

Scale the grid-data figure at element SRF in place. macos.scale_elt_grid(SRF, FACTOR) multiplies every grid node value by the scalar FACTOR and invalidates the cached trace. Errors if SRF is not a grid surface. See also: macos.get_elt_grid, macos.set_elt_grid.

segray

- **mmacos:** `r = macos.segray(iSeg)`
- **pymacos:** `pymacos.segray(i_seg)`

Trace the ray through Segment element iSeg's centre – the CLI's SEGRAYTRACE. r = macos.segray(iSeg) iSeg = the ELEMENT number of a Segment element The ray passes through the segment's reference point (RptElt, the design centre; a perturbation does not move it): parallel to the chief ray for a collimated source, aimed from the chief ray's start for a point source. It is forced onto segment iSeg and traced to the last element (or the deck's

setActivePointSrc

- **pymacos:** `pymacos.getActivePointSrc()`
- **mmacos:** *not available*

TODO: expand — dialog details, behavior notes, related commands.

setEltSrfFFZern

- **pymacos:** `pymacos.getEltSrfFFZern(iElt, zernMode)`
- **mmacos:** *not available*

Get current FFZernCoef values at the requested modes on a FreeForm element (SrfType=14).

setEltSrfMonZern

- **pymacos:** `pymacos.getEltSrfMonZern(iElt, zernMode)`
- **mmacos:** *not available*

Get current MonZernCoef values at the requested mode indices on a FreeForm element (SrfType=14).

setEltSrfZern

- **pymacos:** `pymacos.getEltSrfZern(iElt)`
- **mmacos:** *not available*

Retrieve Zernike Srf. settings of Zernike Srf. element(s)

setEltSrfZernCoef

- **pymacos:** `pymacos.setEltSrfZernCoef(iElt, zernMode, zernCoef, reset)`
- **mmacos:** *not available*

Set ZernCoef values at the given modes on a Zernike (SrfType=8) or ZrnGrData (SrfType=13) element.

srt

- **mmacos:** `r = macos.srt(x, y, endElt)`
- **pymacos:** `pymacos.srt(x, y, end_elt)`

Trace ONE ray from a normalized source-aperture point (the CLI's SRT). r = macos.srt(x, y) trace to the last element r = macos.srt(x, y, endElt) trace through element endElt x, y: the point in the normalized source aperture, |x|, |y| <= 0.5; (0, 0) is the chief ray. Fields (column k = element k, BaseUnits / global frame): r.pos 3 x nElt where the ray meets element k

unload

- **mmacos:** `macos.unload()`
- **pymacos:** *not available*

Release the engine's memory back to its minimum footprint. macos.unload() re-initialises the engine at the smallest supported model size (128), which deallocates and rebuilds every model-sized array. Use it when a big run is finished and you want the memory back WITHOUT quitting MATLAB. The engine stays usable: call macos.init(N) (or construct a new macos.Session) and carry on.

v2m

- **mmacos:** `mat = macos.v2m(vec, indx)`
- **pymacos:** *not available*

Reconstruct a 2D matrix from m2v's compressed vec + indx. mat = macos.v2m(vec, indx) un-vectorises the result of macos.m2v. Sparse scatter into a full matrix of size indx.size.

vector_diffraction

- **mmacos:** `macos.vector_diffraction(on)`
- **pymacos:** `pymacos.vector_diffraction(on)`

Toggle vector (3-component) diffraction. macos.vector_diffraction(true) -> VECTOR: propagate Ex/Ey/Ez as three independent fields. Since PLAN_POLARIZATION Phase 3a Tranche 1 this covers the WHOLE chain – every near-field, plane-to-plane, spherical, Fresnel and DFT leg, plus FFObscure and the ray-side aperture masking – not just the far-field FFT

Requires polarization ON first; errors otherwise (the CLI VECtor silently reverts instead). Since Phase 3a Tranche 1 every propagation leg is vectorized – far-field, near-field sphere/plane, plane-to-plane, spatial-filter, Fresnel and the DFT legs – along with the diffraction-grid obscuration apply and the ray-side aperture/apodization masking, so a multi-leg chain preserves the vector field. Intensity sums the three components. Two limits to know. (1) Vector mode repurposes the three wavefront planes as Ex/Ey/Ez, so only ONE wavefront can be in flight – not usable with multi-WF / COMPOSE. (2) Between two physical legs the grid field is advanced by a scalar phase, exact only when the intervening elements are non-polarizing (Obscuring / Reference / FocalPlane – the coronagraph pupil->FPM->Lyot->focal case). A COATED or reflecting surface BETWEEN legs needs the per-ray running-Jones work of Tranche 2, which is not implemented. *Related:* polarization; CLI VECtor/SCAlar.

view_rx

- **mmacos:** `fig = macos.view_rx(opts)`
- **pymacos:** *not available*

3-D visualization of the LOADED prescription: beam, optics, and MET paths if present. Works for ANY Rx – no design-layer structs required; everything is read back from the engine (the modern equivalent of the old MACOS 3-D model visualizer): beam a sparse-but-FILLED ray bundle from the engine's per-trace

view_std

- **mmacos:** `fig = macos.view_std(opts)`
- **pymacos:** *not available*

Standard 3-view layout figure for the LOADED prescription. fig = macos.view_std() draws four beam-aligned panels via macos.view_rx – FRONT (POV behind the SOURCE, angled, looking at the first optic's reflective face), BACK (POV behind the first optic), ISO (angled), SIDE (elevation) – with the optical-layout convention: the LIGHT SOURCE IS AT THE LEFT and light travels to

waveplate

- **mmacos:** `out = macos.waveplate(srf, opts)`
- **pymacos:** `pymacos.waveplate(srf, axis, retardance)`

Set or query a linear retarder (waveplate) element. macos.waveplate(SRF, 'axis', A, 'retardance', R) configures the WavePlate element at SRF. A is the FAST axis as a 3-vector in GLOBAL coordinates (projected into each ray's transverse plane by the engine). R is the retardance in WAVES at the CURRENT wavelength: 0.25 for a quarter-wave plate, 0.5 for a half-wave plate.

Element type. Requires `Element= WavePlate` (EltID 18, added in PLAN_POLARIZATION Phase 3, extending `mEltTypes` to 18). Any other type is refused.

Retardance sign is derived from the engine, not chosen. MACOS propagates a field as `exp(-i*2*pi*L*N/lambda)` (`elemsub.F:395`), i.e. `exp(+i*omega*t)` time dependence, so the slow axis accumulates the more negative phase. With the declared axis as FAST, the element Jones in its own eigenbasis is `diag(1, exp(-i*delta))`, `delta = 2*pi*R`. Pinned by `tPolElement/test_retardance_sign_matches_engine_convention` and, more sharply, by the SIGNED circular Stokes parameter in `test_qwp_linear_to_circular` – a suite that only checked `|S3|` would accept either convention.

Chromatic by construction. The stored quantity is the physical retardance `(n_slow - n_fast)*d = R*lambda`, so a plate set to 0.25 waves at 1 um is 0.125 waves at 2 um. This is the same treatment `Coating=` layer thickness already receives, and it means a query after a wavelength change returns a different R than was set – physics, not a round-trip failure.

Thin idealization. No o/e walk-off, no ray splitting, no Fresnel loss at the faces, no substrate thickness. The retardance is also independent of incidence angle, where a real crystal plate's is not (the field-of-view effect behind compound and Pancharatnam designs); bounding that needs a birefringent-plate model with o/e indices and thickness. The output is purely transverse: any longitudinal component present at the surface is discarded, which is what a 2x2 Jones element means (exactly zero for a collimated normal-incidence beam, O(NA) otherwise). The element is also the primitive for bounding stress birefringence in a transmissive optic.

Off normal, the declared axis is the material axis. The crystal fast axis is the direction fixed in the substance of the plate, so it is what the engine projects into the ray's transverse plane – the material-axis rule, which for a waveplate is just the declared axis. `macos.polarizer` projects a *different* vector from the same keyword (its material axis is the absorbing direction), so the two elements are not interchangeable off normal.

Unitary. Gated for linear and circular input states, both against the field power and against `J'*J == I` from `elt_jones`, with the (non-unitary) polarizer as the non-vacuity companion.

xp

- **mmacos:** `s = macos.get_xp()`
- **mmacos:** `macos.set_xp(vpt, psi, rad)`
- **pymacos:** `pymacos.xp(vpt, psi, ref_rad)`

Exit-pupil geometry (at element nElt-1). s = macos.get_xp() returns a struct: .vpt 3×1 pupil vertex (BaseUnits, global frame) .psi 3×1 pupil-surface normal (global) .rad reference-sphere radius (BaseUnits)

Session and user options: one default each

Every session or user option the bindings expose has ONE default, held by the engine (set at allocation when it is engine state); the CLI, mmacos and pymacos inherit it. **A veneer never carries a default an api getter can supply** (Dave 2026-10-10). *Status*: **inherits** – the veneer reads the engine; **agrees** – the veneer restates the engine's value and the api has no getter to inherit from; **fixed** – a restatement removed 2026-10-16 (A.6); **Dave** – a judgment call, routed with the measurement. Gates: mmacos `tDefaults`, pymacos `tests/test_defaults.py` (same decks, same numbers).

Option	Engine default	api get / set	CLI	mmacos	pymacos	Override	Status
Session modes: POLARIZATION, VECTOR/SCALAR, CENTROID, LNEG, CIR	`.FALSE.` (`macos_mod.F`): ifPol/ifVecDif3/ifCentroid/ifPGColor .FALSE., ifLNegOK .TRUE., at allocation (macos_mod_init); survive LOAD	`modes_get` / `pol_set`, `vecdif_set`	POLARIZATION, VECTOR/SCALAR, CENTROID/CHIEFRAY, LNEG/NOLNEG, CIR/GIR; LOAD names the non-default ones	`macos.modes()`; `load_rx` warns `mmacos:modesInForce`	`modes()`; `load` warns (Runtime Warning)	`polarization(...)`; CENTROID/LNEG/CIR have no api setter (CLI only)	**inherits**

Option	Engine default	api get / set	CLI	mmacos	pymacos	Override	Status
OPD reference (chief ray / aperture mean)	`.TRUE.` (`traceutil_mod.F`): session LOpdRef Chief (chief), at allocation; per deck `UseChfRay4 OPD= Y|N`; per loaded deck `opd_ref_set`	`opd_ref_get` / `opd_ref_set`	OPDREF CHIEF| MEAN (session); `UseChfRay4 OPD=` (deck)	`''` (`dw_dx.m`): `opd()` / `opd_ref()` inherit; the dw_* harvests' `opd_ref`: the deck's own as loaded (was `'chief'`, re-applied after the reload: it overrode `UseChfRay4 OPD= N` and left the session on chief)	`opd()` / `opd_ref()` inherit (no harvest veneer)	`opd_ref('mean'|'chief')` for the loaded deck; a harvest's `opd_ref`	**fixed**
FEX / SXP axis (chief ray / beam centroid)	`.FALSE.` (`macos_mod.F`): session mode ifCentroid (.FALSE. = chief), at allocation	`modes_get` / `xp_fnd`, `sxp_fnd`	CENTROID / CHIEFRAY, then FEX / SXP	`[]` (`fex.m`): `fex(mode)`, `sxp(mode)`: empty = the session axis read through `modes_get` (was a fixed 1; `sxp` also refused 0)	`None` (`macos.py`): `fex(mode)`, `sxp(mode)`: None = the session axis (was a fixed 1)	`fex(0)` / `fex(1)` per call	**fixed**
SPOT centre (element vertex / chief ray)	`1` (`macos_init.inc`): spcOption (1 = element vertex), at allocation	`spc_option_get` / `spot_cmd`	SPCENTER ELT| CHFRAY	`''` (`spot.m`): `spot(srf, at=)`: empty = the session centre (`spc_option_get`)	`None` (`macos.py`): `spot(srf, vpt_center=)`: None = the session centre	per call (`at='chief'` / `vpt_center=False`); `spot_cmd` restores the session option after the call	**inherits**
Obscurati	`1` (`macos_ini`	`obs_opt_get` / `obs_set`	OBS	`obs()` queries	`None` (`macos.py`):	`obs('all'|'negative`	**inherits**

Option	Engine default	api get / set	CLI	mmacos	pymacos	Override	Status
on option (OBS ALL / POSITIVE / NEGATIVE)	t.inc): iObsOpt (1 = POSITIVE: unobscured rays), at allocation			(obs_opt_get), obs(opt) sets	obs_set() queries (was: SET positive), obs_set(opt) sets	') / obs_set(...)	
Far-field evanescent cut	.FALSE. (dftsub.F): dft_modifFFEvanCut (off), at allocation	ffcut_get / ffcut_set	(none: api only)	ffcut() queries, ffcut(on) sets	ffcut() queries, ffcut(on) sets	ffcut(true)	**inherits**
OPD / mask array orientation	no engine state: the raw array is [i = global X, j = global Y]	opd_val, opd_mask_get / –	(OPD writes the raw array)	raw (opd.m): opd(orient=), opd_mask(orient=)	raw (macos.py): opd(orient=), opd_mask(orient=)	orient='xy' (the transpose)	**agrees**
Valid-pixel mask (opdValid)	opdValid from the last OPD trace (zero is NOT “no ray”)	opd_mask_get / –	(OPD)	opd_mask()	opd_mask()	–	**inherits**
remove_ptt in the sensitivity harvests	no engine state	(no getter) / –	–	false (dw_dsurf.m): dw_dsurf, dw_dsurf_multi	(no harvest veneer; tests/sensitivities drivers)	remove_ptt=true	**agrees**
Model size	256 (macos_mod.F): the CLI’s start-up prompt: Return = this default; SMACOS with no size: 256 too (param_m	(no getter) / init	prompted at start-up; Return = 256	256 (init.m): init(model_size) default	256 (macos.py): init(model_size=) default	init(N)	**agrees**

Option	Engine default	api get / set	CLI	mmacos	pymacos	Override	Status
	od)						
Polarization source state on POLARIZATION ON	Ex0/Ey0 = 0 (unset) until set; `pol_get` reads it	`pol_get` / `pol_set`	POLARIZATION prompts for Ex, Ey	`[1 0]` (`polarization.m`): `polarization('on', Ex=, Ey=)` Ex default	`(1.0, 0.0)` (`macos.py`): `polarization('on', Ex=, Ey=)` Ex default	`Ex=`, `Ey=`	**agrees**

Part III — Higher-level MATLAB layers (current state)

These packages sit on top of the Part II bindings. They are under active development; interfaces here reflect the current tree and may evolve faster than Parts I–II.

macos.design — +design

Parametric telescope design layer: build a System from first-order parameters, add mirrors, solve conics/Zernikes against multi-field wavefront targets, and emit a MACOS .in prescription.

design.Bench

```
classdef Bench < handle
```

MACOS.DESIGN.BENCH Sequential optical-bench builder (add-optic utilities). B = macos.design.Bench(NAME, …) starts a bench layout at a point source and lets you append optics ONE AT A TIME, each placed a given DISTANCE along the current chief ray. The builder tracks the chief ray analytically as it goes – reflecting it at mirrors and Snell-

design.System

```
classdef System < handle
```

MACOS.DESIGN.SYSTEM Imported optical system — the design-layer analysis front-end. The front-end-agnostic analysis core (PLAN_DESIGN_LAYER §1.0, Sprint 2A-i). `from_rx` loads a prescription via SMACOS and reads its element parameters back through the existing `+macos` getter surface — engine readback, NOT a MATLAB text parser — into a plain

design.Telescope

```
classdef Telescope < handle
```

MACOS.DESIGN.TELESCOPE De-novo two-mirror telescope builder (Sprint 2A-ii). The fixed-topology builder front-end of the design layer (PLAN_DESIGN_LAYER §1.0/§2/§5). The user states design intent (family + first-order parameters); the builder derives the full first-order layout and conic constants in closed form (Schroeder

design.add_met

```
macos.design.out = add_met(in_path, seg, opts)
```

ADD_MET Emit Stewart-truss laser metrology into a segmented Rx. out = macos.design.add_met(in_path, seg, ‘hub’, k, ‘r_fid’, r) appends engine met blocks (nMetPos/tMetElt/metBeamFlg) to the prescription: 6 launchers on EACH segment (at the segment edge by

design.append_rx

```
macos.design.info = append_rx(base_in, add_in, out_in, opts)
```

APPEND_RX Splice one prescription's elements onto the end of another's. info = MACOS.DESIGN.APPEND_RX(BASE_IN, ADD_IN, OUT_IN) writes OUT_IN: the BASE deck's header and source, then the BASE deck's elements, then the ADD deck's elements renumbered to follow them, then the standard

design.assert_draw_frame_global

```
macos.design.assert_draw_frame_global(plane, caller)
```

ASSERT_DRAW_FRAME_GLOBAL Refuse to read draw_rays' U/V as global x,y,z unless the deck's source frame actually is the global frame. macos.draw_rays does NOT return global coordinates. It projects each crossing onto the SOURCE / RAY-GRID basis – DRAW sets

design.configs_from_table

```
macos.design.cfgs = configs_from_table(T, opts)
```

MACOS.DESIGN.CONFIGS_FROM_TABLE Build a configuration array from a schedule. cfgs = macos.design.configs_from_table(T) turns a zoom / compensation SCHEDULE – the shape one naturally arrives in, a spreadsheet with one row per configuration – into the 'configs' struct array the macos.dw_d*_multi supervisors and run_sensitivities take.

design.dldx_analytic

```
macos.design.dldx = dldx_analytic(bodies, src_pts, tgt_pts, src_body, tgt_body, unit_to_m)
```

DLDX_ANALYTIC Closed-form MET Jacobian for straight-line gauges. dldx = macos.design.dldx_analytic(BODIES, SRC, TGT, SB, TB) returns the nbeam x 6*numel(BODIES) sensitivity of each gauge length l = |s - t| to the rigid DOFs of the bodies carrying its

design.dmet_dfig

```
macos.design.out = dmet_dfig(seg, es, gm, opts)
```

DMET_DFIG Figure-state measurement Jacobians dmdz / dmdgrid. out = macos.design.dmet_dfig(SEG, ES, GM, 'z_names', CN, …) builds the measurement-model sensitivity of the laser gauges l and the edge sensors e to the FIGURE states of the segmented forward model

design.dmet_dx

```
macos.design.out = dmet_dx(elts, opts)
```

DMET_DX FD metrology Jacobian dl/dx over per-element rigid DOFs. out = macos.design.dmet_dx(ELTS) pokes each element in ELTS through its six rigid-body DOFs and finite-differences the engine metrology readings (macos.met): the dldx block of the segmented forward model

design.edge_sensors

```
macos.design.out = edge_sensors(hx_path)
```

EDGE_SENSORS Ingest a SegMirMaker Hx.m edge-sensor model (dedx). out = macos.design.edge_sensors(hx_path) loads the MATLAB-loadable edge-sensor measurement matrix SegMirMaker writes alongside the .presc (see macos.design.segment_rx: out.hx) and returns it as the

design.field_cross

```
macos.design.F = field_cross(fov, n, opts)
```

FIELD_CROSS Field-angle CROSS (x-arm + y-arm) – the cheap field set. F = macos.design.field_cross(fov, n) returns the (thx,thy) field points on a CROSS through the origin: n samples along +-x and n along +-y, spanning [-fov,+fov] (radians). The cross captures the two PRINCIPAL field directions at a fraction of a full grid's points – the right set

design.field_grid

```
macos.design.F = field_grid(fov, n, opts)
```

FIELD_GRID n x n field-angle GRID – the "area" field set. F = macos.design.field_grid(fov, n) returns an (n*n) x 2 list of (thx,thy) field points spanning [-fov,+fov] in BOTH axes, edge to edge (corners + edge midpoints + center) – the field set for AREA scoring (optimize / evaluate over a 2-D field), as opposed to a single +y fan

design.field_ring

```
macos.design.F = field_ring(radius, opts)
```

FIELD_RING Ring field set – the CIRCULAR-field spec. F = macos.design.field_ring(radius) returns (thx,thy) field points on a ring of the given field RADIUS: n equally-spaced azimuths, plus an optional inner ring. A circular field of diameter 2*radius is the honest spec for a round detector / instrument patch: a square

design.grid_augment_rx

```
macos.design.out = grid_augment_rx(rx_in, rx_out, opts)
```

GRID_AUGMENT_RX Add per-segment grid channels in the CLOCKED Mon frames. out = macos.design.grid_augment_rx(RX_IN, RX_OUT) rewrites every Element= Segment block of RX_IN with a flat grid-data channel whose coordinate frame is that segment's OWN clocked Mon frame

design.hex_tile

```
macos.design.T = hex_tile(seg, off)
```

HEX_TILE Boundary-true hex tile geometry for a segmented primary. T = macos.design.hex_tile(SEG) reconstructs each segment's ACTUAL hex boundary from the tiling itself (SEG = macos.design.segment_rx output): - apothem a = width/2 (manual: width = flat-to-flat, gap = the

design.ideal_lens

```
macos.design.L = ideal_lens(f, D, opts)
```

MACOS.DESIGN.IDEAL_LENS Ideal focusing-lens primitive (2-surface singlet). L = macos.design.ideal_lens(F, D) returns a spec for a singlet of effective focal length F and clear diameter D that images the INFINITE conjugate. MACOS has no paraxial/ideal-lens engine primitive, so an ideal lens is realized as a refractive singlet

design.ideal_lens_emit

```
macos.design.txt = ideal_lens_emit(L, iElt0, vpt_front, psi, opts)
```

MACOS.DESIGN.IDEAL_LENS_EMIT Render an ideal_lens spec to Rx element text. txt = macos.design.ideal_lens_emit(L, IELT0, VPT_FRONT, PSI) returns the two Refractor element blocks

(front + powered, in light order) for the lens spec L (from macos.design.ideal_lens), numbered starting at IELT0, with the FRONT vertex at VPT_FRONT (1x3) and optical axis

design.met_bodies

```
macos.design.bodies = met_bodies(elts)
```

MET_BODIES Rigid-body frames for dldx_analytic, from the ENGINE state. bodies = macos.design.met_bodies(ELTS) queries the loaded Rx for each element's perturbation frame – RptElt (the rotation pivot CPERTURB_PROG rotates points about) and TElt (the 6x6 local->global perturbation map macos.perturb applies in 'local' frame) – and

design.met_layout_opt

```
macos.design.out = met_layout_opt(seg, D, E, X, opts)
```

MET_LAYOUT_OPT Tier-3 MET-layout optimization with SHAPE-CLASS patterns. out = macos.design.met_layout_opt(SEG, D, E, X, 'hub', h, …) minimizes the post-control wavefront residual trace(dwdx * P_dx * dwdx'),

design.met_view

```
macos.design.fig = met_view(seg, am, opts)
```

MET_VIEW 3-D visualization of a segmented-primary laser-MET setup. fig = macos.design.met_view(SEG, AM) draws the MET configuration built by macos.design.add_met on the segmentation from macos.design.segment_rx: LEFT - 3-D scene: segment hex tiles (face triads), launchers (green),

design.pbs_macneille

```
macos.design.S = pbs_macneille(opts)
```

MACOS.DESIGN.PBS_MACNEILLE MacNeille polarizing-beamsplitter cube coating. S = MACOS.DESIGN.PBS_MACNEILLE(…) returns the layer stack and the substrate index of a MacNeille-type polarizing beamsplitter: a quarter-wave H/L stack cemented between two glass prisms, designed so

design.pie_rings

```
macos.design.R = pie_rings(C2, w)
```

PIE_RINGS Robust ring classification for the pie tiling. R = macos.design.pie_rings(C2, W) clusters segment centers (2 x n, tiling-plane coordinates about the tiling center) into the center cell + concentric wedge rings. Single source of truth shared by macos.design.seg_apertures, macos.design.seg_boundary, and

design.pie_wedge_geom

```
macos.design.W = pie_wedge_geom(a0, dth, rc, w, g, off, has_outer, has_center_hex)
```

PIE_WEDGE_GEOM Physical pie-wedge edge geometry with UNIFORM gaps. W = macos.design.pie_wedge_geom(A0, DTH, RC, W_, G, OFF, HAS_OUTER, HAS_CENTER_HEX) is the single source of truth for the pie-tiling wedge boundary shared by macos.design.seg_boundary (tiling overlay), macos.design.seg_apertures (emitted PolyApVec), and

design.promote_segments_freeform

```
macos.design.out = promote_segments_freeform(rx_in, rx_out, opts)
```

PROMOTE_SEGMENTS_FREEFORM Turn conic optics into inert FreeForm carriers. out = macos.design.promote_segments_freeform(RX_IN, RX_OUT) rewrites every Element= Segment block of RX_IN into a Surface= FreeForm (SrfType 14) block that carries a MonZernike figure CHANNEL – so the

design.prop_layout

```
macos.design.info = prop_layout(src, kinds, opts)
```

PROP_LAYOUT Turn a geometric deck into a DIFFRACTION deck. info = MACOS.DESIGN.PROP_LAYOUT(SRC, KINDS) reads a bare geometric prescription, traces it for the chief-ray geometry, and writes a deck that carries the complex field: focal-mask QUARTETS at the

design.psri_bench

```
macos.design.G = psri_bench(opts)
```

MACOS.DESIGN.PSRI_BENCH A buildable phase-shifting self-referenced interferometer (P/SRI, Dube et al. 2024) on the DM-gauge bench: the TG96 front end (source, collimator, 7 deg splitter, the 96 mm DM on its leg, back through the splitter) followed by a Mach-Zehnder whose REFERENCE arm carries the point-diffraction pinhole (the waveguide

design.seg_apertures

```
macos.design.ap = seg_apertures(seg, opts)
```

SEG_APERTURES Per-segment polygonal aperture blocks from the tiling truth. ap = macos.design.seg_apertures(SEG, Name=Value) generates, for every segment of SEG (= macos.design.segment_rx output), the prescription lines that declare its PHYSICAL boundary as a polygonal aperture: ApType=Polygonal + an explicit xObs (the engine ChkDf2 default

design.seg_boundary

```
macos.design.B = seg_boundary(seg, off, opts)
```

SEG_BOUNDARY True per-segment boundary polylines for a segmented primary. B = macos.design.seg_boundary(SEG, OFF) reconstructs each segment's ACTUAL boundary in the tiling plane (SEG = macos.design.segment_rx output), grown outward by OFF (same units; e.g. a launcher edge clearance). Three sources:

design.seg_footprint_view

```
macos.design.[fig, xun, yun, toimg] = seg_footprint_view(labels, B, seg, opts)
```

SEG_FOOTPRINT_VIEW Footprint map + tiling/aperture overlay figure. [FIG, XUN, YUN, TOIMG] = macos.design.seg_footprint_view(LABELS, B, SEG) renders the per-segment footprint map (LABELS from macos.design.seg_footprints) in PUPIL coordinates, with the tiling boundary (B from macos.design.seg_boundary) and optionally the

design.seg_footprints

```
macos.design.labels = seg_footprints(seg, w0, opts)
```

SEG_FOOTPRINTS Per-segment TRUE ray footprints from the trace. LABELS = macos.design.seg_footprints(SEG, W0) measures which OPD pixels each segment of SEG (= macos.design.segment_rx output) actually owns, by ENGINE TRUTH rather than geometry: poke one segment at a time (local +z translation), re-trace, and mark the

design.seg_from_rx

```
macos.design.seg = seg_from_rx(in_path, opts)
```

SEG_FROM_RX Rehydrate a segment_rx-style struct from a segmented .in. seg = macos.design.seg_from_rx(IN_PATH) rebuilds the struct that macos.design.segment_rx returns – frames, seg_elts, tiling metadata – from the prescription file ALONE, so downstream stage

design.segment_rx

```
macos.design.out = segment_rx(parent_in, opts)
```

SEGMENT_RX Segment one element of a MACOS prescription (SegMirMaker splice). out = macos.design.segment_rx(parent_in, Name=Value) replaces element `elt` of the parent prescription with SegMirMaker-generated segment blocks and returns the merged, load-ready `.in`. Sprint 2D S1:

design.segmirmaker_run

```
macos.design.out = segmirmaker_run(parent_in, opts)
```

SEGMIRMAKER_RUN Drive the SegMirMaker generator non-interactively. out = macos.design.segmirmaker_run(parent_in, Name=Value) runs the standalone SegMirMaker binary (segmirmaker/) on a parent MACOS prescription by scripting its interactive dialog over stdin, in a

design.seidel_seed

```
macos.design.[K, t_focus, EFL, S] = seidel_seed(R, t_between, D, convex, opts)
```

SEIDEL_SEED Third-order anastigmat conic seed for a coaxial mirror train. [K, t_focus, EFL] = macos.design.seidel_seed(R, t_between, D) solves the conic constants K (1xN) that null the Seidel spherical, coma and astigmatism sums (S_I, S_II, S_III) of an N-mirror coaxial reflective train, and returns the paraxial focus distance t_focus (last mirror ->

design.seidel_seed_nflip

```
macos.design.[K, t_focus, EFL] = seidel_seed_nflip(R, t_between, D, convex)
```

SEIDEL_SEED_NFLIP The 2026-06 n-flip Seidel seed, kept VERBATIM for the record (do not call). [K, t_focus, EFL] = macos.design.seidel_seed(R, t_between, D) solves the conic constants K (1xN) that null the 3rd-order Seidel spherical, coma and astigmatism sums (S_I, S_II, S_III) for an N-mirror coaxial reflective layout, using the n-flip unfolded paraxial model (ported

design.thinfilm_rt

```
macos.design.out = thinfilm_rt(layers, n_inc, n_sub, aoi_deg, lambda)
```

MACOS.DESIGN.THINFILM_RT Amplitude/power r and t of a multilayer stack. OUT = MACOS.DESIGN.THINFILM_RT(LAYERS, N_INC, N_SUB, AOI_DEG, LAMBDA) Characteristic-matrix (Abeles) solution for a stratified stack, written

design.tma_layout

```
macos.design.[R, t, info] = tma_layout(D, primary_fnum, system_fnum, opts)
```

TMA_LAYOUT Generic on-axis Korsch TMA first-order layout (j18 / JWST form), with the intermediate focus placed where YOU want it for packaging. [R, t, info] = macos.design.tma_layout(D, primary_fnum, system_fnum, …)

design.twyman_green

```
macos.design.G = twyman_green(opts)
```

MACOS.DESIGN.TWYMAN_GREEN Build a compensated Twyman-Green IFO rig. G = macos.design.twyman_green(…) builds BOTH arms of the generic Twyman-Green interferometer (the templates/40_benches/bench_ifo layout) with the Bench add-optic utilities and returns them ready to emit:

design.zern_seg_eval

```
macos.design.z = zern_seg_eval(frame, mode, P)
```

ZERN_SEG_EVAL Engine-exact segment MonZernike mode value at world points. z = macos.design.zern_seg_eval(FRAME, MODE, P) evaluates the surface sag contribution of a unit MonZernCoef(MODE) poke at the 3 x n world points P on the segment whose face frame is FRAME (a segment_rx /

macos.channels — +channels

Sensitivity-channel classes used by the dw_* Jacobian drivers: each channel enumerates one family of perturbations (rigid-body, Zernike, grid, source, …) and knows how to apply/revert them.

channels.FFZernChannel

```
macos.channels.ch = FFZernChannel(session, iElt, mode)
```

MACOS.CHANNELS.FFZERNCHANNEL Factory for an FFZern coefficient channel. ch = macos.channels.FFZernChannel(SESSION, IELT, MODE) constructs a ZernikeCoefChannel rooted in the FFZernCoef array of a FreeForm element.

channels.FocalPlaneChannel

```
classdef FocalPlaneChannel < macos.channels.RigidBodyChannel
```

MACOS.CHANNELS.FOCALPLANECHANNEL Rigid-body perturbation on the FP element. macos's standard trace does NOT propagate FP perturbations back to the EP-OPD measurement – without compensation all six FP DOFs come out as zero sensitivities (physically wrong: the EP is a

channels.GridChannel

```
classdef GridChannel < handle
```

MACOS.CHANNELS.GRIDCHANNEL One grid-data influence-function sensitivity channel. A single (element, influence-map) pair on a grid-bearing surface of ANY GridData-enabled SrfType (GridData / AsGrData / MonGrData / ZrnGridData / FreeForm). The DOF coefficient scales an N×N influence map that is ADDED

channels.GroupedRigidBodyChannel

```
classdef GroupedRigidBodyChannel < handle
```

MACOS.CHANNELS.GROUPEDRIGIDBODYCHANNEL Rigid-body perturbation of a member group. Dispatches macos's GPERTURB (CPERTURB_GRP_DVR) to perturb the declared members as a single rigid unit. Like the per-element RigidBodyChannel this is INCREMENTAL: each apply(value) sends

channels.MonZernChannel

```
macos.channels.ch = MonZernChannel(session, iElt, mode)
```

MACOS.CHANNELS.MONZERNCHANNEL Factory for a MonZern coefficient channel. ch = macos.channels.MonZernChannel(SESSION, IELT, MODE) constructs a ZernikeCoefChannel rooted in the MonZernCoef array of a FreeForm element. Thin convenience wrapper over the underlying classdef constructor.

channels.RigidBodyChannel

```
classdef RigidBodyChannel < handle
```

MACOS.CHANNELS.RIGIDBODYCHANNEL One-DOF rigid-body perturbation channel. A single (element, DOF) pair with DOF in {Rx, Ry, Rz, Tx, Ty, Tz}. Rotations are in radians; translations are passed in SI metres (macos.perturb converts to BaseUnits via CBM internally).

channels.SourceChannel

```
classdef SourceChannel < handle
```

MACOS.CHANNELS.SOURCECHANNEL One-DOF rigid-body perturbation on the source. Wraps macos.perturb_src (the iElt=0 branch of CPERTURB). The source rotates ChfRayDir and translates ChfRayPos. A source perturbation moves the chief ray off the stop centre, so this

channels.SurfChannel

```
classdef SurfChannel < handle
```

MACOS.CHANNELS.SURFCHANNEL One-DOF powered-surface perturbation (Kr or Kc). A single (element, param) pair with param in {‘Kr’,‘Kc’} – the base radius of curvature (KrElt, BaseUnits) or conic constant (KcElt, dimensionless) of a POWERED optic (Reflector / Refractor with

channels.ZernChannel

```
macos.channels.ch = ZernChannel(session, iElt, mode)
```

MACOS.CHANNELS.ZERNCHANNEL Factory for a Zern coefficient channel. ch = macos.channels.ZernChannel(SESSION, IELT, MODE) constructs a ZernikeCoefChannel rooted in the ZernCoef array of a Zern-typed element (Surface=Zernike or Surface=ZrnGridData).

channels.ZernikeCoefChannel

```
classdef ZernikeCoefChannel < handle
```

MACOS.CHANNELS.ZERNIKECOEFCHANNEL One-mode Zernike sensitivity channel. A single (element, mode) pair on either a MonZern, FFZern, or Zern coefficient array. Snapshots its nominal coefficient on the first apply() so that apply(+d) followed by apply(-d) and

channels.freeform_ffzern_channels

```
macos.channels.chans = freeform_ffzern_channels(session, rx_path, opts)
```

MACOS.CHANNELS.FREEFORM_FFZERN_CHANNELS Build FFZern channels per FF element. chans = macos.channels.freeform_ffzern_channels(SESSION, RX_PATH) returns a cell array of ZernikeCoefChannel handles, one per (FreeForm element, FFZern mode) pair declared in the Rx.

channels.freeform_monzern_channels

```
macos.channels.chans = freeform_monzern_channels(session, rx_path, opts)
```

MACOS.CHANNELS.FREEFORM_MONZERN_CHANNELS Build MonZern channels per FF element. chans = macos.channels.freeform_monzern_channels(SESSION, RX_PATH) returns a cell array of ZernikeCoefChannel handles, one per (FreeForm element, MonZern mode) pair declared in the Rx.

channels.grid_channels

```
macos.channels.chans = grid_channels(session, influence, opts)
```

MACOS.CHANNELS.GRID_CHANNELS Build grid-data channels per grid element × map. chans = macos.channels.grid_channels(SESSION, INFLUENCE) returns a cell array of GridChannel handles, one per (grid-bearing element, influence map). Channels are built in canonical order: element-major, map-minor. Eligibility (any GridData-enabled SrfType) comes from macos.find_grid_elts;

channels.grouped_rigid_body_channels

```
macos.channels.chans = grouped_rigid_body_channels(session, groups, opts)
```

MACOS.CHANNELS.GROUPED_RIGID_BODY_CHANNELS Build group channels. chans = macos.channels.grouped_rigid_body_channels(SESSION, GROUPS) returns a cell array of GroupedRigidBodyChannel handles, one per (group, DOF) pair.

channels.parse_rx_groups

```
macos.channels.groups = parse_rx_groups(rx_path)
```

MACOS.CHANNELS.PARSE_RX_GROUPS Parse EltGrp= declarations from an Rx. groups = macos.channels.parse_rx_groups(RX_PATH) scans the .in file for EltGrp= lines and returns a containers.Map (char -> column vector of double) mapping group names (‘min-max’ of member ids) to their member lists.

channels.rigid_body_channels

```
macos.channels.chans = rigid_body_channels(session, rx_path, opts)
```

MACOS.CHANNELS.RIGID_BODY_CHANNELS Build per-element rigid-body channels. chans = macos.channels.rigid_body_channels(SESSION, RX_PATH) returns a cell array of channel handles (RigidBodyChannel or FocalPlaneChannel), one per (actual optic, DOF) pair.

channels.source_channels

```
macos.channels.chans = source_channels(session, opts)
```

MACOS.CHANNELS.SOURCE_CHANNELS Build SourceChannel instances. chans = macos.channels.source_channels(SESSION) returns a cell array of SourceChannel handles, one per DOF (Rx,Ry,Rz,Tx,Ty,Tz) in DOF order.

channels.surf_channels

```
macos.channels.chans = surf_channels(session, rx_path, opts)
```

MACOS.CHANNELS.SURF_CHANNELS Build Kr/Kc channels for powered optics. chans = macos.channels.surf_channels(SESSION, RX_PATH) returns a cell array of SurfChannel handles – one per (powered optic, param) pair, with param in {Kr, Kc} – for every POWERED optic (Reflector / Refractor with |Kr| << 1e22) in the loaded Rx. Channels are emitted

channels.zernike_channels

```
macos.channels.chans = zernike_channels(session, rx_path, opts)
```

MACOS.CHANNELS.ZERNIKE_CHANNELS Build Zern channels per Zern-typed element. chans = macos.channels.zernike_channels(SESSION, RX_PATH) returns a cell array of ZernikeCoefChannel handles, one per (Zern-typed element, Zern mode) pair declared in the Rx.

Appendix — CLI-suite coverage of the engine commands

Of the 128 engine commands, 128 are issued by at least one journal of the CLI regression suite (`cli_tests/cmd/*.jou`, run by `cli_tests/run_cli_tests.sh cmd` on both compilers); each entry of Part I names its journals. The 0 commands below have no journal yet — the Phase B worklist. A journal is one pty transcript with `# EXPECT:` / `# EXPECT-NOT:` assertions, so writing one is writing the entry's example and its test at once.

Appendix — binding coverage of the engine commands

Each Part I entry's *Binding:* line says how mmacos / pymacos reach it: **SMACOS command** (an api routine issues it through SMACOS, detected in `macos_api_mod.F90`), **direct setter** (an api routine under another name), **CLI-only** (session, shell, plot style, file modes), **CLI-only, legacy** (the internal linear model and the differential commands, superseded by the `dw_dx` route — Dave 2026-10-10) or **missing**. Generated with the binding generator (`MACOS_resources/mmacos/src/gen_mex_wrappers.py`, `binding_classes`). Counts: SMACOS command 30, direct setter 17, CLI-only 29, CLI-only, legacy 14, missing 38.

The **missing** commands, by the group that will carry them (PLAN_CONSOLIDATION 1.4b: single-ray tracing first, the Zernike / analysis set second, the image-plane set third):

- Zernike / analysis set (B.2, second): ELTS, MAP, ZABerr, ZCOef, SZCo
- field / beam set: FID , WLENS, SWL, GBS, AFOV , FOVS, SFOV
- the validator (validate_prescription.F90) has no api entry: VALidate
- through `compose_*` (B.2, image-plane set): MULtispec, CADD
- image-plane post-processing (B.2, third): NFIlt, RFIlt, SFIlt, ATMosphere, TPR, FSR, FDP, NOIse, SEEd, MTF, CMTF, BLUr, GBLur, GAIn, ROW
- source-adjustment mode: SAOpt
- beam coordinates at an element (the frames alone are `elt_csys_get`): COOrd
- all-element coordinates to xyzLocal.txt: ACOor
- chief-ray walk: BWK
- STOP's sibling (center the beam in the stop); no verified api twin: CENter
- no api setter; read by `modes_get` (a session mode – NOT the linear model, whatever its L): LNEg/NOLNeg
- a per-element resampling flag: REGrid/NOREGrid
- single-ray tracing (B.2, first): SINt
- builder: UDSinit
- line of sight (the harvests' `compute_los` reads it from spots): LOS
- plot pixel-array location: PLOcate
- lists the CALIB variables; `calib_set_var_elt` sets them, no getter: VARS
- builder; the Vis3d ray history is `ray_hist_set` / `ray_pos_hist_get`: JWST_v3d, Vis3d